

O LABORATÓRIO DE IA

DEEPSEEK HARNNESS: TORNE-SE UM ESPECIALISTA

◆ PARA INICIANTES

16 capítulos

projetos práticos

Heverton Eduardo Peres

Especialista em Marketing e Desenvolvimento de Soluções

8
CAPÍTULOS

70+
PÁGINAS

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Sumário

Prefácio	10
Sumário	11
Parte I — Fundamentos — As Primeiras Estações	11
Parte II — Domínio — As Estações Avançadas	11
Parte I — Fundamentos — As Primeiras Estações	12
Capítulo 1: O Ecossistema DeepSeek: Mapa do Laboratório	13
1. Introdução	13
2. Explica	13
O que é a DeepSeek	13
Arquitetura MoE: Muitos Expertos, Poucos Ativos	14
Multi-head Latent Attention (MLA)	15
Família de Modelos	15
O Ecossistema de Ferramentas	16
A Questão do Custo	17
3. Ilustra	17
O Laboratório e suas Estações	17
A Progressão do Laboratório	18
4. Técnica	18
Instalando o DeepSeek Harness	18
Configurando a API DeepSeek	19
Modelos Disponíveis na API	21
Thinking Mode e Reasoning Effort	21
Verificando o Ecossistema	22
5. Aplica	23
A Primeira Vez no Laboratório	23
Exercício	24
6. Conclusão	24
7. Referências Bibliográficas	25
Capítulo 2: DeepSeek API: Primeiros Contatos	27
1. Introdução	27
2. Explica	27
Autenticação e Endpoints	27
Modelos e Suas Forças	28
Thinking Mode: Raciocínio Profundo	28

Streaming	29
Compatibilidade com Ferramentas Externas	29
3. Ilustra	30
A Estação de Comunicação	30
Comparação de Modelos na Prática	31
4. Técnica	31
Exemplo Completo com Python	31
Exemplo com curl	33
Exemplo com Node.js	34
Tratamento de Erros	35
Monitoramento de Custos	36
5. Aplica	37
O Dia a Dia com a API	37
Métricas de Sucesso	38
Exercício	38
6. Conclusão	38
7. Referências Bibliográficas	39
Capítulo 3: Integração com Ferramentas Populares	41
1. Introdução	41
2. Explica	41
O Ecossistema de Coding Assistants	41
Claude Code	42
GitHub Copilot	42
Cline	43
OpenCode	43
DeepSeek-TUI	43
Comparação entre Ferramentas	44
3. Ilustra	44
A Estação de Conexão	44
4. Técnica	45
Configuração do Claude Code	45
Configuração do Cline no VS Code	45
Configuração do OpenCode	46
Script de Verificação de Integração	46
5. Aplica	47
O Problema da Integração Fragmentada	47
Exercício	48
6. Conclusão	49
7. Referências Bibliográficas	49
8. Apêndice — Integração MCP com DeepSeek	50
O que é MCP	50

Configurando MCP com DeepSeek	51
Benefícios do MCP com DeepSeek	51
Exemplo Prático: Code Review com MCP	52
9. Troubleshooting — Problemas Comuns e Soluções	52
Erro 401: Unauthorized	52
Erro 429: Rate Limit	53
Erro 400: Bad Request	53
Resposta truncada	53
Resposta em inglês quando pediu português	53
Thinking mode não funciona	53
A. Boas Práticas de Integração	54
Segurança	54
Resiliência	54
Monitoramento	55
Cache Inteligente	56
Capítulo 4: Prompt Engineering para DeepSeek	57
1. Introdução	57
2. Explica	57
A Estrutura de um Prompt Eficaz	57
System Prompts no DeepSeek	58
Thinking Mode e Prompt	58
Few-Shot Learning	59
Temperature e Top-P	59
Chain-of-Thought (CoT)	60
3. Ilustra	61
A Estação de Calibração	61
4. Técnica	61
Biblioteca de Prompts para DeepSeek	61
Prompt para Reasoning Chain	63
Otimização de Prompts por Tarefa	64
Padrões de Anti-Patterns	64
5. Aplica	65
O Prompt que Não Funciona	65
Métricas de Qualidade de Prompt	66
Exercício	66
6. Conclusão	67
7. Referências Bibliográficas	67
8. Apêndice — Técnicas Avançadas de Prompt	68
Self-Consistency	68
Tree of Thoughts	69
Prompt Chaining	69

9. Apêndice — Calibração e Eval de Prompts	70
Framework de Eval	70
A/B Testing de Prompts	72
A. Referência Rápida — Modelos e Parâmetros	73
Tabela de Modelos	73
Tabela de Parâmetros	73
Exemplos por Tarefa	73
Limites da API	74
Dicas de Otimização	74
Parte II — Domínio — As Estações Avançadas	75
Capítulo 5: Arquitetura do DeepSeek Harness	76
1. Introdução	76
2. Explica	76
Cordis: O Meta-Framework	76
A Arquitetura “Everything is a Plugin”	77
Web UI e CLI	77
Gerenciamento de Estado	77
Comunicação entre Plugins	78
3. Ilustra	78
A Central de Comando	78
Fluxo de uma Mensagem	79
4. Técnica	79
Estrutura de um Plugin	79
Plugin com Persistência	81
Plugin com Rate Limiting	81
Carregando Plugins	82
Integração com MCP	83
5. Aplica	84
O Problema da Monoliticidade	84
Exercício	85
6. Conclusão	85
7. Referências Bibliográficas	85
8. Apêndice — Hooks e Middleware no Harness	87
Sistema de Hooks	87
Middleware Pattern	88
9. Apêndice — Debugging de Plugins	88
Logging Estruturado	88
Métricas de Plugin	89
A. Padrões Avançados de Arquitetura	91
Plugin Composition	91
Event-Driven Architecture	91

State Machine Pattern	92
B. Padrão Observer para Monitoramento	93
Plugin de Métricas Completo	93
Capítulo 6: Desenvolvimento de Plugins para o Harness	95
1. Introdução	95
2. Explica	95
Estrutura de um Plugin	95
Ciclo de Vida de um Plugin	96
APIs Disponíveis	97
Publicação e Descoberta	97
Padrões de Design	97
3. Ilustra	98
A Estação Personalizada	98
4. Técnica	98
Criando um Plugin Completo	98
Testes	101
Publicando o Plugin	102
5. Aplica	103
O Plugin que Faltava	103
Exercício	104
6. Conclusão	105
7. Referências Bibliográficas	105
8. Apêndice — Testes Avançados de Plugins	107
Testes de Integração	107
Testes de Performance	108
Mock de Context	109
9. Apêndice — Publicação e Distribuição de Plugins	109
Preparando para Publicação	109
Documentação do Plugin	110
Configuração	111
Funcionalidades	111
Desenvolvimento	111
Licença	111
Capítulo 7: Otimização de Inferência: FlashMLA, DeepEP e DeepGEMM	113
1. Introdução	113
2. Explica	113
FlashMLA: Atenção Otimizada	113
DeepEP: Comunicação Expert-Parallel	114
DeepGEMM: Kernels BLAS	115
3FS: Sistema de Arquivos Distribuído	115
DeepSpec: Speculative Decoding	115

3. Ilustra	115
A Estação de Alta Performance	115
A Pirâmide de Performance	116
4. Técnica	116
Instalando FlashMLA	116
Usando FlashMLA para Decoding	117
Instalando DeepEP	117
Usando DeepEP para MoE	118
Benchmark de Performance	119
5. Aplica	120
O Gargalo que Ninguém Vê	120
Métricas de Performance	120
Exercício	120
6. Conclusão	121
7. Referências Bibliográficas	121
8. Apêndice — Comparativo de Hardware para Inferência	123
GPUs e Suporte	123
Configurações Recomendadas	123
Custos de Infraestrutura	123
9. Apêndice — Otimizações Avançadas de Inferência	124
Quantização	124
KV Cache Optimization	124
Speculative Decoding	125
Batching Eficiente	126
Capítulo 8: Deploy, Escalabilidade e Custo	127
1. Introdução	127
2. Explica	127
Três Caminhos para Produção	127
Frameworks de Inferência	128
Estratégias de Escalabilidade	128
Otimização de Custo	129
3. Ilustra	129
A Rota até a Produção	129
Comparativo de Custos	130
4. Técnica	131
Deploy com SGLang (Recomendado)	131
Deploy com vLLM	132
Docker Compose para Produção	133
Script de Monitoramento de Custo	134
Cache de Respostas	135
5. Aplica	137

A Decisão de Deploy	137
Checklist de Produção	137
Exercício	137
6. Conclusão	138
7. Referências Bibliográficas	138
8. Apêndice — Casos de Uso Reais	140
Caso 1: Chatbot de Atendimento ao Cliente	140
Caso 2: Code Review Automatizado	140
Caso 3: Geração de Documentação	141
Caso 4: Análise de Sentimento em Tempo Real	141
Conclusão Geral	142

Prefácio

Apresentar o ecossistema DeepSeek como um laboratório completo de IA, situando o leitor como um profissional que vai dominar cada módulo — dos modelos à API, do Harness aos kernels de baixo nível — até se tornar um especialista capaz de construir soluções completas.

Sumário

Parte I — Fundamentos — As Primeiras Estações

1. O Ecossistema DeepSeek: Mapa do Laboratório
2. DeepSeek API: Primeiros Contatos
3. Integração com Ferramentas Populares
4. Prompt Engineering para DeepSeek

Parte II — Domínio — As Estações Avançadas

5. Arquitetura do DeepSeek Harness
 6. Desenvolvimento de Plugins para o Harness
 7. Otimização de Inferência: FlashMLA, DeepEP e DeepGEMM
 8. Deploy, Escalabilidade e Custo
-

Parte I — Fundamentos — As Primeiras Estações

Capítulo 1: O Ecossistema DeepSeek: Mapa do Laboratório

1. Introdução

Imagine que você acabou de receber a chave de um laboratório de última geração. À sua frente, há dezenas de estações de trabalho — cada uma com uma ferramenta poderosa, cada uma projetada para resolver um tipo específico de problema. Algumas são familiares; outras parecem saídas de um futuro que ainda não chegou. O seu trabalho é explorar cada estação, entender o que ela faz e, ao final, ser capaz de orquestrar todas elas para construir algo extraordinário.

Esse laboratório é o ecossistema DeepSeek. Não se trata de uma única ferramenta, mas de um conjunto integrado de modelos de linguagem, APIs, frameworks de agentes e bibliotecas de otimização de baixo nível — tudo open-source, tudo projetado para funcionar em conjunto [1]. A DeepSeek AI, empresa por trás desse ecossistema, adota uma filosofia radicalmente aberta: seus principais modelos, bibliotecas e ferramentas são disponibilizados sob licença MIT, permitindo uso comercial, modificações e derivações sem restrições significativas [2].

Ao final deste capítulo, você terá o mapa completo do laboratório: saberá o que é cada peça, como se relacionam e qual delas usar em cada situação. Como Engenheiro de IA, esse mapa é a sua planta baixa antes de começar a construir. A progressão é intencional — começaremos pelos modelos (o coração do laboratório), passaremos pela API (a porta de entrada), conheceremos o Harness (o orquestrador) e finalizaremos com as bibliotecas de baixo nível (as otimizações que tornam tudo mais rápido).

2. Explica

O que é a DeepSeek

A DeepSeek AI é uma empresa de inteligência artificial focada em desenvolver modelos de linguagem de última geração com foco em eficiência e performance. Diferente de muitas

empresas do setor que tratam seus modelos como caixas-pretas proprietárias, a DeepSeek adota uma abordagem radicalmente aberta — seus modelos são disponibilizados com licença MIT, permitindo uso comercial, modificações e derivações [1].

O que diferencia a DeepSeek do concorrência não é apenas a abertura — é a eficiência. O modelo DeepSeek-V3, por exemplo, foi treinado com apenas 2.788 milhões de horas de GPU H800, uma fração do custo estimado de modelos equivalentes como GPT-4 [3]. Essa eficiência não é um acidente: é resultado de inovações arquiteturais que permitem obter mais performance com menos recursos. A empresa investe pesadamente em otimização de treinamento e inferência, e suas descobertas são compartilhadas com a comunidade através de papers acadêmicos e repositórios open-source.

O impacto dessa abordagem é profundo. Quando uma empresa de IA de ponta disponibiliza seus modelos e ferramentas sob licença MIT, ela reduz drasticamente a barreira de entrada para desenvolvedores e empresas que querem usar IA de alta qualidade. Você não precisa de milhões de dólares em infraestrutura para usar o DeepSeek-V3 — basta uma API key e um computador [2].

Arquitetura MoE: Muitos Expertos, Poucos Ativos

Para entender por que a DeepSeek é eficiente, você precisa compreender a arquitetura Mixture-of-Experts (MoE). Em modelos tradicionais como GPT-4 ou LLaMA, todos os parâmetros são ativados para cada token processado — se o modelo tem 70 bilhões de parâmetros, todos os 70 bilhões trabalham para cada palavra [4].

O MoE muda completamente essa equação. Em vez de um único modelo monolítico, o MoE mantém centenas de “especialistas” (experts), mas ativa apenas um subconjunto deles para cada token processado [5]. O DeepSeek-V3 possui 671 bilhões de parâmetros no total, mas ativa apenas 37 bilhões por token — apenas 5,5% do total [3]. É como ter uma equipe de 671 especialistas, mas para cada tarefa, contratar apenas 37 deles. O resultado: a potência de um modelo gigante com o custo de um modelo moderado.

O roteamento entre experts é controlado por uma rede de gating — um mecanismo que decide quais especialistas receberão cada token. O DeepSeek inovou ao introduzir uma estratégia de load balancing sem perda auxiliar (auxiliary-loss-free), que distribui a carga entre os experts sem degradar a performance do modelo [3]. Em modelos MoE anteriores, o balanceamento de carga era forçado através de uma penalidade que reduzia a qualidade das respostas. A abordagem da DeepSeek elimina essa penalidade, mantendo a qualidade enquanto distribui a carga de forma eficiente.

A implicação prática é significativa: você obtém a capacidade de um modelo com 671B parâmetros (qualidade de raciocínio, conhecimento amplo, multilinguismo) com o custo de inferência de um modelo com 37B parâmetros (latência baixa, custo reduzido, menor consumo de memória). Essa é a revolução silenciosa que torna a IA de ponta acessível [3].

Multi-head Latent Attention (MLA)

A segunda grande inovação arquitetural é a Multi-head Latent Attention (MLA). Para entender por que ela é importante, vamos primeiro revisar como a atenção funciona em modelos de linguagem.

Em modelos baseados em Transformer, a cada token processado, é necessário armazenar um vetor de chave (key) e valor (value) para cada cabeça de atenção — o chamado KV cache. Em modelos com contexto longo (128K tokens ou mais), isso consome uma quantidade enorme de memória. O DeepSeek-V3, com 128K de contexto, precisaria armazenar centenas de gigabytes de KV cache para processar sequências longas [6].

A MLA resolve isso comprimindo o KV cache através de uma projeção latente. Em vez de armazenar vetores completos de chave e valor, o modelo armazena uma representação comprimida em um espaço latente de menor dimensão e os reconstrói sob demanda. O resultado é uma redução drástica de memória com perda mínima de qualidade [6]. Essa técnica é a base do FlashMLA, que veremos no Capítulo 7 — um conjunto de kernels CUDA otimizados que implementam a MLA com performance de até 660 TFLOPS em GPUs H800 [7].

A MLA não é apenas uma otimização de memória — ela é o que permite ao DeepSeek-V3 suportar contexto de 128K tokens sem estourar a memória disponível. Sem a MLA, um modelo com 671B parâmetros e contexto de 128K simplesmente não caberia na maioria das configurações de hardware [3].

Família de Modelos

A DeepSeek mantém uma família de modelos para diferentes necessidades. Cada modelo foi projetado para um cenário específico, e entender as diferenças entre eles é crucial para escolher o certo para cada tarefa.

DeepSeek-V3 — O modelo principal para uso geral. 671B parâmetros no total, 37B ativados por token, contexto de 128K tokens. Treinado em 14.8 trilhões de tokens com FP8 mixed precision. É o modelo base de onde derivam os outros [3]. No benchmark MMLU (compreensão multitarefa), alcançou 88.5%, comparável ao GPT-4o (87.2%). Em programação (LiveCodeBench), alcançou 40.5%, superando o Claude-3.5-Sonnet (36.3%) [3].

DeepSeek-R1 — O modelo de raciocínio. Treinado via reforço em larga escala (RL) sem supervisão preliminar, o R1 desenvolveu capacidades espontâneas de auto-verificação e reflexão [8]. No benchmark AIME 2024 (matemática avançada), alcançou 79.8% de acerto, comparável ao OpenAI o1 (79.2%). No MATH-500, alcançou 97.3%, superando o o1 (96.4%) [8]. O R1 é treinado em cima do DeepSeek-V3-Base e demonstra que raciocínio profundo pode ser incentivado puramente através de reforço.

DeepSeek-V4 — A geração mais recente, com variantes V4-Flash (velocidade), V4-Pro (qualidade) e V4-Flash-Vision (entrada de imagens). Compatível com as APIs OpenAI

e Anthropic [9]. O V4-Pro suporta `thinking mode` e `reasoning_effort`, permitindo ao modelo raciocinar antes de responder — um recurso que veremos detalhadamente no Capítulo 2.

Modelos Distilados — Seis variações menores (1.5B a 70B parâmetros) derivadas do R1, treinadas com dados de raciocínio gerados pelo modelo maior. O DeepSeek-R1-Distill-Qwen-32B supera o o1-mini em vários benchmarks, demonstrando que modelos menores podem ser poderosos quando treinados com dados de qualidade [8]. Esses modelos distilados são ideais para deploy local quando hardware limitado é um fator.

O Ecossistema de Ferramentas

Além dos modelos, a DeepSeek mantém um ecossistema de ferramentas open-source que cobrem toda a pipeline de IA — do desenvolvimento de agentes à otimização de inferência em GPU.

DeepSeek Harness — Framework de agentes com arquitetura de plugins, powered pelo Cordis. Permite construir agentes complexos com memória, ferramentas e integrações externas [1]. Veremos detalhadamente nos Capítulos 5 e 6.

FlashMLA — Kernels otimizados de atenção para inferência ultrarrápida. Implementa a MLA com suporte a dense e sparse attention, FP8 KV cache, e até 660 TFLOPS em H800 [7]. Veremos no Capítulo 7.

DeepEP — Biblioteca de comunicação expert-parallel para treinamento distribuído. Na versão V2, introduziu o ElasticBuffer e reduziu uso de SM de 24 para 4-6 [10]. Veremos no Capítulo 7.

DeepGEMM — Biblioteca de kernels BLAS eficientes para GPU, com compilação JIT [11]. Veremos no Capítulo 7.

3FS — Sistema de arquivos distribuído de alta performance para treinamento e inferência de IA, resolvendo gargalos de I/O [12].

DeepSpec — Framework para treinar e avaliar algoritmos de speculative decoding [13].

TileKernels — Biblioteca de kernels escrita em tilelang [14].

DeepSeek-OCR-2 — Modelo de reconhecimento óptico com “Visual Causal Flow” [15].

Cada peça deste ecossistema se conecta: os modelos usam FlashMLA para inferência, o Harness orquestra agentes que chamam os modelos, e o DeepEP distribui o treinamento em cluster. Essa integração é o que torna o ecossistema DeepSeek mais do que a soma de suas partes [1].

A Questão do Custo

Uma das razões pelas quais a DeepSeek se tornou tão relevante é a relação custo-benefício. Enquanto modelos como GPT-4 estimam custos de treinamento na casa dos bilhões de dólares, o DeepSeek-V3 foi treinado com apenas 2.788 milhões de horas de GPU H800 [3]. Isso não é uma economia marginal — é uma redução de ordens de magnitude.

Para o desenvolvedor final, isso se traduz em custos de API significativamente menores. O DeepSeek-V4-Flash custa \$0.07 por milhão de tokens de entrada e \$0.27 por milhão de tokens de saída [9]. Comparado com o GPT-4o (\$2.50/\$10.00), isso representa uma redução de ~97% no custo de entrada e ~97% no custo de saída. Para uma aplicação que processa 1 milhão de tokens por mês, a economia pode ser de centenas de dólares [9].

3. Ilustra

O Laboratório e suas Estações

Pense no ecossistema DeepSeek como um laboratório de pesquisa de alta tecnologia. Cada estação de trabalho é uma ferramenta especializada, e a eficiência do laboratório depende de como você combina essas estações.

```
%% legenda: Mapa do laboratório DeepSeek – como as peças se conectam
flowchart TD
    A[DeepSeek Harness<br/>Orquestrador de Agentes] --> B[Modelos<br/>V3 / R1 / V4]
    A --> C[FlashMLA<br/>Atenção Otimizada]
    A --> D[DeepEP<br/>Comunicação Multi-GPU]
    B --> E[API DeepSeek<br/>OpenAI/Anthropic Compatível]
    B --> F[3FS<br/>Arquivo Distribuído]
    D --> G[DeepGEMM<br/>Kernels BLAS]
    E --> H[Ferramentas Externas<br/>Claude Code / Copilot / Cline]
    B --> I[Modelos Distilados<br/>1.5B a 70B]
    I --> J[Deploy Local<br/>SGLang / vLLM]
```

O DeepSeek Harness é a estação de controle central — ele orquestra tudo ao redor. Os modelos (V3, R1, V4) são os processadores principais. FlashMLA e DeepGEMM são as otimizações de baixo nível que tornam tudo mais rápido. DeepEP cuida da comunicação quando você precisa de múltiplas GPUs. E a API DeepSeek é a porta de entrada para quem quer usar tudo isso sem configurar infraestrutura [1].

A Progressão do Laboratório

Ao longo deste livro, você vai visitar cada estação progressivamente, construindo conhecimento camada por camada:

Parte	Estações	O que você domina	Resultado
I — Fundamentos	Modelos, API, Integrações, Prompts	Uso prático das ferramentas	Você consegue usar DeepSeek em qualquer ferramenta
II — Domínio	Harness, Plugins, Inferência, Deploy	Construção e otimização	Você consegue construir soluções completas

A Parte I é como um tour guiado pelas estações básicas — você aprende a ligar cada equipamento e a usá-lo para tarefas reais. A Parte II é quando você abre as máquinas, entende como funcionam por dentro e começa a construir suas próprias soluções. Essa progressão não é arbitrária — cada capítulo da Parte II depende do conhecimento adquirido na Parte I.

4. Técnica

Instalando o DeepSeek Harness

O DeepSeek Harness é a porta de entrada para o ecossistema de agentes. Ele roda localmente e fornece uma Web UI para interagir com os modelos DeepSeek. A instalação é direta — o Harness é distribuído como um pacote npm que pode ser executado sem instalação permanente.

Instalação via npm (método mais simples):

```
# Certifique-se de que o Node.js está instalado (v18+)
node --version

# Instale e inicie o Harness
npx @deepseek-ai/dsh web
```

O comando inicia a Web UI em `http://127.0.0.1:3080` e abre automaticamente no navegador. Para executar sem abrir o navegador (útil em servidores):

```
npx @deepseek-ai/dsh web --no-open
```

Instalação a partir do código-fonte (para desenvolvimento):

```
git clone https://github.com/deepseek-ai/deepseek-harness.git
cd deepseek-harness
pnpm install
pnpm run build
pnpm dsh web
```

O comando `pnpm run build` prepara os artefatos. O `pnpm dsh web` usa esses artefatos sem rebuildar [1]. A instalação a partir da fonte é recomendada se você pretende desenvolver plugins ou contribuir com o projeto.

Verificando a instalação:

```
# Verifique se o Harness está rodando
curl -s http://127.0.0.1:3080/api/health

# Se retornar {"status":"ok"}, o Harness está funcionando
```

Configurando a API DeepSeek

Para usar os modelos via API, você precisa de uma API key obtida em platform.deepseek.com. A API DeepSeek é compatível com o formato OpenAI, o que significa que você pode usar o SDK do OpenAI com apenas uma mudança de URL [9]:

Usando Python com o SDK OpenAI:

```
import os
from openai import OpenAI

# Configura o cliente apontando para a API DeepSeek
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

# Faz uma chamada simples
response = client.chat.completions.create(
    model="deepseek-v4-pro",
    messages=[
        {"role": "system", "content": "Você é um assistente útil."},
        {"role": "user", "content": "Explique o que é MoE em 3 frases."}
    ],
    stream=False,
    reasoning_effort="high",
    extra_body={"thinking": {"type": "enabled"}}
)
```

```
)  
  
print(response.choices[0].message.content)
```

Usando curl:

```
curl https://api.deepseek.com/chat/completions \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer ${DEEPSEEK_API_KEY}" \  
-d '{  
  "model": "deepseek-v4-pro",  
  "messages": [  
    {"role": "system", "content": "Você é um assistente útil."},  
    {"role": "user", "content": "Explique o que é MoE em 3 frases."}  
  ],  
  "thinking": {"type": "enabled"},  
  "reasoning_effort": "high",  
  "stream": false  
}'
```

Usando Node.js:

```
import OpenAI from "openai";  
  
const openai = new OpenAI({  
  baseURL: "https://api.deepseek.com",  
  apiKey: process.env.DEEPSEEK_API_KEY,  
});  
  
async function main() {  
  const completion = await openai.chat.completions.create({  
    messages: [  
      { role: "system", content: "Você é um assistente útil." },  
      { role: "user", content: "Explique o que é MoE em 3 frases." }  
    ],  
    model: "deepseek-v4-pro",  
    thinking: { type: "enabled" },  
    reasoning_effort: "high",  
    stream: false,  
  });  
  console.log(completion.choices[0].message.content);  
}  
main();
```

Modelos Disponíveis na API

A API oferece três modelos principais, cada um otimizado para um cenário diferente [9]:

Modelo	Uso Recomendado	Diferencial	Custo (input/output)
deepseek-v4-flash	Tarefas de alta velocidade	Latência mínima	\$0.07/\$0.27 por 1M tokens
deepseek-v4-pro	Tarefas que exigem qualidade	Raciocínio profundo	\$0.27/\$1.10 por 1M tokens
deepseek-v4-flash-vision-exp	Entrada de imagens	Suporte a imagens + texto	Similar ao flash

Para a maioria das integrações com ferramentas de coding, `deepseek-v4-flash` é suficiente. Use `deepseek-v4-pro` quando precisar de raciocínio complexo (resolução de problemas, arquitetura de software, debug). O `v4-flash-vision-exp` é experimental e aceita imagens além de texto [9].

Thinking Mode e Reasoning Effort

O DeepSeek-V4 suporta um modo de “pensamento” que permite ao modelo raciocinar antes de responder. Isso é ativado pelo parâmetro `thinking` e controlado por `reasoning_effort` [9]:

```
# Sem thinking (resposta direta, mais rápida)
response = client.chat.completions.create(
    model="deepseek-v4-flash",
    messages=[{"role": "user", "content": "Qual a capital da França?"}],
)

# Com thinking (raciocínio profundo, mais lento mas mais preciso)
response = client.chat.completions.create(
    model="deepseek-v4-pro",
    messages=[{"role": "user", "content": "Resolva: se  $f(x) = 3x^2 + 2x - 5$ , encontre  $f'(2)$ " }],
    extra_body={"thinking": {"type": "enabled"}},
    reasoning_effort="high"
)
```

O `reasoning_effort` aceita três valores: `low`, `medium` e `high`. Para tarefas simples, `low` é suficiente. Para problemas matemáticos ou de programação complexos, use `high`. A diferença

é significativa: com `high`, o modelo pode gastar vários segundos “pensando” antes de gerar a resposta, mas a qualidade da resposta melhora drasticamente para problemas complexos [9].

Verificando o Ecossistema

Para ter certeza de que tudo está configurado corretamente, execute este script de verificação:

```
import os
import requests

def verificar_ecossistema():
    """Verifica se todos os componentes do ecossistema estão acessíveis."""
    resultados = {}

    # 1. Verificar API DeepSeek
    try:
        r = requests.get(
            "https://api.deepseek.com/models",
            headers={"Authorization": f"Bearer {os.environ.get('DEEPSEEK_API_KEY')}"},
            timeout=10
        )
        resultados["API DeepSeek"] = "✅" if r.status_code == 200 else f"❌ {r.status_code}"
    except Exception as e:
        resultados["API DeepSeek"] = f"❌ {e}"

    # 2. Verificar Harness (se rodando)
    try:
        r = requests.get("http://127.0.0.1:3080/api/health", timeout=5)
        resultados["Harness"] = "✅" if r.status_code == 200 else f"❌ {r.status_code}"
    except:
        resultados["Harness"] = "⚠ Não está rodando (execute npx @deepseek-ai/dsh web)"

    # 3. Verificar Node.js
    import subprocess
    try:
        v = subprocess.check_output(["node", "--version"],
text=True).strip()
        resultados["Node.js"] = f"✅ {v}"
    except:
        resultados["Node.js"] = "❌ Não encontrado"

    # 4. Verificar Python
```

```

resultados["Python"] = f"✅ {os.sys.version.split()[0]}"

for componente, status in resultados.items():
    print(f" {componente}: {status}")

if __name__ == "__main__":
    print("=== Verificação do Ecosistema DeepSeek ===")
    verificar_ecossistema()

```

5. Aplica

A Primeira Vez no Laboratório

Você acabou de receber acesso ao ecossistema DeepSeek. Seu gerente pediu para avaliar se a IA pode acelerar o time de desenvolvimento. Você abre o terminal, instala o Harness com `npx @deepseek-ai/dsh web` e vê a Web UI aparecer. Tudo parece promissor — até você perceber que não sabe qual modelo escolher, nem como configurar a API.

O erro mais comum aqui é começar pelo modelo “mais forte” (`deepseek-v4-pro`) para todas as tarefas. Isso é como usar um microscópio eletrônico para ler um livro — funciona, mas é desperdício brutal de custo e tempo. A prática correta é mapear cada tipo de tarefa ao modelo adequado:

Tipo de Tarefa	Modelo Recomendado	reasoning_effort	Custo estimado
Pergunta simples	v4-flash	—	\$0.00001
Geração de código	v4-pro	medium	\$0.0001
Debug complexo	v4-pro	high	\$0.0003
Revisão de PR	v4-flash	low	\$0.00002
Análise de arquitetura	v4-pro	high	\$0.0003
Chat rápido	v4-flash	—	\$0.00001
Classificação	v4-flash	—	\$0.000005
Resumo de documento	v4-flash	—	\$0.00002

Outro erro frequente é esquecer de configurar a variável de ambiente `DEEPSEEK_API_KEY`. Sem ela, todas as chamadas retornam 401 (Unauthorized). O setup correto:

```
# Adicione ao seu .env ou .bashrc
export DEEPSEEK_API_KEY="sua-chave-aqui"

# Verifique se está funcionando
curl -s https://api.deepseek.com/models \
-H "Authorization: Bearer ${DEEPSEEK_API_KEY}" | python -m json.tool
```

Se a resposta listar os modelos disponíveis (`deepseek-v4-flash` , `deepseek-v4-pro` , etc.), a chave está configurada corretamente.

Exercício

- ☐ Instale o DeepSeek Harness via `npx @deepseek-ai/dsh web` e acesse a Web UI
- ☐ Obtenha uma API key em `platform.deepseek.com`
- ☐ Configure a variável de ambiente `DEEPSEEK_API_KEY`
- ☐ Faça uma chamada à API usando o SDK OpenAI em Python
- ☐ Teste os três modelos (flash, pro, vision) com a mesma pergunta e compare as respostas
- ☐ Experimente ativar e desativar o thinking mode e observe a diferença
- ☐ Execute o script de verificação do ecossistema e confirme que todos os componentes estão OK

6. Conclusão

Neste capítulo, você traçou o mapa completo do ecossistema DeepSeek. Viu que não é uma ferramenta isolada, mas um laboratório integrado: modelos MoE eficientes (V3, R1, V4) que ativam apenas 5,5% dos parâmetros por token, uma API compatível com OpenAI, o Harness para orquestração de agentes com arquitetura de plugins, e bibliotecas de baixo nível (FlashMLA, DeepEP, DeepGEMM) que tornam tudo mais rápido. A chave para usar esse laboratório não é dominar tudo de uma vez — é saber qual estação visitar para cada tarefa.

A DeepSeek se destaca pela combinação de abertura (licença MIT), eficiência (treinamento com 2.788M GPU hours) e qualidade (88.5% no MMLU, comparável ao GPT-4o). Essa combinação é rara no mercado e torna o DeepSeek uma opção séria para desenvolvedores e empresas que querem IA de ponta sem o custo de ponta [1].

No próximo capítulo, você vai tirar a primeira estação do modo demonstração e começar a usá-la de verdade: configurar a API, fazer chamadas reais e explorar os recursos avançados como streaming e thinking mode.

7. Referências Bibliográficas

- [1] DEEPSEEK-AI. *DeepSeek Harness: Everything is a Plugin*. Disponível em: <https://github.com/deepseek-ai/deepseek-harness>. Acesso em: 21 ago. 2026.
- [2] DEEPSEEK-AI. *DeepSeek Platform — Licensing*. Disponível em: <https://platform.deepseek.com/>. Acesso em: 21 ago. 2026.
- [3] DEEPSEEK-AI. *DeepSeek-V3 Technical Report*. Disponível em: <https://arxiv.org/abs/2412.19437>. Acesso em: 21 ago. 2026.
- [4] VASWANI, A. et al. *Attention Is All You Need*. In: Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [5] FEDUS, W. et al. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. In: Journal of Machine Learning Research, 2022.
- [6] AINSLEE, J. et al. *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. In: Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.
- [7] LI, Jiashi; LIU, Shengyu. *FlashMLA: Efficient Multi-head Latent Attention Kernels*. Disponível em: <https://github.com/deepseek-ai/FlashMLA>. Acesso em: 21 ago. 2026.
- [8] DEEPSEEK-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Disponível em: <https://arxiv.org/abs/2501.12948>. Acesso em: 21 ago. 2026.
- [9] DEEPSEEK-AI. *DeepSeek API Documentation*. Disponível em: <https://api-docs.deepseek.com/>. Acesso em: 21 ago. 2026.
- [10] DEEPSEEK-AI. *DeepEP: An Efficient Expert-Parallel Communication Library*. Disponível em: <https://github.com/deepseek-ai/DeepEP>. Acesso em: 21 ago. 2026.
- [11] DEEPSEEK-AI. *DeepGEMM: Clean and Efficient BLAS Kernel Library on GPU*. Disponível em: <https://github.com/deepseek-ai/DeepGEMM>. Acesso em: 21 ago. 2026.
- [12] DEEPSEEK-AI. *3FS: A High-Performance Distributed File System for AI Training and Inference Workloads*. Disponível em: <https://github.com/deepseek-ai/3FS>. Acesso em: 21 ago. 2026.
- [13] DEEPSEEK-AI. *DeepSpec: Full-Stack Codebase for Training and Evaluating Speculative Decoding Algorithms*. Disponível em: <https://github.com/deepseek-ai/DeepSpec>. Acesso em: 21 ago. 2026.
- [14] DEEPSEEK-AI. *TileKernels: Kernel Library Written in Tilelang*. Disponível em: <https://github.com/deepseek-ai/TileKernels>. Acesso em: 21 ago. 2026.
- [15] DEEPSEEK-AI. *DeepSeek-OCR-2: Visual Causal Flow*. Disponível em: <https://github.com/deepseek-ai/DeepSeek-OCR-2>. Acesso em: 21 ago. 2026.

- [16] CORDIVERSE. *Cordis: A Meta-Framework of Spatiotemporal Composability*. Disponível em: <https://github.com/cordiverse/cordis>. Acesso em: 21 ago. 2026.
- [17] DEEPSEEK-AI. *Awesome DeepSeek Agent*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-agent>. Acesso em: 21 ago. 2026.
- [18] DEEPSEEK-AI. *Awesome DeepSeek Integration*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-integration>. Acesso em: 21 ago. 2026.
- [19] DEEPSEEK-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. Disponível em: <https://arxiv.org/abs/2405.04434>. Acesso em: 21 ago. 2026.
- [20] SHAZEER, N. et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. In: International Conference on Learning Representations (ICLR), 2017.

Capítulo 2: DeepSeek API: Primeiros Contatos

1. Introdução

No capítulo anterior, você traçou o mapa do laboratório DeepSeek — viu cada peça, entendeu como se conectam e sabe qual usar para cada situação. Agora é hora de ligar a primeira estação de trabalho. A API DeepSeek é a porta de entrada para tudo: é ela que permite que suas aplicações, ferramentas e agentes conversem com os modelos de linguagem da DeepSeek.

A boa notícia é que a API é compatível com o formato OpenAI. Se você já usou o SDK do OpenAI em Python ou Node.js, está praticamente pronto — basta trocar a URL base [1]. Mas há nuances importantes que separam uma integração medíocre de uma excepcional: o thinking mode muda a dinâmica dos prompts, o streaming melhora drasticamente a percepção de velocidade, e a escolha do modelo certo para cada tarefa pode reduzir custos em 90% sem perder qualidade. Neste capítulo, você vai dominar cada um desses recursos e sair com um toolkit completo de integração.

2. Explica

Autenticação e Endpoints

A API DeepSeek usa autenticação via Bearer token — o mesmo mecanismo usado pela maioria das APIs modernas. Você precisa de uma API key obtida em platform.deepseek.com. A autenticação é incluída no cabeçalho `Authorization` de cada requisição [1]:

```
Authorization: Bearer sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Existem dois endpoints principais, dependendo do formato de API que sua ferramenta espera [1]:

Formato	Base URL	Compatível com
OpenAI	<code>https://api.deepseek.com</code>	SDK OpenAI, LangChain, CrewAI, tools que usam OpenAI
Anthropic	<code>https://api.deepseek.com/anthropic</code>	SDK Anthropic, Claude Code, tools que usam Anthropic

A escolha do endpoint depende da ferramenta que você está integrando. A maioria dos coding assistants (Claude Code, Cline, OpenCode) usa o formato Anthropic. Ferramentas como LangChain, CrewAI e AutoGPT usam o formato OpenAI [1]. Misturar os formatos resultará em erros 400 ou 422 que são difíceis de debugar — sempre verifique qual formato sua ferramenta espera antes de configurar.

Modelos e Suas Forças

A DeepSeek oferece três modelos na API, cada um otimizado para um cenário diferente [1]:

deepseek-v4-flash — O modelo de alta velocidade. Latência mínima, custo reduzido (\$0.07/\$0.27 por 1M tokens input/output). Ideal para tarefas que exigem respostas rápidas: chat, resumos, traduções, classificações. Atualizado para V4-Flash-0731. O flash é o modelo recomendado para a maioria das integrações — ele oferece 90% da qualidade do pro a 25% do custo [1].

deepseek-v4-pro — O modelo de alta qualidade (\$0.27/\$1.10 por 1M tokens). Suporta thinking mode e reasoning_effort, permitindo raciocínio profundo. Ideal para programação complexa, resolução de problemas, análise de arquitetura, debug de código legado. Atualizado para V4-Pro-0813. O pro é necessário quando a qualidade da resposta é mais importante que a velocidade [1].

deepseek-v4-flash-vision-exp — Modelo experimental que aceita entrada de imagens além de texto. Útil para análise de screenshots, diagramas, interfaces, código-fonte em imagem. O vision é experimental e pode ter comportamento imprevisível em alguns casos — use com cautela em produção [1].

A regra prática é: comece sempre pelo flash. Mude para o pro apenas quando o flash não entregar qualidade suficiente. Essa estratégia pode reduzir seus custos de API em 70-80% [1].

Thinking Mode: Raciocínio Profundo

O thinking mode é um recurso exclusivo dos modelos V4 que permite ao modelo “pensar” antes de responder. Quando ativado, o modelo gera uma cadeia de raciocínio interna — uma sequência de passos lógicos que ele percorre antes de produzir a resposta final [1].

Isso é especialmente útil para problemas que requerem múltiplos passos lógicos: resolução de equações, debug de código complexo, análise de arquitetura, comparação de alternativas. Sem thinking mode, o modelo gera a resposta diretamente. Com thinking mode, ele “pensa” primeiro e depois responde — como um programador que desenha o fluxo antes de codificar [1].

```
{
  "thinking": {"type": "enabled"},
  "reasoning_effort": "high"
}
```

O parâmetro `reasoning_effort` controla a profundidade do raciocínio: - `low` — Raciocínio superficial, rápido. Para perguntas diretas e tarefas simples. - `medium` — Raciocínio moderado. Para a maioria das tarefas de código. - `high` — Raciocínio profundo. Para problemas complexos que exigem análise múltipla.

A diferença é significativa: com `high`, o modelo pode gastar vários segundos “pensando” antes de gerar a resposta. Para um problema de debug complexo, isso pode significar a diferença entre uma resposta genérica e uma resposta que identifica a causa raiz do bug [1].

Streaming

O streaming permite receber a resposta token a token, em vez de esperar a resposta completa. Isso melhora drasticamente a percepção de velocidade para o usuário final — em vez de ver uma espera de 5 segundos seguida de uma resposta completa, o usuário vê a resposta sendo construída em tempo real [1].

Para ativar o streaming, basta definir `stream: true` na requisição. A resposta vem como uma sequência de eventos Server-Sent Events (SSE), cada um contendo um ou mais tokens. O cliente pode processar cada token conforme ele chega, atualizando a interface do usuário em tempo real [1].

O streaming é especialmente importante para aplicações de chat, onde a experiência do usuário depende da velocidade percebida. Mesmo que a latência total seja a mesma, o streaming faz com que a resposta comece a aparecer imediatamente — o usuário vê os primeiros tokens em menos de 100ms [1].

Compatibilidade com Ferramentas Externas

Um dos maiores diferenciais da API DeepSeek é sua compatibilidade com dezenas de ferramentas populares. De acordo com o repositório `awesome-deepseek-agent`, mais de 20 coding assistants suportam DeepSeek como backend [2]. Isso significa que você pode usar DeepSeek no Claude Code, GitHub Copilot, Cline, OpenCode, Codex e muitos outros — sem escrever código de integração.

A razão dessa compatibilidade universal é a aderência aos formatos de API padronizados. A API DeepSeek segue exatamente o formato de chat completions do OpenAI, incluindo suporte a system prompts, function calling, JSON mode e streaming [1]. Qualquer ferramenta que funcione com o formato OpenAI pode, automaticamente, funcionar com DeepSeek — basta trocar a URL base e a chave de API.

3. Ilustra

A Estação de Comunicação

Se o ecossistema DeepSeek é um laboratório, a API é a estação de comunicação — o ponto onde você conecta suas próprias ferramentas ao laboratório. Pense nela como um telefone bidirecional avançado: você envia uma pergunta (request) e recebe uma resposta (response). Mas esse telefone tem recursos que um telefone comum não tem — pode transmitir em tempo real (streaming), pode “pensar” antes de responder (thinking mode), e pode se conectar a diferentes tipos de aparelhos (formatos OpenAI e Anthropic) [1].

```
%% legenda: Fluxo de uma chamada à API DeepSeek com thinking mode
sequenceDiagram
    participant C as Cliente
    participant A as API DeepSeek
    participant M as Modelo (V4-Pro)
    C->>A: POST /chat/completions
    A->>M: Envia mensagens
    alt Thinking Mode Ativado
        M->>M: Raciocínio interno (passos lógicos)
        M->>A: Resposta + metadados de raciocínio
    else Sem Thinking
        M->>A: Resposta direta
    end
    alt Streaming
        A-->>C: Token 1
        A-->>C: Token 2
        A-->>C: ...
        A-->>C: Token N (fim)
    else Resposta Completa
        A->>C: JSON com resposta completa
    end
end
```

A mágica acontece quando você combina streaming com thinking mode: o cliente recebe os tokens de raciocínio em tempo real, permitindo mostrar ao usuário “pensando...” enquanto a resposta é construída. Isso cria uma experiência fluida e transparente [1].

Comparação de Modelos na Prática

Para ilustrar as diferenças entre os modelos, vamos fazer a mesma pergunta para os três:

```
pergunta = "Implemente uma fila de prioridade em Python com suporte a  
atualização de prioridade"  
  
# Flash: resposta rápida, código funcional  
flash = chat_simples(cliente, pergunta, "deepseek-v4-flash")  
# Tempo: ~1.5s | Custo: ~$0.0001  
  
# Pro sem thinking: resposta completa, sem raciocínio visível  
pro_direto = chat_com_raciocinio(cliente, pergunta, "low")  
# Tempo: ~3s | Custo: ~$0.0003  
  
# Pro com thinking high: raciocínio profundo + código otimizado  
pro_pensando = chat_com_raciocinio(cliente, pergunta, "high")  
# Tempo: ~8s | Custo: ~0.001
```

Para uma fila de prioridade simples, o flash é suficiente. Para uma implementação que precise de $O(\log n)$ para update, o pro com thinking high vai gerar uma solução mais sofisticada.

4. Técnica

Exemplo Completo com Python

Vamos construir um cliente completo que explora todos os recursos da API:

```
import os  
import time  
from openai import OpenAI  
  
class DeepSeekClient:  
    """Cliente completo para a API DeepSeek."""  
  
    def __init__(self):
```

```

self.cliente = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

def chat_simples(self, pergunta, modelo="deepseek-v4-flash"):
    """Chat simples sem thinking mode."""
    inicio = time.time()
    resposta = self.cliente.chat.completions.create(
        model=modelo,
        messages=[
            {"role": "system", "content": "Você é um assistente de IA especializado em DeepSeek."},
            {"role": "user", "content": pergunta}
        ],
        stream=False
    )
    latencia = time.time() - inicio
    tokens = len(resposta.choices[0].message.content.split())
    return {
        "resposta": resposta.choices[0].message.content,
        "latencia": latencia,
        "tokens": tokens,
        "modelo": modelo
    }

def chat_com_raciocinio(self, pergunta, esforco="high"):
    """Chat com thinking mode para problemas complexos."""
    inicio = time.time()
    resposta = self.cliente.chat.completions.create(
        model="deepseek-v4-pro",
        messages=[{"role": "user", "content": pergunta}],
        stream=False,
        reasoning_effort=esforco,
        extra_body={"thinking": {"type": "enabled"}}
    )
    latencia = time.time() - inicio
    return {
        "resposta": resposta.choices[0].message.content,
        "latencia": latencia,
        "modelo": "deepseek-v4-pro",
        "esforco": esforco
    }

def chat_streaming(self, pergunta, modelo="deepseek-v4-flash"):
    """Chat com streaming – recebe token a token."""
    stream = self.cliente.chat.completions.create(

```



```

        model=modelo,
        messages=[{"role": "user", "content": pergunta}],
        stream=True
    )
    resposta_completa = []
    for chunk in stream:
        if chunk.choices[0].delta.content:
            token = chunk.choices[0].delta.content
            resposta_completa.append(token)
            print(token, end="", flush=True)
    print()
    return "".join(resposta_completa)

def comparar_modelos(self, pergunta):
    """Compara respostas de todos os modelos."""
    print(f"Pergunta: {pergunta}\n")

    # Flash
    r1 = self.chat_simples(pergunta, "deepseek-v4-flash")
    print(f"Flash ({r1['latencia']:.1f}s): {r1['resposta'][:200]}...
\n")

    # Pro sem thinking
    r2 = self.chat_com_raciocinio(pergunta, "low")
    print(f"Pro low ({r2['latencia']:.1f}s): {r2['resposta'][:200]}...
\n")

    # Pro com thinking high
    r3 = self.chat_com_raciocinio(pergunta, "high")
    print(f"Pro high ({r3['latencia']:.1f}s): {r3['resposta'][:200]}...
\n")

# Uso
cliente = DeepSeekClient()
cliente.comparar_modelos("O que é MoE e por que é eficiente?")

```

Exemplo com curl

```

# Chat simples
curl -s https://api.deepseek.com/chat/completions \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ${DEEPSEEK_API_KEY}" \
-d '{
  "model": "deepseek-v4-flash",
  "messages": [{"role": "user", "content": "Liste 3 vantagens do MoE"}],

```

```

    "stream": false
  }' | python -m json.tool

# Com thinking mode
curl -s https://api.deepseek.com/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ${DEEPSEEK_API_KEY}" \
  -d '{
    "model": "deepseek-v4-pro",
    "messages": [{"role": "user", "content": "Projete um sistema de cache
LRU em Python com suporte a TTL"}],
    "thinking": {"type": "enabled"},
    "reasoning_effort": "high",
    "stream": false
  }' | python -m json.tool

# Com streaming
curl -s https://api.deepseek.com/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ${DEEPSEEK_API_KEY}" \
  -d '{
    "model": "deepseek-v4-flash",
    "messages": [{"role": "user", "content": "Resuma DeepSeek em 5 bullet
points"}],
    "stream": true
  }'

```

Exemplo com Node.js

```

import OpenAI from "openai";

const cliente = new OpenAI({
  baseURL: "https://api.deepseek.com",
  apiKey: process.env.DEEPSEEK_API_KEY,
});

// Chat simples
async function chatSimples(pergunta) {
  const inicio = Date.now();
  const resposta = await cliente.chat.completions.create({
    model: "deepseek-v4-flash",
    messages: [{ role: "user", content: pergunta }],
    stream: false,
  });
  const latencia = Date.now() - inicio;
}

```

```

    return { resposta: resposta.choices[0].message.content, latencia };
}

// Com thinking mode
async function chatRaciocinio(pergunta) {
    const resposta = await cliente.chat.completions.create({
        model: "deepseek-v4-pro",
        messages: [{ role: "user", content: pergunta }],
        stream: false,
        reasoning_effort: "high",
        extra_body: { thinking: { type: "enabled" } },
    });
    return resposta.choices[0].message.content;
}

// Streaming
async function chatStreaming(pergunta) {
    const stream = await cliente.chat.completions.create({
        model: "deepseek-v4-flash",
        messages: [{ role: "user", content: pergunta }],
        stream: true,
    });
    for await (const chunk of stream) {
        process.stdout.write(chunk.choices[0]?.delta?.content || "");
    }
    console.log();
}

// Uso
const r1 = await chatSimples("O que é MLA?");
console.log(`Flash: ${r1.resposta} (${r1.latencia}ms)`);

const r2 = await chatRaciocinio("Implemente um B+ tree em TypeScript");
console.log(`Pro: ${r2}`);

await chatStreaming("Resuma DeepSeek em 5 bullet points");

```

Tratamento de Erros

```

from openai import BadRequestError, AuthenticationError, RateLimitError,
APIError

def chamada_segura(cliente, pergunta):
    """Chamada com tratamento de erros comuns."""
    try:

```

```

        return cliente.chat_simples(pergunta)
    except AuthenticationError:
        print("❌ API key inválida ou ausente. Verifique DEEPSEEK_API_KEY.")
        return None
    except RateLimitError:
        print("⌚ Rate limit atingido. Aguarde 60s e tente novamente.")
        return None
    except BadRequestError as e:
        print(f"❌ Requisição inválida: {e}")
        return None
    except APIError as e:
        print(f"❌ Erro na API: {e.status_code} - {e.message}")
        return None
    except Exception as e:
        print(f"❌ Erro inesperado: {e}")
        return None

```

Monitoramento de Custos

```

class MonitorCustos:
    """Monitora custos de uso da API DeepSeek."""

    PRECOS = {
        "deepseek-v4-flash": {"input": 0.07, "output": 0.27},
        "deepseek-v4-pro": {"input": 0.27, "output": 1.10},
    }

    def __init__(self):
        self.historico = []

    def registrar(self, modelo, tokens_input, tokens_saida):
        preco = self.PRECOS.get(modelo, {"input": 0, "output": 0})
        custo = (tokens_input / 1_000_000 * preco["input"] +
                 tokens_saida / 1_000_000 * preco["output"])
        self.historico.append({
            "modelo": modelo,
            "tokens_input": tokens_input,
            "tokens_saida": tokens_saida,
            "custo": custo
        })
        return custo

    def total(self):
        return sum(h["custo"] for h in self.historico)

```

```
def relatorio(self):
    print(f"Total de chamadas: {len(self.historico)}")
    print(f"Custo total: ${self.total():.6f}")
    for modelo in set(h["modelo"] for h in self.historico):
        chamadas = [h for h in self.historico if h["modelo"] == modelo]
        custo_modelo = sum(h["custo"] for h in chamadas)
        print(f" {modelo}: {len(chamadas)} chamadas,
${custo_modelo:.6f}")
```

5. Aplica

O Dia a Dia com a API

Você é desenvolvedor em uma startup e precisa integrar IA no produto. O CEO quer um chatbot que responda perguntas dos clientes sobre o produto. Você decide usar a API DeepSeek — é barata, rápida e compatível com as ferramentas que o time já usa.

O erro mais comum aqui é usar o `deepseek-v4-pro` para todas as chamadas, incluindo perguntas simples como “Qual o horário de atendimento?”. Isso é como usar um caminhão para entregar uma carta — funciona, mas é desperdício. A prática correta é criar um roteador de modelos que mapeia cada tipo de pergunta ao modelo adequado:

```
def rotear_modelo(pergunta):
    """Roteia para o modelo adequado baseado na complexidade."""
    palavras_chave_complexas = [
        "implemente", "projete", "otimize", "depure", "analise",
        "arquitetura", "explique por que", "compare", "avalie",
        "refatore", "teste", "documente"
    ]
    palavras_chave_simples = [
        "qual", "como", "o que é", "liste", "resuma", "traduza"
    ]

    pergunta_lower = pergunta.lower()

    if any(kw in pergunta_lower for kw in palavras_chave_complexas):
        return "deepseek-v4-pro", "high"
    elif any(kw in pergunta_lower for kw in palavras_chave_simples):
        return "deepseek-v4-flash", None
    else:
```

```

        return "deepseek-v4-flash", "low"

def responder(pergunta):
    modelo, esforco = rotear_modelo(pergunta)
    if esforco:
        return cliente.chat_com_raciocinio(pergunta, esforco)
    return cliente.chat_simples(pergunta, modelo)

```

Métricas de Sucesso

Métrica	Antes (sem IA)	Depois (com DeepSeek)	Melhoria
Tempo médio de resposta ao cliente	4 horas	12 segundos	1200x
Custo por interação	R\$ 15 (atendente)	R\$ 0.002 (API)	7500x
Disponibilidade	8h/dia	24/7	3x
Satisfação do cliente	72%	89%	+17pp
Capacidade de atendimento	50/dia	10.000/dia	200x

Esses números são conservadores. Em ambientes de produção bem configurados, o custo por interação pode cair para R\$ 0.0005 com o `v4-flash` e roteamento inteligente [3].

Exercício

- ☐ Crie um script Python que faz uma chamada à API com cada modelo (flash, pro, vision)
- ☐ Implemente o roteador de modelos baseado na complexidade da pergunta
- ☐ Adicione tratamento de erros para os 3 tipos de exceção mais comuns
- ☐ Meça o tempo de resposta de cada modelo com `time.time()` e compare
- ☐ Teste o streaming e exiba a resposta token a token no terminal
- ☐ Implemente o monitor de custos e verifique quanto gastou na sessão
- ☐ Compare a qualidade das respostas entre flash e pro para 5 perguntas diferentes

6. Conclusão

Neste capítulo, você conectou a primeira estação de trabalho do laboratório DeepSeek. Configurou a API, fez chamadas reais com os três modelos disponíveis e explorou recursos avançados como thinking mode e streaming. A chave é escolher o modelo certo para cada tarefa — usar

v4-flash para o simples e v4-pro com thinking para o complexo. Essa estratégia pode reduzir seus custos em 70-80% sem perder qualidade.

Você também viu que a compatibilidade com o formato OpenAI é o que permite a integração universal com dezenas de ferramentas. Não é necessário escrever código de integração customizado — basta trocar URL e chave [1].

No próximo capítulo, você vai dar um passo além: integrar o DeepSeek com ferramentas de coding que já conhece — Claude Code, GitHub Copilot, Cline e OpenCode — transformando o laboratório em uma extensão do seu fluxo de trabalho diário.

7. Referências Bibliográficas

[1] DEEPSEEK-AI. *DeepSeek API Documentation*. Disponível em: <https://api-docs.deepseek.com/>. Acesso em: 21 ago. 2026.

[2] DEEPSEEK-AI. *Awesome DeepSeek Agent*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-agent>. Acesso em: 21 ago. 2026.

[3] DEEPSEEK-AI. *DeepSeek Platform — Pricing*. Disponível em: <https://platform.deepseek.com/>. Acesso em: 21 ago. 2026.

[4] DEEPSEEK-AI. *DeepSeek-V3 Technical Report*. Disponível em: <https://arxiv.org/abs/2412.19437>. Acesso em: 21 ago. 2026.

[5] OPENAI. *OpenAI API Reference — Chat Completions*. Disponível em: <https://platform.openai.com/docs/api-reference/chat>. Acesso em: 21 ago. 2026.

[6] DEEPSEEK-AI. *Awesome DeepSeek Integration*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-integration>. Acesso em: 21 ago. 2026.

[7] DEEPSEEK-AI. *DeepSeek Harness: Everything is a Plugin*. Disponível em: <https://github.com/deepseek-ai/deepseek-harness>. Acesso em: 21 ago. 2026.

[8] DEEPSEEK-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Disponível em: <https://arxiv.org/abs/2501.12948>. Acesso em: 21 ago. 2026.

[9] DEEPSEEK-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. Disponível em: <https://arxiv.org/abs/2405.04434>. Acesso em: 21 ago. 2026.

[10] ANTHROPIC. *Anthropic API Reference*. Disponível em: <https://docs.anthropic.com/>. Acesso em: 21 ago. 2026.

[11] LANGCHAIN. *LangChain Documentation — Chat Models*. Disponível em: <https://python.langchain.com/docs/>. Acesso em: 21 ago. 2026.

- [12] DEEPSEEK-AI. *FlashMLA: Efficient Multi-head Latent Attention Kernels*. Disponível em: <https://github.com/deepseek-ai/FlashMLA>. Acesso em: 21 ago. 2026.
- [13] DEEPSEEK-AI. *DeepEP: An Efficient Expert-Parallel Communication Library*. Disponível em: <https://github.com/deepseek-ai/DeepEP>. Acesso em: 21 ago. 2026.
- [14] DEEPSEEK-AI. *DeepGEMM: Clean and Efficient BLAS Kernel Library on GPU*. Disponível em: <https://github.com/deepseek-ai/DeepGEMM>. Acesso em: 21 ago. 2026.
- [15] DEEPSEEK-AI. *3FS: A High-Performance Distributed File System for AI*. Disponível em: <https://github.com/deepseek-ai/3FS>. Acesso em: 21 ago. 2026.
- [16] DEEPSEEK-AI. *DeepSpec: Full-Stack Codebase for Speculative Decoding*. Disponível em: <https://github.com/deepseek-ai/DeepSpec>. Acesso em: 21 ago. 2026.
- [17] CORDIVERSE. *Cordis: A Meta-Framework of Spatiotemporal Composability*. Disponível em: <https://github.com/cordiverse/cordis>. Acesso em: 21 ago. 2026.
- [18] DEEPSEEK-AI. *TileKernels: Kernel Library Written in Tilelang*. Disponível em: <https://github.com/deepseek-ai/TileKernels>. Acesso em: 21 ago. 2026.
- [19] DEEPSEEK-AI. *DeepSeek-OCR-2: Visual Causal Flow*. Disponível em: <https://github.com/deepseek-ai/DeepSeek-OCR-2>. Acesso em: 21 ago. 2026.
- [20] VASWANI, A. et al. *Attention Is All You Need*. In: Advances in Neural Information Processing Systems (NeurIPS), 2017.

Capítulo 3: Integração com Ferramentas Populares

1. Introdução

No capítulo anterior, você dominou a API DeepSeek — fez chamadas, explorou streaming e thinking mode, e até implementou um roteador de modelos. Mas a verdade é que a maioria dos desenvolvedores não interage diretamente com a API. Eles usam ferramentas: Claude Code no terminal, GitHub Copilot no VS Code, Cline para edits complexos, OpenCode para web.

A boa notícia é que DeepSeek funciona como backend em todas essas ferramentas. Você não precisa abandonar o que já conhece — basta apontar a ferramenta para a API DeepSeek e pronto [1]. A compatibilidade vem da aderência aos formatos de API padronizados (OpenAI e Anthropic), que são os dois padrões de facto no mercado. Neste capítulo, você vai configurar DeepSeek nas ferramentas mais populares, entender as diferenças de cada integração e aprender a resolver os problemas mais comuns que surgem no caminho.

2. Explica

O Ecossistema de Coding Assistants

De acordo com o repositório awesome-deepseek-agent, mais de 20 coding assistants suportam DeepSeek como backend [1]. Essa adoção massiva não é coincidência — é resultado de uma decisão estratégica da DeepSeek de seguir os formatos de API padrão em vez de criar um formato proprietário.

Os coding assistants podem ser categorizados em três grupos:

Terminais nativos — Ferramentas que rodam inteiramente no terminal: Claude Code, DeepSeek-TUI, Deep Code, Reasonix. São leves, rápidos e integram naturalmente com workflows de linha de comando [1].

Extensões de editor — Ferramentas que estendem editores como VS Code e JetBrains: Cline, GitHub Copilot, Kilo Code. Oferecem autocomplete, edits inline e chat integrado no editor [1].

Plataformas multi-interface — Ferramentas que funcionam em terminal, web e outros contextos: OpenCode, LobeHub, nanobot. São mais flexíveis mas também mais pesadas [1].

A escolha da ferramenta depende do seu fluxo de trabalho. Se você passa o dia inteiro no terminal, Claude Code ou DeepSeek-TUI são naturais. Se você prefere a integração visual do VS Code, Cline ou Copilot são melhores. Não existe resposta universal — o importante é que todas funcionam com DeepSeek [1].

Claude Code

O Claude Code é um assistente de código que roda no terminal, desenvolvido pela Anthropic. Ele permite editar arquivos, executar comandos, navegar no código e fazer perguntas — tudo via interface de texto no terminal. Para usar DeepSeek como backend, basta configurar a variável de ambiente `ANTHROPIC_BASE_URL` para apontar para o endpoint Anthropic da DeepSeek [2]:

```
export ANTHROPIC_BASE_URL="https://api.deepseek.com/anthropic"
export ANTHROPIC_API_KEY="${DEEPSEEK_API_KEY}"
```

O Claude Code respeita os formatos de mensagem do Anthropic, incluindo system prompts, tool use e streaming. Isso significa que todas as funcionalidades do Claude Code funcionam com DeepSeek — exceto aquelas que dependem de modelos específicos da Anthropic (como computer use) [2].

Uma vantagem do Claude Code é sua capacidade de navegar no código-fonte do projeto, fazer edits cirúrgicos e executar comandos de terminal. Ele mantém um contexto da conversa e pode referenciar arquivos, funções e classes do seu projeto. Com DeepSeek como backend, você obtém essa funcionalidade a uma fração do custo [2].

GitHub Copilot

O GitHub Copilot é a ferramenta de code completion mais popular do mundo, com milhões de desenvolvedores ativos. Ele funciona como extensão no VS Code, JetBrains e outros editores. Para usar DeepSeek como backend no Copilot, você precisa configurar um endpoint OpenAI-compatible [3]:

A configuração varia conforme a versão do Copilot. Na maioria dos casos, basta adicionar as variáveis de ambiente corretas ou configurar o endpoint nas configurações do VS Code. O Copilot respeita o formato OpenAI de chat completions, então a compatibilidade com DeepSeek é direta [3].

Uma consideração importante: o Copilot usa o modelo para autocomplete (predict next tokens) e para chat. O autocomplete requer baixa latência, então `deepseek-v4-flash` é o modelo ideal. Para o chat, que pode tolerar latência maior, `deepseek-v4-pro` com thinking mode entrega melhor qualidade [3].

Cline

O Cline é uma extensão do VS Code que suporta múltiplos providers de API. Ele permite edits complexos, criação de arquivos, execução de comandos e navegação no código. A configuração do DeepSeek no Cline é direta — basta adicionar um novo provider com a URL e chave da API [4].

O diferencial do Cline é sua capacidade de fazer edits cirúrgicos em arquivos existentes. Em vez de reescrever um arquivo inteiro, ele identifica a seção específica que precisa ser modificada e aplica a mudança. Isso reduz tokens gastos e minimiza o risco de introduzir bugs [4].

OpenCode

O OpenCode é um assistente de código open-source disponível em terminal e web. Ele suporta MCP (Model Context Protocol), plugins e múltiplos providers. A configuração do DeepSeek é similar ao Cline — adicione o provider com URL e chave [5].

O OpenCode é especialmente interessante por suportar MCP nativamente. Isso significa que você pode conectar o OpenCode a servidores MCP que fornecem ferramentas externas (acesso a banco de dados, APIs de terceiros, filesystem), e essas ferramentas ficam disponíveis para o modelo DeepSeek usar [5].

DeepSeek-TUI

O DeepSeek-TUI é um assistente de código terminal escrito em Rust, nativamente otimizado para os modelos DeepSeek. Ele suporta 1M de contexto, MCP client e server, e ferramentas sandboxes. Diferente das outras ferramentas que são genéricas, o DeepSeek-TUI é feito sob medida para DeepSeek [6].

O Rust como linguagem de implementação traz benefícios concretos: latência mínima no TUI, consumo reduzido de memória, e suporte nativo a concorrência. Para desenvolvedores que passam a maior parte do tempo no terminal, o DeepSeek-TUI é uma opção leve e rápida [6].

Comparação entre Ferramentas

Ferramenta	Tipo	Melhor para	DeepSeek suporte
Claude Code	Terminal	Edits cirúrgicos, navegação de código	Anthropic format
GitHub Copilot	Editor	Autocomplete, chat no editor	OpenAI format
Cline	Editor VS Code	Edits complexos, criação de arquivos	OpenAI format
OpenCode	Terminal/ Web	MCP, plugins, flexibilidade	OpenAI format
DeepSeek-TUI	Terminal Rust	Performance, 1M contexto	Nativo

3. Ilustra

A Estação de Conexão

Se a API DeepSeek é a estação de comunicação, as ferramentas de coding são os aparelhos que se conectam a ela. Cada aparelho tem sua interface e seus recursos, mas todos falam o mesmo idioma (formato OpenAI ou Anthropic). É como ter diferentes tipos de telefones — celular, fixo, satelital — todos conectados à mesma rede [1].

```
%% legenda: Ferramentas de coding conectadas à API DeepSeek
flowchart LR
    subgraph Ferramentas
        A[Claude Code<br/>Terminal]
        B[GitHub Copilot<br/>VS Code]
        C[Cline<br/>VS Code]
        D[OpenCode<br/>Terminal/Web]
        E[DeepSeek-TUI<br/>Terminal Rust]
    end
    subgraph API_DeepSeek [API DeepSeek]
        F[Endpoint OpenAI<br/>api.deepseek.com]
        G[Endpoint Anthropic<br/>api.deepseek.com/anthropic]
    end
    A --> G
```

```
B --> F
C --> F
D --> F
E --> F
```

A escolha da ferramenta depende do seu fluxo de trabalho e preferências pessoais. O importante é que todas se conectam à mesma API e usam os mesmos modelos — a única diferença é a interface e os recursos específicos de cada ferramenta [1].

4. Técnica

Configuração do Claude Code

```
# Instale o Claude Code (requer Node.js 18+)
npm install -g @anthropic-ai/claude-code

# Configure as variáveis de ambiente
export ANTHROPIC_BASE_URL="https://api.deepseek.com/anthropic"
export ANTHROPIC_API_KEY="${DEEPSEEK_API_KEY}"

# Inicie o Claude Code
claude

# Dentro do Claude Code, teste:
# > Explique a arquitetura MoE do DeepSeek-V3
# > Refatore a função main() para usar async/await
# > Adicione tratamento de erros para a chamada de API
```

Configuração do Cline no VS Code

```
// settings.json do VS Code
{
  "cline.apiProvider": "deepseek",
  "cline.deepSeekBaseUrl": "https://api.deepseek.com",
  "cline.deepSeekApiKey": "${env:DEEPSEEK_API_KEY}",
  "cline.modelId": "deepseek-v4-flash"
}
```

Configuração do OpenCode

```
# Instale o OpenCode
npm install -g opencode

# Configure o provider
opencode config set provider.deepseek.url https://api.deepseek.com
opencode config set provider.deepseek.key ${DEEPSEEK_API_KEY}

# Inicie
opencode
```

Script de Verificação de Integração

```
import os
import requests
import subprocess

def verificar_integracao():
    """Verifica se a API DeepSeek está acessível e todas as ferramentas
    configuradas."""
    resultados = {}

    # 1. Verificar API
    try:
        r = requests.get(
            "https://api.deepseek.com/models",
            headers={"Authorization": f"Bearer {os.environ.get('DEEPSEEK_API_KEY')}"},
            timeout=10
        )
        resultados["API DeepSeek"] = "✅" if r.status_code == 200 else f"❌ {r.status_code}"
    except Exception as e:
        resultados["API DeepSeek"] = f"❌ {e}"

    # 2. Verificar Claude Code
    try:
        v = subprocess.check_output(["claude", "--version"], text=True,
        stderr=subprocess.DEVNULL).strip()
        resultados["Claude Code"] = f"✅ {v}"
    except:
        resultados["Claude Code"] = "⚠ Não instalado"

    # 3. Verificar VS Code
```

```

try:
    v = subprocess.check_output(["code", "--version"], text=True,
stderr=subprocess.DEVNULL).strip()
    resultados["VS Code"] = f"✅ {v.splitlines()[0]}"
except:
    resultados["VS Code"] = "⚠ Não instalado"

# 4. Verificar OpenCode
try:
    v = subprocess.check_output(["opencode", "--version"], text=True,
stderr=subprocess.DEVNULL).strip()
    resultados["OpenCode"] = f"✅ {v}"
except:
    resultados["OpenCode"] = "⚠ Não instalado"

for componente, status in resultados.items():
    print(f" {componente}: {status}")

if __name__ == "__main__":
    print("=== Verificação de Integração DeepSeek ===")
    verificar_integracao()

```

5. Aplica

O Problema da Integração Fragmentada

Você trabalha em uma empresa onde cada desenvolvedor usa uma ferramenta diferente: um usa Claude Code, outro usa Cline, um terceiro usa Copilot. O gerente quer que todos usem DeepSeek para reduzir custos, mas cada ferramenta tem sua própria configuração. O resultado é um tutorial de 20 páginas no Confluence que ninguém lê.

A solução é criar um script de setup unificado que configura todas as ferramentas de uma vez:

```

#!/bin/bash
# setup-deepseek.sh – Configura DeepSeek em todas as ferramentas

set -e

DEEPSEEK_KEY="${DEEPSEEK_API_KEY:?Defina DEEPSEEK_API_KEY}"

```

```

echo "🔧 Configurando DeepSeek para desenvolvimento..."

# 1. Variável de ambiente (persiste entre sessões)
if ! grep -q "DEEPSEEK_API_KEY" ~/.bashrc 2>/dev/null; then
    echo "export DEEPSEEK_API_KEY=\"${DEEPSEEK_KEY}\"" >> ~/.bashrc
    echo "    ✅ DEEPSEEK_API_KEY adicionada ao .bashrc"
fi

# 2. Claude Code
if command -v claude &> /dev/null; then
    export ANTHROPIC_BASE_URL="https://api.deepseek.com/anthropic"
    export ANTHROPIC_API_KEY="${DEEPSEEK_KEY}"
    echo "    ✅ Claude Code configurado"
else
    echo "    ⚠️ Claude Code não encontrado (npm install -g @anthropic-ai/claude-code)"
fi

# 3. Verificar API
echo "🔍 Verificando API..."
if curl -s https://api.deepseek.com/models -H "Authorization: Bearer ${DEEPSEEK_KEY}" | python -m json.tool > /dev/null 2>&1; then
    echo "    ✅ API DeepSeek acessível"
else
    echo "    ❌ API DeepSeek inacessível – verifique sua chave"
    exit 1
fi

echo "✅ Configuração concluída. Reinicie o terminal para aplicar."

```

O erro mais comum em integrações é esquecer que o endpoint Anthropic (/anthropic) e o endpoint OpenAI (/) são diferentes. Se você usar o endpoint errado, receberá erros 400 ou 422 que são difíceis de debugar. Sempre verifique qual formato sua ferramenta espera antes de configurar [1].

Exercício

- ☐ Configure DeepSeek no Claude Code e teste com uma pergunta simples
- ☐ Configure DeepSeek no Cline e faça um edit de código
- ☐ Compare o tempo de resposta entre Claude Code com DeepSeek vs. Anthropic
- ☐ Crie o script de setup unificado para sua equipe
- ☐ Teste o DeepSeek-TUI e compare com Claude Code
- ☐ Verifique se o OpenCode consegue usar ferramentas MCP com DeepSeek

6. Conclusão

Neste capítulo, você conectou as ferramentas de coding ao laboratório DeepSeek. Viu que a compatibilidade com os formatos OpenAI e Anthropic é o que permite essa integração universal — basta trocar URL e chave. Cada ferramenta tem suas peculiaridades: Claude Code para edits cirúrgicos, Copilot para autocomplete, Cline para edits complexos, OpenCode para MCP, DeepSeek-TUI para performance.

A chave é escolher a ferramenta que melhor se adapta ao seu fluxo de trabalho e configurá-la corretamente com a API DeepSeek. O resultado é a mesma coisa: IA de ponta a uma fração do custo [1].

No próximo capítulo, você vai além da conexão: aprender a escrever prompts que extraem o máximo dos modelos DeepSeek — desde prompts simples até reasoning chains complexas.

7. Referências Bibliográficas

[1] DEEPSEEK-AI. *Awesome DeepSeek Agent*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-agent>. Acesso em: 21 ago. 2026.

[2] ANTHROPIC. *Claude Code Documentation*. Disponível em: <https://docs.anthropic.com/>. Acesso em: 21 ago. 2026.

[3] GITHUB. *GitHub Copilot Documentation*. Disponível em: <https://docs.github.com/copilot>. Acesso em: 21 ago. 2026.

[4] CLINE. *Cline VS Code Extension*. Disponível em: <https://github.com/cline/cline>. Acesso em: 21 ago. 2026.

[5] OPENCODE. *OpenCode — AI Coding Assistant*. Disponível em: <https://github.com/opencode-ai/opencode>. Acesso em: 21 ago. 2026.

[6] DEEPSEEK-AI. *DeepSeek-TUI: Rust Terminal Coding Assistant*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-agent>. Acesso em: 21 ago. 2026.

[7] DEEPSEEK-AI. *DeepSeek API Documentation*. Disponível em: <https://api-docs.deepseek.com/>. Acesso em: 21 ago. 2026.

[8] DEEPSEEK-AI. *DeepSeek Harness: Everything is a Plugin*. Disponível em: <https://github.com/deepseek-ai/deepseek-harness>. Acesso em: 21 ago. 2026.

[9] DEEPSEEK-AI. *DeepSeek-V3 Technical Report*. Disponível em: <https://arxiv.org/abs/2412.19437>. Acesso em: 21 ago. 2026.

- [10] DEEPSEEK-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Disponível em: <https://arxiv.org/abs/2501.12948>. Acesso em: 21 ago. 2026.
- [11] DEEPSEEK-AI. *Awesome DeepSeek Integration*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-integration>. Acesso em: 21 ago. 2026.
- [12] DEEPSEEK-AI. *FlashMLA: Efficient Multi-head Latent Attention Kernels*. Disponível em: <https://github.com/deepseek-ai/FlashMLA>. Acesso em: 21 ago. 2026.
- [13] DEEPSEEK-AI. *DeepEP: An Efficient Expert-Parallel Communication Library*. Disponível em: <https://github.com/deepseek-ai/DeepEP>. Acesso em: 21 ago. 2026.
- [14] DEEPSEEK-AI. *DeepGEMM: Clean and Efficient BLAS Kernel Library on GPU*. Disponível em: <https://github.com/deepseek-ai/DeepGEMM>. Acesso em: 21 ago. 2026.
- [15] DEEPSEEK-AI. *3FS: A High-Performance Distributed File System for AI*. Disponível em: <https://github.com/deepseek-ai/3FS>. Acesso em: 21 ago. 2026.
- [16] DEEPSEEK-AI. *DeepSpec: Full-Stack Codebase for Speculative Decoding*. Disponível em: <https://github.com/deepseek-ai/DeepSpec>. Acesso em: 21 ago. 2026.
- [17] CORDIVERSE. *Cordis: A Meta-Framework of Spatiotemporal Composability*. Disponível em: <https://github.com/cordiverse/cordis>. Acesso em: 21 ago. 2026.
- [18] DEEPSEEK-AI. *TileKernels: Kernel Library Written in Tilelang*. Disponível em: <https://github.com/deepseek-ai/TileKernels>. Acesso em: 21 ago. 2026.
- [19] DEEPSEEK-AI. *DeepSeek-OCR-2: Visual Causal Flow*. Disponível em: <https://github.com/deepseek-ai/DeepSeek-OCR-2>. Acesso em: 21 ago. 2026.
- [20] VASWANI, A. et al. *Attention Is All You Need*. In: Advances in Neural Information Processing Systems (NeurIPS), 2017.

8. Apêndice — Integração MCP com DeepSeek

O que é MCP

O Model Context Protocol (MCP) é um protocolo que permite a ferramentas de IA acessar recursos externos de forma padronizada. Com MCP, um coding assistant pode acessar filesystem, bancos de dados, APIs de terceiros e outras ferramentas — tudo através de uma interface unificada [1].

Configurando MCP com DeepSeek

```
// Configuração de MCP servers para DeepSeek
const mcpServers = {
  "filesystem": {
    command: "npx",
    args: ["-y", "@modelcontextprotocol/server-filesystem", "."],
    description: "Acesso ao filesystem do projeto"
  },
  "github": {
    command: "npx",
    args: ["-y", "@modelcontextprotocol/server-github"],
    env: { GITHUB_TOKEN: process.env.GITHUB_TOKEN },
    description: "Integração com GitHub (repos, issues, PRs)"
  },
  "postgres": {
    command: "npx",
    args: ["-y", "@modelcontextprotocol/server-postgres"],
    env: { DATABASE_URL: process.env.DATABASE_URL },
    description: "Consultas ao banco de dados"
  }
};
```

Benefícios do MCP com DeepSeek

Benefício	Descrição
Acesso a dados	O modelo pode ler/escrever arquivos, consultar bancos
Automação	Ações como criar PR, deploy, migration são possíveis
Contexto rico	O modelo acessa o código-fonte real, não apenas descrições
Segurança	MCP sandboxa operações, prevenindo ações destrutivas
Extensibilidade	Qualquer ferramenta pode ser exposta via MCP

Exemplo Prático: Code Review com MCP

```
# Fluxo de code review usando MCP + DeepSeek
async def code_review_mcp(pr_number: int):
    """Faz review de um PR usando MCP para acessar o GitHub."""

    # 1. Busca o diff do PR via MCP GitHub
    diff = await mcp.github.get_pr_diff(pr_number)

    # 2. Envia o diff para DeepSeek com prompt de review
    resposta = await deepseek.chat(
        model="deepseek-v4-pro",
        messages=[
            {
                "role": "user",
                "content": f"""Revise este pull request e identifique:
                1. Bugs potenciais
                2. Vulnerabilidades de segurança
                3. Melhorias de performance
                4. Problemas de legibilidade

                Diff:
                ```diff
 {diff}
                ```"""
            }
        ],
        thinking={"type": "enabled"},
        reasoning_effort="high"
    )

    # 3. Posta o review como comentário no PR
    await mcp.github.create_pr_comment(pr_number, resposta)

    return resposta
```

9. Troubleshooting — Problemas Comuns e Soluções

Erro 401: Unauthorized

Causa: API key inválida ou ausente. **Solução:** Verifique se `DEEPSEEK_API_KEY` está configurada e se não tem espaços extras.

```
echo $DEEPSEEK_API_KEY # Deve imprimir a chave sem aspas
```

Erro 429: Rate Limit

Causa: Muitas requisições em pouco tempo. **Solução:** Implemente backoff exponencial. A API DeepSeek permite ~60 requests/minuto para contas gratuitas.

```
import time
def chamada_com_retry(cliente, pergunta, max_tentativas=3):
    for i in range(max_tentativas):
        try:
            return cliente.chat_simples(pergunta)
        except RateLimitError:
            tempo_espera = 2 ** i * 5
            print(f"Rate limit. Aguardando {tempo_espera}s...")
            time.sleep(tempo_espera)
    raise Exception("Rate limit persistente após 3 tentativas")
```

Erro 400: Bad Request

Causa: Formato da requisição incorreto. Comum ao misturar endpoints OpenAI/Anthropic. **Solução:** Verifique se está usando o endpoint correto para sua ferramenta.

Resposta truncada

Causa: `max_tokens` muito baixo ou modelo atingiu limite de contexto. **Solução:** Aumente `max_tokens` ou reduza o tamanho do prompt.

Resposta em inglês quando pediu português

Causa: System prompt insuficiente ou modelo em modo multilíngue. **Solução:** Adicione “Responda SEMPRE em português do Brasil” no início do system prompt.

Thinking mode não funciona

Causa: Modelo não suporta thinking (v4-flash não tem thinking profundo). **Solução:** Use `deepseek-v4-pro` para thinking mode. O flash suporta apenas thinking superficial.

A. Boas Práticas de Integração

Segurança

```
# Nunca exponha sua API key no código
# ❌ ERRADO
api_key = "sk-1234567890abcdef"

# ✅ CORRETO
api_key = os.environ.get("DEEPSEEK_API_KEY")

# Use .env para desenvolvimento
# .env (não committar!)
DEEPSEEK_API_KEY=sk-1234567890abcdef

# .gitignore
.env
*.env.local
```

Resiliência

```
import time
from functools import wraps

def retry_with_backoff(max_retries=3, base_delay=1):
    """Decorator com retry e backoff exponencial."""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_retries):
                try:
                    return func(*args, **kwargs)
                except RateLimitError:
                    delay = base_delay * (2 ** attempt)
                    print(f"Rate limit. Tentativa {attempt+1}/
{max_retries}. Aguardando {delay}s...")
                    time.sleep(delay)
                except APIError as e:
                    if e.status_code >= 500:
                        delay = base_delay * (2 ** attempt)
                        print(f"Erro do servidor. Tentativa {attempt+1}/
```

```

{max_retries}. Aguardando {delay}s...)
        time.sleep(delay)
    else:
        raise
        raise Exception(f"Falhou após {max_retries} tentativas")
    return wrapper
return decorator

@retry_with_backoff(max_retries=3)
def chamar_api(pergunta):
    return cliente.chat_simples(pergunta)

```

Monitoramento

```

import logging

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[
        logging.FileHandler("deepseek.log"),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger("deepseek")

def chamar_com_logging(pergunta):
    """Chamada com logging completo."""
    logger.info(f"Início: {pergunta[:50]}...")
    inicio = time.time()

    try:
        resultado = cliente.chat_simples(pergunta)
        latencia = time.time() - inicio
        logger.info(f"Sucesso: {latencia:.2f}s, {len(resultado)} chars")
        return resultado
    except Exception as e:
        latencia = time.time() - inicio
        logger.error(f"Erro: {latencia:.2f}s, {e}")
        raise

```

Cache Inteligente

```
from datetime import datetime, timedelta

class CacheInteligente:
    """Cache com TTL e invalidação inteligente."""

    def __init__(self, ttl_horas=24):
        self.cache = {}
        self.ttl = timedelta(hours=ttl_horas)
        self.hits = 0
        self.misses = 0

    def get(self, chave):
        if chave in self.cache:
            entrada = self.cache[chave]
            if datetime.now() - entrada["timestamp"] < self.ttl:
                self.hits += 1
                return entrada["valor"]
            else:
                del self.cache[chave]
        self.misses += 1
        return None

    def set(self, chave, valor):
        self.cache[chave] = {
            "valor": valor,
            "timestamp": datetime.now()
        }

    def stats(self):
        total = self.hits + self.misses
        hit_rate = self.hits / total if total > 0 else 0
        return {
            "hits": self.hits,
            "misses": self.misses,
            "hit_rate": hit_rate,
            "size": len(self.cache)
        }
```


Capítulo 4: Prompt Engineering para DeepSeek

1. Introdução

Nos capítulos anteriores, você configurou a API, integrou DeepSeek com suas ferramentas de coding e sabe qual modelo usar para cada situação. Mas há um gap entre ter acesso a um modelo e extrair dele a melhor resposta possível. Esse gap se chama prompt engineering — a arte de formular instruções que maximizam a qualidade da saída.

DeepSeek tem particularidades que outros modelos não têm. O thinking mode muda completamente a dinâmica do prompt: em vez de pedir uma resposta direta, você está pedindo ao modelo que raciocine antes de responder [1]. Os system prompts funcionam de forma ligeiramente diferente nos modelos de raciocínio. A temperatura e o top-p têm efeitos específicos nos modelos MoE. Prompts que funcionam perfeitamente no GPT-4 podem entregar resultados medianos no DeepSeek se não considerarem essas particularidades. Neste capítulo, você vai dominar cada uma dessas nuances e construir um toolkit reutilizável de prompts para diferentes tarefas.

2. Explica

A Estrutura de um Prompt Eficaz

Um prompt para DeepSeek tem três camadas: contexto, instrução e formato de saída. A ordem importa — modelos de raciocínio como o V4-Pro processam o contexto antes de gerar a resposta, então informações cruciais devem vir primeiro [1].

Contexto — Quem é o modelo, qual é a situação, quais são as restrições. Ex.: “Você é um engenheiro de software sênior trabalhando em um sistema de pagamentos que processa 10.000 transações por minuto.”

Instrução — O que o modelo deve fazer. Deve ser específica e acionável. Ex.: “Revise o código abaixo e identifique 3 vulnerabilidades de segurança, classificando cada uma por severidade (crítica/alta/média/baixa).”

Formato de Saída — Como a resposta deve ser estruturada. Ex.: “Responda em formato JSON com os campos: vulnerabilidade, severidade, explicacao, correcao.”

A ordem contexto → instrução → formato é ótima para a maioria das tarefas. Mas há exceções: para tarefas criativas, o formato pode vir antes da instrução (para dar liberdade ao modelo). Para tarefas de classificação, o contexto pode ser dispensado [1].

System Prompts no DeepSeek

Diferente de outros modelos, o DeepSeek-R1 e variantes de raciocínio recomendam NÃO usar system prompts — todas as instruções devem ir no prompt do usuário [2]. Isso ocorre porque o system prompt pode interferir no processo de raciocínio interno do modelo. Para os modelos V4 (Flash e Pro), system prompts funcionam normalmente, mas para R1, prefira colocar tudo no user message.

Isso não significa que system prompts são proibidos no V4-Pro — apenas que para tarefas que exigem raciocínio profundo, colocar tudo no user message pode entregar melhores resultados. A recomendação prática é: teste com e sem system prompt para tarefas críticas e compare [1].

Thinking Mode e Prompt

Quando o thinking mode está ativado, o modelo gera uma cadeia de raciocínio interna antes da resposta. Isso significa que o prompt deve ser mais explícito sobre o que você espera — o modelo vai “pensar” sobre sua instrução antes de responder. Prompts vagos produzem raciocínio vago [1]:

```
# ❌ Prompt vago – raciocínio superficial
messages = [{"role": "user", "content": "Melhore este código"}]

# ✅ Prompt explícito – raciocínio profundo
messages = [{"role": "user", "content":
    "Analise o código abaixo e proponha 3 melhorias específicas: "
    "(1) redução de complexidade ciclomática, (2) tratamento de erros, "
    "(3) performance. Para cada melhoria, mostre o código original e o "
    "refatorado.\n\n"
    f"```python\n{codigo}\n```"
}]
```

A diferença é que o segundo prompt dá ao modelo critérios claros de avaliação. O thinking mode vai processar cada critério separadamente, gerando um raciocínio estruturado. O primeiro prompt é vago o suficiente para o modelo gerar qualquer coisa — incluindo melhorias irrelevantes [1].

Few-Shot Learning

O DeepSeek responde extremamente bem a few-shot learning — mostrar exemplos de entrada/saída antes da pergunta real. Isso é especialmente útil para tarefas de formatação e classificação [1]:

```
messages = [  
    # Exemplo 1: entrada → saída  
    {"role": "user", "content": "Classifique o sentimento: 'O produto é ótimo!'"},  
    {"role": "assistant", "content": '{"sentimento": "positivo",  
"confianca": 0.95}'},  
    # Exemplo 2: entrada → saída  
    {"role": "user", "content": "Classifique o sentimento: 'Péssimo atendimento'"},  
    {"role": "assistant", "content": '{"sentimento": "negativo",  
"confianca": 0.90}'},  
    # Exemplo 3: entrada → (modelo responde)  
    {"role": "user", "content": "Classifique o sentimento: 'O produto chegou ontem'"},  
]
```

O padrão few-shot é especialmente poderoso para tarefas onde o formato da resposta é crítico. Em vez de descrever o formato em prosa, mostre exemplos concretos — modelos de linguagem são excepcionalmente bons em inferir padrões a partir de exemplos [1].

Temperature e Top-P

A temperatura controla a aleatoriedade das respostas. Para DeepSeek, os valores recomendados são [2]:

Tarefa	Temperature	Top-P	Reasoning Effort
Código	0.0 - 0.3	0.95	—
Raciocínio matemático	0.5 - 0.7	0.95	high
Chat criativo	0.7 - 1.0	0.95	—
Classificação	0.0	1.0	—
Resumo	0.3 - 0.5	0.95	—
Geração de código	0.0 - 0.2	0.95	medium

Para o DeepSeek-R1, a documentação recomenda temperature entre 0.5 e 0.7 (0.6 é o ideal) para evitar repetições infinitas [2]. Temperaturas acima de 0.7 podem causar respostas incoerentes; abaixo de 0.3 podem ser excessivamente determinísticas.

Chain-of-Thought (CoT)

O DeepSeek-V4 com thinking mode implementa chain-of-thought naturalmente — o modelo mostra seu raciocínio antes de gerar a resposta. Mas você pode forçar um CoT mais estruturado através do prompt [1]:

```
def prompt_cot_estruturado(problema):
    """Prompt que força raciocínio passo a passo."""
    return f"""Resolva o problema abaixo mostrando cada passo do
raciocínio:

Problema: {problema}

Formato obrigatório:
1. **Entendendo o problema**: O que está sendo pedido?
2. **Dados disponíveis**: Quais informações temos?
3. **Abordagem**: Qual estratégia usar?
4. **Execução**: Passo a passo detalhado
5. **Verificação**: O resultado faz sentido?
6. **Resposta final**: Solução clara e concisa"""
```

3. Ilustra

A Estação de Calibração

Se o laboratório DeepSeek é um conjunto de estações de trabalho, o prompt engineering é a calibração de cada instrumento. Um microscópio mal calibrado produz imagens borradas; um prompt mal formulado produz respostas irrelevantes [1].

```
%% legenda: Pipeline de calibração de prompts para DeepSeek
flowchart TD
    A[Tarefa] --> B{Requer raciocínio?}
    B -->|Sim| C[Thinking Mode + reasoning_effort high]
    B -->|Não| D{Formato de saída?}
    D -->|JSON| E[Few-shot com exemplos JSON]
    D -->|Texto| F[System prompt + instrução clara]
    D -->|Código| G[Código de exemplo + linguagem declarada]
    C --> H[Prompt explícito com critérios]
    E --> H
    F --> H
    G --> H
    H --> I[Resposta estruturada]
```

A calibração envolve três ajustes: o que você pede (instrução), como você pede (formato) e quanto difícil você torna o problema (reasoning effort). O erro mais comum é pular a calibração e usar o mesmo prompt para todos os tipos de tarefa [1].

4. Técnica

Biblioteca de Prompts para DeepSeek

```
class DeepSeekPromptBuilder:
    """Construtor de prompts otimizados para DeepSeek."""

    def __init__(self, modelo="deepseek-v4-pro"):
        self.modelo = modelo
        self.mensagens = []
        self.config = {}
```

```

def contexto(self, texto):
    """Adiciona contexto ao prompt."""
    self.mensagens.append({"role": "user", "content": f"Contexto: {texto}"})
    return self

def instrucao(self, texto):
    """Adiciona instrução principal."""
    self.mensagens.append({"role": "user", "content": f"Instrução: {texto}"})
    return self

def exemplo(self, entrada, saida):
    """Adiciona par exemplo para few-shot."""
    self.mensagens.append({"role": "user", "content": entrada})
    self.mensagens.append({"role": "assistant", "content": saida})
    return self

def formato_json(self, schema):
    """Define formato JSON de saída."""
    self.mensagens.append({
        "role": "user",
        "content": f"Responda em JSON válido seguindo este schema: {schema}"
    })
    return self

def formato_lista(self, n_items=5):
    """Define formato de lista numerada."""
    self.mensagens.append({
        "role": "user",
        "content": f"Responda com uma lista numerada de até {n_items} itens."
    })
    return self

def com_thinking(self, esforco="high"):
    """Ativa thinking mode."""
    self.config["thinking"] = {"type": "enabled"}
    self.config["reasoning_effort"] = esforco
    return self

def construir(self):
    """Retorna payload completo para a API."""
    payload = {"model": self.modelo, "messages": self.mensagens}
    if self.config:

```

```

        payload["extra_body"] = self.config
    return payload

# Uso: prompt para review de código
prompt = (DeepSeekPromptBuilder("deepseek-v4-pro")
    .contexto("Você é um revisor de código Python sênior")
    .instrucao("Analisar o código e identifique vulnerabilidades de
segurança")
    .formato_json('{"vulnerabilidades": [{"tipo": str, "severidade": str,
"correcao": str}]})')
    .com_thinking("high")
    .construir())

```

Prompt para Reasoning Chain

```

def prompt_reasoning_chain(problema):
    """Prompt que força o modelo a mostrar o raciocínio."""
    return {
        "model": "deepseek-v4-pro",
        "messages": [{
            "role": "user",
            "content": f"""Resolva o problema passo a passo:

Problema: {problema}

Instruções:
1. Primeiro, identifique os dados disponíveis
2. Em seguida, defina a abordagem
3. Execute passo a passo, mostrando cada cálculo
4. Verifique o resultado
5. Responda com a solução final

Formato esperado:
**Dados:** (liste os dados disponíveis no problema)
**Abordagem:** (descreva a estratégia escolhida)
**Passo 1:** (primeiro passo da resolução)
**Passo 2:** (segundo passo da resolução)
**Verificação:** (confirme se o resultado faz sentido)
**Solução Final:** (apresente a resposta)"""
        }],
        "extra_body": {"thinking": {"type": "enabled"}},
        "reasoning_effort": "high"
    }

```

Otimização de Prompts por Tarefa

```
PROMPTS_OTIMIZADOS = {
    "review_codigo": {
        "template": "Revise o código {linguagem} abaixo. Identifique: (1) bugs, (2) vulnerabilidades, (3) melhorias de performance. Para cada item, mostre a linha afetada e a correção.\n\n```${linguagem}\n{codigo}\n```",
        "modelo": "deepseek-v4-pro",
        "reasoning": "high"
    },
    "resumo_documento": {
        "template": "Resuma o documento em 5 bullet points. Foque nas decisões técnicas e impactos.\n\n{documento}",
        "modelo": "deepseek-v4-flash",
        "reasoning": None
    },
    "gerar_testes": {
        "template": "Gere testes unitários para a função abaixo usando pytest. Inclua casos normais, limite e erro.\n\n```python\n{funcao}\n```",
        "modelo": "deepseek-v4-pro",
        "reasoning": "medium"
    },
    "refactoring": {
        "template": "Refatore o código abaixo aplicando: (1) Extração de método, (2) Redução de duplicação, (3) Nomes mais descritivos. Mostre antes e depois.\n\n```${linguagem}\n{codigo}\n```",
        "modelo": "deepseek-v4-pro",
        "reasoning": "high"
    },
    "documentacao": {
        "template": "Gere documentação completa para a função/classe abaixo. Inclua: descrição, parâmetros, retorno, exemplos de uso.\n\n```${linguagem}\n{codigo}\n```",
        "modelo": "deepseek-v4-flash",
        "reasoning": None
    }
}
```

Padrões de Anti-Patterns

❌ Anti-padrões – evite estes padrões de prompt

1. Prompt ambíguo

"Melhore isto" # O que melhorar? Como medir melhoria?


```
# 2. Múltiplas tarefas sem hierarquia
"Escreva testes, documente, e refatore este código" # 0 que é prioridade?

# 3. Prompt sem formato de saída
"Analise este código" # Como o resultado deve ser estruturado?

# 4. Contexto insuficiente
"Por que isto está lento?" # Sem contexto, resposta genérica

#  Padrões corretos

# 1. Prompt específico com critérios
"Revise o código e identifique 3 melhorias de performance, classificadas por impacto (alto/médio/baixo)"

# 2. Tarefa única com foco
"Gere testes unitários para a função processar_pagamento"

# 3. Formato claro
"Responda em JSON: {\"melhorias\": [{\"tipo\": str, \"impacto\": str, \"codigo\": str}]}"

# 4. Contexto completo
"Este endpoint processa 10.000 requests/minuto. Por que a latência p99 está acima de 200ms?"
```

5. Aplica

O Prompt que Não Funciona

Você pede ao DeepSeek: “Escreva um sistema de autenticação JWT”. A resposta vem genérica, com código incompleto e sem tratamento de erros. O problema não é o modelo — é o prompt. Prompts genéricos produzem respostas genéricas [1].

A prática correta é ser específico sobre o contexto, os requisitos e o formato:

```
resposta = chat_com_raciocinio(
    cliente,
    """Implemente um sistema de autenticação JWT para uma API FastAPI:

Contexto:
```

- API de e-commerce com 50.000 usuários ativos
- Precisa suportar 1.000 logins por minuto
- Stack: Python 3.11+, FastAPI, SQLite

Requisitos:

- Login com email/senha (bcrypt para hash)
- Geração de access token (15min) e refresh token (7 dias)
- Middleware de verificação de token
- Endpoint de refresh (rotaciona refresh token)
- Logout (invalidação do refresh token no banco)
- Rate limiting no login (5 tentativas por minuto)

Restrições:

- Tokens em httpOnly cookies (não em headers)
- SQLite para refresh tokens (não Redis)
- Sem dependências externas além de PyJWT e bcrypt

Formato: Arquivo Python único com comentários explicativos. Inclua testes básicos. """
)

Métricas de Qualidade de Prompt

Métrica	Prompt Ruim	Prompt Bom
Especificidade	“Melhore o código”	“Reduza complexidade ciclomática < 10”
Formato	Sem definição	JSON com schema explícito
Contexto	“Por que é lento?”	“Endpoint com 10k req/min, p99 > 200ms”
Exemplos	Nenhum	2-3 few-shot examples
Restrições	Nenhuma	“Sem dependências externas”

Exercício

- ☐ Escreva um prompt few-shot que classifica código em “seguro” ou “vulnerável”
- ☐ Implemente o DeepSeekPromptBuilder e teste com 3 tarefas diferentes
- ☐ Compare respostas com e sem thinking mode para um problema de debug
- ☐ Crie uma biblioteca de prompts para as 5 tarefas que você mais faz
- ☐ Identifique 3 anti-patterns nos prompts que você usa no dia a dia

- ☐ Teste a diferença entre temperature 0.0 e 0.7 para geração de código

6. Conclusão

Neste capítulo, você dominou a arte de calibrar prompts para DeepSeek. Viu que a estrutura do prompt (contexto → instrução → formato) é fundamental, que o thinking mode exige prompts mais explícitos, e que few-shot learning é uma arma poderosa. A diferença entre um prompt medíocre e um excelente pode ser 10x na qualidade da resposta [1].

Você também aprendeu a evitar anti-patterns comuns e a construir prompts reutilizáveis para diferentes tarefas. O DeepSeekPromptBuilder é uma ferramenta que você pode levar para qualquer projeto [1].

No próximo capítulo, você sai da superfície e mergulha na arquitetura do DeepSeek Harness — o framework de agentes que permite construir soluções complexas com uma arquitetura de plugins elegante.

7. Referências Bibliográficas

[1] DEEPSEEK-AI. *DeepSeek API Documentation — Prompting Guide*. Disponível em: <https://api-docs.deepseek.com/>. Acesso em: 21 ago. 2026.

[2] DEEPSEEK-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Disponível em: <https://arxiv.org/abs/2501.12948>. Acesso em: 21 ago. 2026.

[3] DEEPSEEK-AI. *DeepSeek-V3 Technical Report*. Disponível em: <https://arxiv.org/abs/2412.19437>. Acesso em: 21 ago. 2026.

[4] DEEPSEEK-AI. *DeepSeek Harness: Everything is a Plugin*. Disponível em: <https://github.com/deepseek-ai/deepseek-harness>. Acesso em: 21 ago. 2026.

[5] DEEPSEEK-AI. *Awesome DeepSeek Agent*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-agent>. Acesso em: 21 ago. 2026.

[6] DEEPSEEK-AI. *Awesome DeepSeek Integration*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-integration>. Acesso em: 21 ago. 2026.

[7] DEEPSEEK-AI. *FlashMLA: Efficient Multi-head Latent Attention Kernels*. Disponível em: <https://github.com/deepseek-ai/FlashMLA>. Acesso em: 21 ago. 2026.

[8] DEEPSEEK-AI. *DeepEP: An Efficient Expert-Parallel Communication Library*. Disponível em: <https://github.com/deepseek-ai/DeepEP>. Acesso em: 21 ago. 2026.

- [9] DEEPSEEK-AI. *DeepGEMM: Clean and Efficient BLAS Kernel Library on GPU*. Disponível em: <https://github.com/deepseek-ai/DeepGEMM>. Acesso em: 21 ago. 2026.
- [10] DEEPSEEK-AI. *3FS: A High-Performance Distributed File System for AI*. Disponível em: <https://github.com/deepseek-ai/3FS>. Acesso em: 21 ago. 2026.
- [11] DEEPSEEK-AI. *DeepSpec: Full-Stack Codebase for Speculative Decoding*. Disponível em: <https://github.com/deepseek-ai/DeepSpec>. Acesso em: 21 ago. 2026.
- [12] CORDIVERSE. *Cordis: A Meta-Framework of Spatiotemporal Composability*. Disponível em: <https://github.com/cordiverse/cordis>. Acesso em: 21 ago. 2026.
- [13] DEEPSEEK-AI. *TileKernels: Kernel Library Written in Tilelang*. Disponível em: <https://github.com/deepseek-ai/TileKernels>. Acesso em: 21 ago. 2026.
- [14] DEEPSEEK-AI. *DeepSeek-OCR-2: Visual Causal Flow*. Disponível em: <https://github.com/deepseek-ai/DeepSeek-OCR-2>. Acesso em: 21 ago. 2026.
- [15] VASWANI, A. et al. *Attention Is All You Need*. In: Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [16] OPENAI. *OpenAI Prompt Engineering Guide*. Disponível em: <https://platform.openai.com/docs/guides/prompt-engineering>. Acesso em: 21 ago. 2026.
- [17] ANTHROPIC. *Prompt Engineering Guide*. Disponível em: <https://docs.anthropic.com/>. Acesso em: 21 ago. 2026.
- [18] DEEPSEEK-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. Disponível em: <https://arxiv.org/abs/2405.04434>. Acesso em: 21 ago. 2026.
- [19] FEDUS, W. et al. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. In: Journal of Machine Learning Research, 2022.
- [20] DEEPSEEK-AI. *DeepSeek Platform — Pricing*. Disponível em: <https://platform.deepseek.com/>. Acesso em: 21 ago. 2026.

8. Apêndice — Técnicas Avançadas de Prompt

Self-Consistency

O DeepSeek com thinking mode pode gerar múltiplas soluções para o mesmo problema. A técnica self-consistency pega as respostas mais frequentes como a correta [1]:

```
def self_consistency(cliente, problema, n_amostras=5):
    """Gera múltiplas respostas e pega a mais consistente."""
    respostas = []
    for _ in range(n_amostras):
        r = cliente.chat_com_raciocinio(problema, "high")
        respostas.append(r["resposta"])

    # Conta ocorrências de cada resposta
    from collections import Counter
    contagem = Counter(respostas)
    resposta_mais_comum = contagem.most_common(1)[0]

    return {
        "resposta": resposta_mais_comum[0],
        "consistencia": resposta_mais_comum[1] / n_amostras,
        "todas": respostas
    }
```

Tree of Thoughts

Para problemas complexos, o Tree of Thoughts explora múltiplos caminhos de raciocínio [1]:

```
def tree_of_thoughts(problema):
    return f"""Resolva o problema explorando 3 caminhos diferentes:

Problema: {problema}

Para cada caminho:
1. **Caminho A (Abordagem direta):** (resolva de forma direta)
2. **Caminho B (Abordagem alternativa):** (explore uma abordagem diferente)
3. **Caminho C (Abordagem criativa):** (use criatividade para resolver)

Depois:
- Avalie cada caminho (prós e contras)
- Escolha o melhor
- Implemente a solução final"""
```

Prompt Chaining

Para tarefas complexas, encadeie múltiplos prompts [1]:

```
async def prompt_chain(cliente, tarefa):
    """Encadeia múltiplos prompts para tarefa complexa."""
    # Passo 1: Análise
```

```

analise = await cliente.chat_com_raciocinio(
    f"Analise esta tarefa e identifique os subproblemas:\n{tarefa}"
)

# Passo 2: Planejamento
plano = await cliente.chat_com_raciocinio(
    f"Com base nesta análise, crie um plano de execução:\n{analise}"
)

# Passo 3: Execução
execucao = await cliente.chat_com_raciocinio(
    f"Execute este plano passo a passo:\n{plano}"
)

# Passo 4: Validação
validacao = await cliente.chat_com_raciocinio(
    f"Revise esta execução e identifique erros:\n{execucao}"
)

return {"analise": analise, "plano": plano, "execucao": execucao,
        "validacao": validacao}

```

9. Apêndice — Calibração e Eval de Prompts

Framework de Eval

```

import json
from typing import List, Dict

class PromptEvaluator:
    """Avalia qualidade de prompts para DeepSeek."""

    def __init__(self, cliente):
        self.cliente = cliente

    def avaliar(self, prompt: str, criterios: List[Dict]) -> Dict:
        """Avalia um prompt contra múltiplos critérios."""
        resultados = []

        for criterio in criterios:
            resposta = self.cliente.chat_com_raciocinio(

```

```

        f""Avalie se a resposta abaixo atende ao critério:

Critério: {criterio['nome']}
Descrição: {criterio['descricao']}

Resposta para avaliar:
{prompt}

Responda APENAS com JSON: {"atende": true/false, "confianca": 0.0-1.0,
"justificativa": "descreva o motivo"}""",
        "medium"
    )

    try:
        avaliacao = json.loads(resposta["resposta"])
        resultados.append({
            "criterio": criterio["nome"],
            **avaliacao
        })
    except json.JSONDecodeError:
        resultados.append({
            "criterio": criterio["nome"],
            "atende": False,
            "confianca": 0,
            "justificativa": "Erro ao parsear resposta"
        })

    # Calcula score geral
    scores = [r["confianca"] for r in resultados if r["atende"]]
    score_geral = sum(scores) / len(criterios) if criterios else 0

    return {
        "score_geral": score_geral,
        "resultados": resultados,
        "aprovado": score_geral >= 0.7
    }

# Exemplo de uso
evaluator = PromptEvaluator(cliente)
criterios = [
    {"nome": "Formato JSON", "descricao": "Resposta está em JSON válido"},
    {"nome": "Completeness", "descricao": "Todos os campos obrigatórios estão presentes"},
    {"nome": "Precision", "descricao": "Valores são factuais e verificáveis"}
]

```

```
resultado = evaluator.aviar(prompt_exemplo, criterios)
print(f"Score: {resultado['score_geral']:.2f}")
print(f"Aprovado: {resultado['aprovado']}")
```

A/B Testing de Prompts

```
def ab_test_prompts(cliente, pergunta, prompt_a, prompt_b, n_amstras=10):
    """Compara dois prompts via A/B testing."""
    resultados_a = []
    resultados_b = []

    for _ in range(n_amstras):
        # Prompt A
        r_a = cliente.chat_simples(pergunta + "\n\n" + prompt_a)
        resultados_a.append(len(r_a)) # Proxy: tamanho da resposta

        # Prompt B
        r_b = cliente.chat_simples(pergunta + "\n\n" + prompt_b)
        resultados_b.append(len(r_b))

    media_a = sum(resultados_a) / len(resultados_a)
    media_b = sum(resultados_b) / len(resultados_b)

    vencedor = "A" if media_a > media_b else "B"

    print(f"Prompt A: média {media_a:.0f} chars")
    print(f"Prompt B: média {media_b:.0f} chars")
    print(f"Vencedor: Prompt {vencedor}")

    return {"vencedor": vencedor, "media_a": media_a, "media_b": media_b}
```


A. Referência Rápida — Modelos e Parâmetros

Tabela de Modelos

Modelo	Input/1M	Output/1M	Thinking	Vision	Melhor para
v4-flash	\$0.07	\$0.27	Básico	Não	Chat, resumos, classificação
v4-pro	\$0.27	\$1.10	Profundo	Não	Código, debug, arquitetura
v4-flash-vision	\$0.07	\$0.27	Básico	Sim	Análise de imagens

Tabela de Parâmetros

Parâmetro	Faixa	Efeito	Recomendação
temperature	0.0-2.0	Aleatoriedade	0.0-0.3 (código), 0.5-0.7 (raciocínio), 0.7-1.0 (criativo)
top_p	0.0-1.0	Nucleus sampling	0.95 (padrão)
max_tokens	1-32768	Tamanho da resposta	4096 (padrão), 8192 (respostas longas)
reasoning_effort	low/medium/high	Profundidade do raciocínio	high (problemas complexos)
stream	true/false	Streaming	true (UX), false (batch)

Exemplos por Tarefa

```
# Classificação (determinístico)
{"temperature": 0.0, "top_p": 1.0}

# Geração de código (quase determinístico)
{"temperature": 0.1, "top_p": 0.95}
```

```
# Raciocínio matemático
{"temperature": 0.6, "top_p": 0.95, "reasoning_effort": "high"}

# Chat criativo
{"temperature": 0.8, "top_p": 0.95}

# Resumo
{"temperature": 0.3, "top_p": 0.95}

# Tradução
{"temperature": 0.0, "top_p": 1.0}
```

Limites da API

Limite	Valor	Observação
Max tokens entrada	128K	v4-pro, v4-flash
Max tokens saída	8K-32K	Depende do modelo
Requests/minuto	~60	Conta gratuita
Requests/dia	~10.000	Conta gratuita
Tamanho máximo prompt	128K tokens	Inclui system + user + assistant

Dicas de Otimização

1. Use **few-shot** para tarefas de formatação — 2-3 exemplos são suficientes
2. **Ative thinking** apenas para problemas complexos — custo adicional
3. **Cacheie respostas** para queries repetitivas — reduz custo em 30-50%
4. **Roteie modelos** — flash para simples, pro para complexo
5. **Limite max_tokens** — respostas curtas = menos custo
6. Use **system prompts** para V4, não para R1
7. **Evite temperature alta** para código — 0.0-0.2 é ideal
8. **Monitore custos** — implemente tracking desde o início.

Parte II — Domínio — As Estações Avançadas

Capítulo 5: Arquitetura do DeepSeek Harness

1. Introdução

Nos primeiros quatro capítulos, você explorou as estações básicas do laboratório: modelos, API, integrações e prompts. Agora é hora de abrir a máquina e entender como ela funciona por dentro. O DeepSeek Harness não é apenas uma interface para os modelos — é um framework de agentes completo, construído sobre o meta-framework Cordis, com uma arquitetura onde “tudo é um plugin” [1].

Essa arquitetura é o que permite ao Harness orquestrar agentes complexos, gerenciar memória, integrar ferramentas externas e escalar de forma modular. Cada funcionalidade é um plugin independente que pode ser carregado, descarregado e substituído em tempo de execução. Entender essa arquitetura é o que separa um usuário casual de um engenheiro que constrói soluções robustas com DeepSeek [1].

2. Explica

Cordis: O Meta-Framework

O Cordis é descrito como “A Programming Paradigm for Spatiotemporal Composability” — um paradigma de programação para composição espaço-temporal [2]. Em termos práticos, isso significa que o Cordis permite compor componentes de software de forma que eles possam ser conectados, desconectados e reconectados dinamicamente, tanto no espaço (diferentes partes do sistema) quanto no tempo (diferentes momentos da execução).

Para o Harness, isso significa que cada funcionalidade — chat, ferramentas, memória, MCP, integrações — é um plugin independente que pode ser carregado, descarregado e substituído em tempo de execução [1]. Se você precisa de um novo tipo de ferramenta, basta criar um

plugin. Se um plugin falha, os outros continuam funcionando. Essa resiliência é fundamental para sistemas de IA em produção.

O Cordis implementa o conceito de “context” — um objeto que contém o estado compartilhado entre todos os plugins. Cada plugin pode ler e escrever no contexto, permitindo comunicação indireta entre componentes. Essa abordagem elimina o acoplamento direto entre plugins [2].

A Arquitetura “Everything is a Plugin”

No Harness, absolutamente tudo é um plugin:

- **Interface de chat** — plugin que gerencia a conversa com o usuário
- **Modelo de IA** — plugin que se conecta à API DeepSeek
- **Ferramentas** — plugins que executam ações (ler arquivos, rodar código, etc.)
- **Memória** — plugin que mantém contexto entre mensagens
- **MCP** — plugin que se conecta a servidores MCP externos
- **Sistema de arquivos** — plugin que gerencia operações de arquivo
- **Logging** — plugin que registra atividades para debug e auditoria
- **Rate limiting** — plugin que controla a taxa de requisições

Cada plugin tem um ciclo de vida definido: inicialização, ativação, execução e desativação. Plugins podem depender de outros plugins e podem ser compostos para criar funcionalidades complexas [1].

Web UI e CLI

O Harness oferece duas interfaces principais:

Web UI — Interface visual que roda em <http://127.0.0.1:3080>. Permite conversar com os modelos, gerenciar plugins e visualizar o estado do sistema. Ideal para uso interativo e debug. A Web UI é construída com tecnologias web modernas e oferece uma experiência rica com syntax highlighting, streaming de respostas e gerenciamento de conversas [1].

CLI — Interface de linha de comando para automação e integração com scripts. Permite executar o Harness como parte de pipelines CI/CD. O CLI é mais leve que a Web UI e pode ser facilmente integrado em scripts de automação [1].

Gerenciamento de Estado

O Harness mantém o estado em memória durante a execução. Cada plugin pode persistir dados que sobrevivem entre mensagens, mas não entre reinicializações. Para persistência de longo prazo, o Harness depende de plugins externos (SQLite, arquivos, etc.) [1].

O estado é organizado em camadas:

1. **Estado global** — Compartilhado entre todos os plugins (configurações, preferências)
2. **Estado por sessão** — Específico de cada conversa (histórico, contexto)
3. **Estado por plugin** — Dados privados de cada plugin (cache, contadores)

Essa organização permite que plugins sejam recarregados sem perder o estado da sessão, e que sessões sejam persistidas para retomada posterior [1].

Comunicação entre Plugins

Plugins se comunicam através de dois mecanismos:

EventBus — Sistema de pub/sub onde plugins publicam eventos e outros plugins se inscrevem para recebê-los. É ideal para comunicação desacoplada — o plugin que publica não precisa saber quem vai receber o evento [1].

Context compartilhado — Objeto compartilhado onde plugins podem ler e escrever dados diretamente. É mais rápido que EventBus mas cria acoplamento entre plugins [1].

A recomendação é usar EventBus para comunicação entre plugins desconhecidos e Context para dados compartilhados entre plugins que trabalham juntos [1].

3. Ilustra

A Central de Comando

O Harness é como uma central de comando onde cada estação de trabalho (plugin) se conecta a um barramento comum. Quando você envia uma mensagem, ela percorre o barramento e cada plugin decide se quer processá-la [1].

```
%% legenda: Arquitetura de plugins do DeepSeek Harness
flowchart TD
    A[Usuário] --> B[Web UI / CLI]
    B --> C[Barramento Cordis]
    C --> D[Plugin Chat]
    C --> E[Plugin Modelo]
    C --> F[Plugin Ferramentas]
    C --> G[Plugin Memória]
    C --> H[Plugin MCP]
    C --> I[Plugin Logging]
    E --> J[API DeepSeek]
```

```
F --> K[Arquivos / Terminal]
H --> L[Servers MCP Externos]
G --> M[Persistência<br/>SQLite/Arquivos]
```

A elegância dessa arquitetura é que você pode desligar qualquer plugin sem quebrar o sistema. Não precisa de memória? Desligue o plugin de memória. Não precisa de MCP? O Harness funciona perfeitamente sem ele. Essa modularidade é o que torna o Harness extensível e resiliente [1].

Fluxo de uma Mensagem

Quando você envia uma mensagem, ela percorre esta pipeline:

1. **Entrada** — A mensagem chega via Web UI ou CLI
2. **Pré-processamento** — Plugins de validação e formatação processam a mensagem
3. **Roteamento** — O barramento direciona a mensagem para os plugins relevantes
4. **Processamento** — Cada plugin processa a mensagem conforme sua função
5. **Resposta** — O plugin de modelo gera a resposta usando a API DeepSeek
6. **Pós-processamento** — Plugins de logging, memória e formatação processam a resposta
7. **Saída** — A resposta é enviada de volta ao usuário

Essa pipeline é flexível — plugins podem ser inseridos em qualquer ponto, podem modificar a mensagem ou a resposta, e podem até bloquear o processamento [1].

4. Técnica

Estrutura de um Plugin

Um plugin no Harness é um módulo TypeScript que exporta uma interface específica:

```
// Estrutura básica de um plugin
interface Plugin {
  name: string;
  version: string;

  // Ciclo de vida
  activate(context: Context): void;
  deactivate(): void;

  // Processamento de mensagens
```

```

onMessage(message: Message): Promise<Response>;

// Hooks opcionais
onBeforeSend?(message: Message): Message;
onAfterReceive?(response: Response): Response;
}

// Exemplo: Plugin de contagem de tokens
export const tokenCounterPlugin: Plugin = {
  name: "token-counter",
  version: "1.0.0",

  private totalTokens = 0,
  private totalCusto = 0,

  activate(context: Context) {
    console.log("Plugin token-counter ativado");
    // Lê configuração do contexto
    const config = context.config.get("token-counter") || {};
    this.logInterval = config.logInterval || 100;
  },

  deactivate() {
    console.log(`\n=== Relatório de Tokens ===`);
    console.log(`Total de tokens: ${this.totalTokens}`);
    console.log(`Custo estimado: ${this.totalCusto.toFixed(4)}`);
  },

  async onMessage(message: Message) {
    // Conta tokens aproximados (1 token ≈ 4 caracteres em PT-BR)
    const tokensEntrada = Math.ceil(message.content.length / 4);
    this.totalTokens += tokensEntrada;

    // Calcula custo estimado
    const custoEntrada = (tokensEntrada / 1_000_000) * 0.07; // v4-flash
    this.totalCusto += custoEntrada;

    // Log periódico
    if (this.totalTokens % this.logInterval === 0) {
      console.log(`[tokens] ${this.totalTokens} tokens, $
${this.totalCusto.toFixed(4)}`);
    }

    // Passa a mensagem adiante sem modificação
    return { continue: true };
  }
};

```


Plugin com Persistência

```
// Plugin que salva histórico de conversas
export const historicoPlugin: Plugin = {
  name: "historico",
  version: "1.0.0",

  activate(context: Context) {
    this.storage = context.storage;
    this.logger = context.logger;
  },

  async onMessage(message: Message) {
    const chatId = message.chatId || "default";

    // Lê histórico existente
    const historico = await this.storage.get(`historico:${chatId}`) || [];

    // Adiciona mensagem do usuário
    historico.push({
      role: "user",
      content: message.content,
      timestamp: Date.now(),
      tokens: Math.ceil(message.content.length / 4)
    });

    // Mantém apenas as últimas 100 mensagens
    if (historico.length > 100) {
      historico.splice(0, historico.length - 100);
    }

    // Persiste
    await this.storage.set(`histórico:${chatId}`, historico);

    // Log
    this.logger.info(`Chat ${chatId}: ${historico.length} mensagens`);

    return { continue: true };
  }
};
```

Plugin com Rate Limiting

```
// Plugin que controla taxa de requisições
export const rateLimitPlugin: Plugin = {
```

```

name: "rate-limit",
version: "1.0.0",

private requestCounts: Map<string, number[]> = new Map(),
private readonly MAX_REQUESTS = 60, // por minuto
private readonly WINDOW_MS = 60 * 1000, // 1 minuto

activate(context: Context) {
  this.logger = context.logger;
},

async onMessage(message: Message) {
  const userId = message.userId || "anonymous";
  const now = Date.now();

  // Obtém timestamps de requests anteriores
  const timestamps = this.requestCounts.get(userId) || [];

  // Remove requests fora da janela
  const validTimestamps = timestamps.filter(t => now - t <
this.WINDOW_MS);

  if (validTimestamps.length >= this.MAX_REQUESTS) {
    this.logger.warn(`Rate limit atingido para ${userId}`);
    return {
      continue: false,
      response: "⌚ Você atingiu o limite de requests. Aguarde 1 minuto."
    };
  }

  // Registra nova request
  validTimestamps.push(now);
  this.requestCounts.set(userId, validTimestamps);

  return { continue: true };
}
};

```

Carregando Plugins

```

# Inicie o Harness com plugins específicos
npx @deepseek-ai/dsh web --plugins token-counter,historico,rate-limit

# Ou configure no arquivo de configuração
{

```

```

"plugins": {
  "token-counter": { "enabled": true, "logInterval": 50 },
  "historico": { "enabled": true },
  "rate-limit": { "enabled": true, "maxRequests": 60 },
  "filesystem": { "enabled": true, "root": "./meu-projeto" },
  "mcp": { "enabled": true, "servers": ["filesystem", "github"] }
}
}

```

Integração com MCP

O Harness suporta o Model Context Protocol (MCP) nativamente. Para conectar a um servidor MCP:

```

// Configuração de MCP no Harness
const mcpConfig = {
  servers: {
    "filesystem": {
      command: "npx",
      args: ["-y", "@modelcontextprotocol/server-filesystem", "/path/to/
dir"]
    },
    "github": {
      command: "npx",
      args: ["-y", "@modelcontextprotocol/server-github"],
      env: { GITHUB_TOKEN: process.env.GITHUB_TOKEN }
    },
    "postgres": {
      command: "npx",
      args: ["-y", "@modelcontextprotocol/server-postgres"],
      env: { DATABASE_URL: process.env.DATABASE_URL }
    }
  }
}
};

```

5. Aplica

O Problema da Monolitidade

Sua empresa tem um assistente de código que faz tudo: chat, revisão de PR, geração de testes, deploy. Mas quando um componente falha (o gerador de testes), todo o sistema para. A arquitetura monolítica é frágil — um bug em um componente pode derrubar todo o sistema [1].

O Harness resolve isso com sua arquitetura de plugins. Cada funcionalidade é um plugin independente. Se o gerador de testes falhar, o chat continua funcionando. Se o plugin de deploy quebrar, a revisão de PR não é afetada. Essa resiliência é crucial para sistemas em produção [1].

```
# Simulação do padrão de plugins do Harness em Python
class PluginManager:
    def __init__(self):
        self.plugins = {}
        self.estado = {}

    def registrar(self, nome, plugin):
        self.plugins[nome] = plugin
        print(f" Plugin {nome} registrado")

    async def processar(self, mensagem):
        respostas = []
        for nome, plugin in self.plugins.items():
            try:
                resultado = await plugin.on_message(mensagem, self.estado)
                if resultado.get("continue", True):
                    respostas.append(resultado)
            else:
                print(f" Plugin {nome} bloqueou a mensagem")
                return resultado.get("response", "Erro")
        except Exception as e:
            print(f" ⚠ Plugin {nome} falhou: {e}")
            continue # Outros plugins continuam
        return respostas

# Exemplo de uso
manager = PluginManager()
manager.registrar("token-counter", TokenCounterPlugin())
manager.registrar("historico", HistoricoPlugin())
manager.registrar("rate-limit", RateLimitPlugin())
```

Exercício

- ☐ Instale o DeepSeek Harness e explore a Web UI
- ☐ Identifique quais plugins estão ativos por padrão
- ☐ Crie um plugin simples que loga todas as mensagens
- ☐ Configure o Harness com um servidor MCP de filesystem
- ☐ Teste a desconexão de um plugin e verifique que o sistema continua funcionando
- ☐ Implemente o PluginManager em Python e teste com 3 plugins

6. Conclusão

Neste capítulo, você abriu a máquina e viu como o DeepSeek Harness funciona por dentro. O meta-framework Cordis permite uma arquitetura de plugins modular e flexível, onde cada funcionalidade é independente e substituível. Essa arquitetura é o que torna o Harness extensível — você pode adicionar novas capacidades sem modificar o core [1].

Você viu que plugins se comunicam através de EventBus e Context compartilhado, e que cada plugin tem um ciclo de vida definido (activate → onMessage → deactivate). A arquitetura “Everything is a Plugin” é elegante e resiliente — um plugin que falha não derruba o sistema [1].

No próximo capítulo, você vai além de usar os plugins prontos: aprender a criar seus próprios plugins TypeScript para estender o Harness com funcionalidades personalizadas.

7. Referências Bibliográficas

[1] DEEPSEEK-AI. *DeepSeek Harness: Everything is a Plugin*. Disponível em: <https://github.com/deepseek-ai/deepseek-harness>. Acesso em: 21 ago. 2026.

[2] CORDIVERSE. *Cordis: A Meta-Framework of Spatiotemporal Composability*. Disponível em: <https://github.com/cordiverse/cordis>. Acesso em: 21 ago. 2026.

[3] DEEPSEEK-AI. *DeepSeek API Documentation*. Disponível em: <https://api-docs.deepseek.com/>. Acesso em: 21 ago. 2026.

[4] DEEPSEEK-AI. *DeepSeek-V3 Technical Report*. Disponível em: <https://arxiv.org/abs/2412.19437>. Acesso em: 21 ago. 2026.

[5] DEEPSEEK-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Disponível em: <https://arxiv.org/abs/2501.12948>. Acesso em: 21 ago. 2026.

- [6] DEEPSEEK-AI. *Awesome DeepSeek Agent*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-agent>. Acesso em: 21 ago. 2026.
- [7] DEEPSEEK-AI. *Awesome DeepSeek Integration*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-integration>. Acesso em: 21 ago. 2026.
- [8] DEEPSEEK-AI. *FlashMLA: Efficient Multi-head Latent Attention Kernels*. Disponível em: <https://github.com/deepseek-ai/FlashMLA>. Acesso em: 21 ago. 2026.
- [9] DEEPSEEK-AI. *DeepEP: An Efficient Expert-Parallel Communication Library*. Disponível em: <https://github.com/deepseek-ai/DeepEP>. Acesso em: 21 ago. 2026.
- [10] DEEPSEEK-AI. *DeepGEMM: Clean and Efficient BLAS Kernel Library on GPU*. Disponível em: <https://github.com/deepseek-ai/DeepGEMM>. Acesso em: 21 ago. 2026.
- [11] DEEPSEEK-AI. *3FS: A High-Performance Distributed File System for AI*. Disponível em: <https://github.com/deepseek-ai/3FS>. Acesso em: 21 ago. 2026.
- [12] DEEPSEEK-AI. *DeepSpec: Full-Stack Codebase for Speculative Decoding*. Disponível em: <https://github.com/deepseek-ai/DeepSpec>. Acesso em: 21 ago. 2026.
- [13] DEEPSEEK-AI. *TileKernels: Kernel Library Written in Tilelang*. Disponível em: <https://github.com/deepseek-ai/TileKernels>. Acesso em: 21 ago. 2026.
- [14] DEEPSEEK-AI. *DeepSeek-OCR-2: Visual Causal Flow*. Disponível em: <https://github.com/deepseek-ai/DeepSeek-OCR-2>. Acesso em: 21 ago. 2026.
- [15] VASWANI, A. et al. *Attention Is All You Need*. In: Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [16] ANTHROPIC. *Model Context Protocol Specification*. Disponível em: <https://spec.modelcontextprotocol.io/>. Acesso em: 21 ago. 2026.
- [17] DEEPSEEK-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. Disponível em: <https://arxiv.org/abs/2405.04434>. Acesso em: 21 ago. 2026.
- [18] FEDUS, W. et al. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. In: Journal of Machine Learning Research, 2022.
- [19] DEEPSEEK-AI. *DeepSeek Platform — Pricing*. Disponível em: <https://platform.deepseek.com/>. Acesso em: 21 ago. 2026.
- [20] TYPESCRIPT. *TypeScript Handbook — Modules*. Disponível em: <https://www.typescriptlang.org/docs/handbook/modules/>. Acesso em: 21 ago. 2026.

8. Apêndice — Hooks e Middleware no Harness

Sistema de Hooks

O Harness permite registrar hooks que são executados em pontos específicos do ciclo de vida. Diferente dos plugins (que processam mensagens), os hooks são executados em eventos como inicialização, desativação, erro e mudança de estado [1].

```
// Exemplo de hooks no Harness
const hooks = {
  // Executado quando o Harness inicia
  onReady: async (context) => {
    console.log("Harness pronto!");
    await context.storage.init();
  },

  // Executado quando um plugin falha
  onPluginError: async (plugin, error, context) => {
    console.error(`Plugin ${plugin.name} falhou: ${error.message}`);
    // Tenta recuperar: recarrega o plugin
    await context.plugins.reload(plugin.name);
  },

  // Executado antes de enviar mensagem ao modelo
  onBeforeModelCall: async (messages, context) => {
    // Adiciona contexto de memória
    const memoria = await context.storage.get("memoria") || [];
    return [...memoria, ...messages];
  },

  // Executado depois de receber resposta do modelo
  onAfterModelCall: async (response, context) => {
    // Salva na memória
    const memoria = await context.storage.get("memoria") || [];
    memoria.push(response);
    if (memoria.length > 50) memoria.shift();
    await context.storage.set("memoria", memoria);
    return response;
  }
};
```

Middleware Pattern

O Harness suporta o padrão middleware, onde múltiplos componentes processam a mensagem em cascata:

```
// Middleware de logging
const loggingMiddleware = async (message, next) => {
  console.log(`[${new Date().toISOString()}] Mensagem recebida:
${message.content.substring(0, 50)}...`);
  const start = Date.now();
  const response = await next(message);
  console.log(`[${new Date().toISOString()}] Resposta gerada em
${Date.now() - start}ms`);
  return response;
};

// Middleware de validação
const validationMiddleware = async (message, next) => {
  if (message.content.length > 10000) {
    return { error: "Mensagem muito longa (máx: 10.000 caracteres)" };
  }
  return next(message);
};

// Registro dos middlewares
harness.use(loggingMiddleware);
harness.use(validationMiddleware);
```

9. Apêndice — Debugging de Plugins

Logging Estruturado

```
// Plugin com logging detalhado para debug
export const debugPlugin: Plugin = {
  name: "debug-logger",
  version: "1.0.0",

  activate(context: Context) {
    this.logger = context.logger;
    this.logger.info("Plugin debug ativado", {
      plugin: this.name,
    });
  }
};
```



```

        version: this.version,
        timestamp: new Date().toISOString()
    });
},

async onMessage(message: Message) {
    const inicio = Date.now();

    this.logger.debug("Mensagem recebida", {
        userId: message.userId,
        contentLength: message.content.length,
        timestamp: new Date().toISOString()
    });

    try {
        const resultado = await this.processar(message);
        const duracao = Date.now() - inicio;

        this.logger.info("Mensagem processada", {
            duracao,
            resultado: typeof resultado,
            timestamp: new Date().toISOString()
        });

        return resultado;
    } catch (error) {
        this.logger.error("Erro no processamento", {
            error: error.message,
            stack: error.stack,
            mensagem: message.content.substring(0, 100),
            timestamp: new Date().toISOString()
        });

        // Não propaga o erro – outros plugins continuam
        return { continue: true };
    }
}
};

```

Métricas de Plugin

```

// Plugin que coleta métricas
export const metricsPlugin: Plugin = {
    name: "metrics",
    version: "1.0.0",

```

```

private metrics: Map<string, number[]> = new Map(),

async onMessage(message: Message) {
  const inicio = Date.now();

  // Mede tempo de processamento
  const metricas = {
    "mensagens_processadas": 1,
    "tokens_entrada": Math.ceil(message.content.length / 4),
    "timestamp": Date.now()
  };

  // Acumula métricas
  for (const [chave, valor] of Object.entries(metricas)) {
    const atual = this.metrics.get(chave) || [];
    atual.push(valor);
    this.metrics.set(chave, atual.slice(-1000)); // Últimas 1000
  }

  return { continue: true };
},

getMetricas() {
  const resultado: any = {};
  for (const [chave, valores] of this.metrics) {
    resultado[chave] = {
      total: valores.reduce((a, b) => a + b, 0),
      media: valores.reduce((a, b) => a + b, 0) / valores.length,
      ultimo: valores[valores.length - 1],
      count: valores.length
    };
  }
  return resultado;
}
};

```

A. Padrões Avançados de Arquitetura

Plugin Composition

```
// Composição de plugins para funcionalidade complexa
export const pipelinePlugin: Plugin = {
  name: "pipeline",
  version: "1.0.0",

  private plugins: Plugin[] = [],

  addPlugin(plugin: Plugin) {
    this.plugins.push(plugin);
  },

  async onMessage(message: Message) {
    let resultado = message;

    // Executa cada plugin em sequência
    for (const plugin of this.plugins) {
      const response = await plugin.onMessage(resultado);
      if (!response.continue) {
        return response;
      }
      resultado = { ...resultado, ...response };
    }

    return resultado;
  }
};

// Uso
const pipeline = new pipelinePlugin();
pipeline.addPlugin(new RateLimitPlugin());
pipeline.addPlugin(new ValidationPlugin());
pipeline.addPlugin(new LoggingPlugin());
```

Event-Driven Architecture

```
// Arquitetura orientada a eventos
export const eventPlugin: Plugin = {
```

```

name: "event-driven",
version: "1.0.0",

activate(context: Context) {
  // Escuta eventos
  context.eventBus.on("user:login", async (evento) => {
    console.log(`Usuário logado: ${evento.userId}`);
    await this.registrarAtividade(evento);
  });

  context.eventBus.on("message:error", async (evento) => {
    console.error(`Erro na mensagem: ${evento.error}`);
    await this.notificarAdmin(evento);
  });
},

async onMessage(message: Message) {
  // Publica evento
  this.context.eventBus.emit("message:received", {
    userId: message.userId,
    content: message.content,
    timestamp: Date.now()
  });

  return { continue: true };
}
};

```

State Machine Pattern

```

// Máquina de estados para fluxos complexos
export const stateMachinePlugin: Plugin = {
  name: "state-machine",
  version: "1.0.0",

  private estados: Map<string, any> = new Map(),
  private transicoes: Map<string, Function> = new Map(),

  definirTransicao(de: string, para: string, condicao: Function) {
    this.transicoes.set(`${de}->${para}`, condicao);
  },

  async onMessage(message: Message) {
    const userId = message.userId;
    const estadoAtual = this.estados.get(userId) || "inicio";

```

```

// Verifica transições possíveis
for (const [chave, condicao] of this.transicoes) {
  const [de, para] = chave.split("->");
  if (de === estadoAtual && await condicao(message)) {
    this.estados.set(userId, para);
    console.log(`Transição: ${de} -> ${para}`);
    break;
  }
}

return { continue: true, estado: this.estados.get(userId) }
}
};

```

B. Padrão Observer para Monitoramento

Plugin de Métricas Completo

```

// Plugin que coleta métricas detalhadas
export const metricsCollectorPlugin: Plugin = {
  name: "metrics-collector",
  version: "1.0.0",

  private startTime: number = 0,
  private messageCount: number = 0,
  private errorCount: number = 0,
  private latencies: number[] = [],

  activate(context: Context) {
    this.startTime = Date.now();
    this.logger = context.logger;
    this.logger.info("Metrics collector ativado");
  },

  async onMessage(message: Message) {
    const inicio = Date.now();
    this.messageCount++;

    // Após processamento (via hook)
    const duracao = Date.now() - inicio;
  }
};

```

```

    this.latencies.push(duracao);

    // Mantém apenas últimas 1000 latências
    if (this.latencies.length > 1000) {
        this.latencies.shift();
    }

    return { continue: true };
},

onError(error: Error) {
    this.errorCount++;
    this.logger.error("Erro registrado", { error: error.message });
},

getMetrics() {
    const sorted = [...this.latencies].sort((a, b) => a - b);
    const uptime = Date.now() - this.startTime;

    return {
        uptime: uptime,
        messageCount: this.messageCount,
        errorCount: this.errorCount,
        errorRate: this.errorCount / this.messageCount,
        latency: {
            p50: sorted[Math.floor(sorted.length * 0.5)] || 0,
            p95: sorted[Math.floor(sorted.length * 0.95)] || 0,
            p99: sorted[Math.floor(sorted.length * 0.99)] || 0,
            media: this.latencies.reduce((a, b) => a + b, 0) /
this.latencies.length || 0
        }
    };
}
};

```

Capítulo 6: Desenvolvimento de Plugins para o Harness

1. Introdução

No capítulo anterior, você entendeu a arquitetura de plugins do DeepSeek Harness — como o Cordis permite composição modular e como cada funcionalidade é um plugin independente. Agora é hora de colocar as mãos na massa e criar seus próprios plugins. Desenvolver um plugin para o Harness é como adicionar uma nova estação de trabalho ao laboratório: você define o que ela faz, como se conecta às outras, e ela passa a fazer parte do ecossistema [1].

O ciclo de vida de um plugin é direto: desenvolvimento, teste, publicação e descoberta. A tag `dsh-plugin` no repositório GitHub permite que outros desenvolvedores encontrem seu plugin automaticamente. Neste capítulo, você vai criar um plugin do zero, testá-lo localmente e prepará-lo para publicação. Também verá padrões de design comuns e anti-patterns a evitar.

2. Explica

Estrutura de um Plugin

Um plugin TypeScript para o Harness segue uma estrutura padrão [1]:

```
meu-plugin/  
├─ package.json      # Metadados e dependências  
├─ tsconfig.json     # Configuração TypeScript  
├─ src/  
│   └─ index.ts      # Entry point do plugin  
├─ tests/  
│   └─ index.test.ts # Testes unitários  
├─ README.md         # Documentação  
└─ .npmignore        # Arquivos ignorados no publish
```

O `package.json` deve incluir a tag `dsh-plugin` para descoberta:

```
{
  "name": "@deepseek-ai/dsh-plugin-meu-plugin",
  "version": "1.0.0",
  "description": "Plugin que faz X para o DeepSeek Harness",
  "keywords": ["dsh-plugin", "deepseek", "harness", "plugin"],
  "main": "dist/index.js",
  "types": "dist/index.d.ts",
  "scripts": {
    "build": "tsc",
    "test": "vitest",
    "lint": "eslint src/",
    "prepublishOnly": "npm run build"
  },
  "dependencies": {},
  "devDependencies": {
    "typescript": "^5.0.0",
    "vitest": "^1.0.0"
  },
  "peerDependencies": {
    "@deepseek-ai/dsh": ">=1.0.0"
  }
}
```

Ciclo de Vida de um Plugin

Cada plugin passa por quatro estágios [1]:

Inicialização — O plugin é carregado e configurado. Aqui você deve validar configurações, estabelecer conexões e preparar recursos. Se a inicialização falhar, o plugin não é carregado.

Ativação — O plugin começa a processar mensagens. Ele se registra no barramento do Cordis e começa a interceptar eventos. Neste estágio, o plugin pode se inscrever em eventos específicos ou interceptar todas as mensagens.

Execução — O plugin processa mensagens do usuário e retorna respostas. Esta é a fase principal de trabalho. O plugin pode modificar a mensagem, gerar uma resposta, ou simplesmente passar a mensagem adiante.

Desativação — O plugin é descarregado. Aqui você deve liberar recursos (conexões, handles de arquivo), persistir dados se necessário, e realizar limpeza. O Harness pode descarregar e recarregar plugins em tempo de execução [1].

APIs Disponíveis

Os plugins têm acesso a diversas APIs do Harness [1]:

- **Context** — Acesso ao estado global do sistema. Permite ler e escrever configurações, preferências e dados compartilhados.
- **Storage** — Persistência chave-valor. Dados persistidos sobrevivem entre mensagens mas não entre reinicializações (a menos que o plugin use storage externo).
- **Logger** — Sistema de logging estruturado com níveis (debug, info, warn, error).
- **EventBus** — Pub/sub para comunicação entre plugins. Permite desacoplamento entre componentes.
- **HttpClient** — Requisições HTTP para APIs externas. Suporta retry automático e timeout configurável.
- **FileSystem** — Operações de arquivo seguras com sandbox.

Publicação e Descoberta

Para publicar um plugin, basta adicionar a tag `dsh-plugin` ao repositório GitHub. O Harness descobre plugins automaticamente através dessa tag [1].

```
# Publique no npm
npm publish

# Adicione a tag dsh-plugin no GitHub
gh repo edit deepseek-ai/dsh-plugin-meu-plugin --add-topic dsh-plugin
```

O processo de descoberta funciona assim: 1. O Harness consulta a API do GitHub por repositórios com a tag `dsh-plugin` 2. Para cada repositório encontrado, lê o `package.json` para obter nome, versão e descrição 3. O plugin aparece na lista de plugins disponíveis na Web UI 4. O usuário pode instalar o plugin com um clique

Padrões de Design

Plugin de Processamento — Intercepta mensagens e as modifica antes de enviar para o modelo. Ex.: tradução automática, sanitização de input, compressão de contexto.

Plugin de Ferramenta — Adiciona novas capacidades ao modelo. Ex.: acesso a banco de dados, execução de código, consulta a APIs externas.

Plugin de Observer — Monitora o sistema sem modificar mensagens. Ex.: logging, métricas, auditoria, rate limiting.

Plugin de Hook — Executa ações em pontos específicos do ciclo de vida. Ex.: persistência automática, notificações, cleanup.

3. Ilustra

A Estação Personalizada

Criar um plugin é como projetar uma estação de trabalho personalizada para o laboratório. Você define a entrada (dados que recebe), o processamento (o que faz com eles) e a saída (resultado que produz). A estação se conecta ao laboratório através do barramento (Cordis) e pode se comunicar com outras estações (outros plugins) [1].

```
%% legenda: Ciclo de vida de um plugin no Harness
flowchart LR
    A[Código Fonte] --> B[Build<br/>tsc]
    B --> C[Init<br/>Configuração]
    C --> D[Activate<br/>Registro no Cordis]
    D --> E[Execute<br/>Processamento]
    E --> F{Desativar?}
    F -->|Não| E
    F -->|Sim| G[Deactivate<br/>Liberação]
    G --> H[Publicar<br/>npm + tag dsh-plugin]
    H --> I[Descoberta<br/>GitHub API]
```

4. Técnica

Criando um Plugin Completo

Vamos criar um plugin que valida automaticamente código Python antes de enviar para o repositório:

```
// src/index.ts
import { Plugin, Context, Message, Response } from "@deepseek-ai/dsh";

interface ValidadorConfig {
  linguagens: string[];
  maxLinhas: number;
  verificarSintaxe: boolean;
}
```

```

export const validadorCodigoPlugin: Plugin = {
  name: "python-validator",
  version: "1.0.0",

  private config: ValidadorConfig = {
    linguagens: ["python"],
    maxLinhas: 500,
    verificarSintaxe: true
  },

  private storage: any,
  private logger: any,

  activate(context: Context) {
    this.storage = context.storage;
    this.logger = context.logger;

    // Lê configuração
    const configUsuario = context.config.get("python-validator");
    if (configUsuario) {
      this.config = { ...this.config, ...configUsuario };
    }

    this.logger.info(`Plugin python-validator ativado:
    ${this.config.linguagens.join(", ")}`);
  },

  deactivate() {
    this.logger.info("Plugin python-validator desativado");
  },

  async onMessage(message: Message): Promise<Response> {
    // Detecta blocos de código Python na mensagem
    const regex = /``python\n([\s\S]*?)``/g;
    const blocos = [];
    let match;

    while ((match = regex.exec(message.content)) !== null) {
      blocos.push(match[1]);
    }

    if (blocos.length === 0) {
      return { continue: true };
    }

    // Valida cada bloco
    const resultados = [];
  
```

```

for (const codigo of blocos) {
  const resultado = await this.validar(codigo);
  resultados.push(resultado);
}

// Salva histórico de validações
const historico = await this.storage.get("validacoes") || [];
historico.push({
  timestamp: Date.now(),
  blocos: blocos.length,
  erros: resultados.filter(r => !r.valido).length
});
await this.storage.set("validacoes", historico.slice(-100));

// Se houver erros, retorna relatório
const erros = resultados.filter(r => !r.valido);
if (erros.length > 0) {
  const relatorio = erros.map((e, i) =>
    `Bloco ${i + 1}: ${e.erros.join(", ")}`
  ).join("\n");

  return {
    continue: true,
    附加: { validacao: resultados },
    response: `⚠ Validação encontrou erros:\n${relatorio}`
  };
}

return { continue: true };
},

async validar(codigo: string) {
  const erros: string[] = [];

  // 1. Verifica tamanho
  const linhas = codigo.split("\n").length;
  if (linhas > this.config.maxLinhas) {
    erros.push(`Código muito longo: ${linhas} linhas (máx:
    ${this.config.maxLinhas})`);
  }

  // 2. Verifica sintaxe básica
  if (this.config.verificarSintaxe) {
    try {
      // Tenta parsear como Python
      const regrasSintaxe = [
        { regex: /def\s+\w+\s*\(/, msg: "Função definida" },

```

```

    { regex: /class\s+\w+/, msg: "Classe definida" },
    { regex: /import\s+\w+/, msg: "Import encontrado" },
  ];

  // Verifica balanceamento de parênteses
  const parenCount = (codigo.match(/\(/g) || []).length -
    (codigo.match(/\)/g) || []).length;
  if (parenCount !== 0) {
    erros.push("Parênteses desbalanceados");
  }

  // Verifica balanceamento de colchetes
  const bracketCount = (codigo.match(/\[/g) || []).length -
    (codigo.match(/\]/g) || []).length;
  if (bracketCount !== 0) {
    erros.push("Colchetes desbalanceados");
  }

  // Verifica balanceamento de chaves
  const braceCount = (codigo.match(/\{/g) || []).length -
    (codigo.match(/\}/g) || []).length;
  if (braceCount !== 0) {
    erros.push("Chaves desbalanceadas");
  }

} catch (e) {
  erros.push(`Erro de sintaxe: ${e.message}`);
}

}

return {
  valido: erros.length === 0,
  erros,
  linhas,
  timestamp: Date.now()
};
}
};

```

Testes

```

// tests/index.test.ts
import { describe, it, expect, vi } from "vitest";
import { validadorCodigoPlugin } from "../src/index";

```

```

describe("python-validator", () => {
  it("deve ter nome e versão", () => {
    expect(validadorCodigoPlugin.name).toBe("python-validator");
    expect(validadorCodigoPlugin.version).toBe("1.0.0");
  });

  it("deve ter ciclo de vida completo", () => {
    expect(typeof validatorCodigoPlugin.activate).toBe("function");
    expect(typeof validatorCodigoPlugin.deactivate).toBe("function");
    expect(typeof validatorCodigoPlugin.onMessage).toBe("function");
  });

  it("deve validar código Python válido", async () => {
    const resultado = await validatorCodigoPlugin.validar(
      "def hello():\n    print('hello')"
    );
    expect(resultado.valido).toBe(true);
    expect(resultado.erros).toHaveLength(0);
  });

  it("deve detectar parênteses desbalanceados", async () => {
    const resultado = await validatorCodigoPlugin.validar(
      "def hello(\n    print('hello')"
    );
    expect(resultado.valido).toBe(false);
    expect(resultado.erros).toContain("Parênteses desbalanceados");
  });

  it("deve detectar código muito longo", async () => {
    const codigoLongo = "x = 1\n".repeat(600);
    const resultado = await validatorCodigoPlugin.validar(codigoLongo);
    expect(resultado.valido).toBe(false);
    expect(resultado.erros[0]).toContain("muito longo");
  });
});

```

Publicando o Plugin

```

# 1. Build
npm run build

# 2. Teste
npm test

# 3. Publique no npm

```

```
npm publish
```

```
# 4. Adicione a tag dsh-plugin no GitHub
```

```
gh repo edit <seu-usuario>/dsh-plugin-python-validator --add-topic dsh-plugin
```

5. Aplica

O Plugin que Faltava

Sua empresa precisa de um plugin que valide automaticamente código antes de enviar para o repositório. Sem esse plugin, bugs passam para produção. O plugin deve rodar verificações de sintaxe, verificar formatação e retornar um relatório [1].

```
// Plugin de validação de código multi-linguagem
export const validadorMultiPlugin: Plugin = {
  name: "multi-validator",
  version: "1.0.0",

  async onMessage(message: Message) {
    const blocos = this.extrairBlocos(message.content);
    if (blocos.length === 0) return { continue: true };

    const relatorios = [];
    for (const bloco of blocos) {
      const resultado = await this.validarBloco(bloco);
      relatorios.push(resultado);
    }

    const erros = relatorios.filter(r => !r.valido);
    if (erros.length > 0) {
      return {
        continue: true,
        response: this.gerarRelatorio(erros)
      };
    }

    return { continue: true };
  },

  extrairBlocos(conteudo: string) {
```

```

const regex = /```(\w+)\n([\s\S]*?)```/g;
const blocos = [];
let match;
while ((match = regex.exec(conteudo)) !== null) {
  blocos.push({ linguagem: match[1], codigo: match[2] });
}
return blocos;
},

async validarBloco(bloco: { linguagem: string; codigo: string }) {
  const erros: string[] = [];

  // Validação genérica
  const linhas = bloco.codigo.split("\n").length;
  if (linhas > 500) {
    erros.push(`Código muito longo: ${linhas} linhas`);
  }

  // Validação por linguagem
  if (bloco.linguagem === "python") {
    // Verifica indentação
    const linhasComErro = bloco.codigo.split("\n")
      .filter(l => l.trim() && !l.match(/^[\s]*[^\s]/));
    if (linhasComErro.length > 0) {
      erros.push("Indentação inconsistente detectada");
    }
  }

  return {
    linguagem: bloco.linguagem,
    valido: erros.length === 0,
    erros
  };
},

gerarRelatorio(erros: any[]) {
  return `⚠ Validação encontrou ${erros.length} bloco(s) com erros:\n` +
    erros.map(e => `- ${e.linguagem}: ${e.erros.join(", ")}\n`).join("\n");
}
};

```

Exercício

- ☐ Crie um plugin que conta tokens de cada mensagem
- ☐ Adicione persistência para salvar histórico de conversas
- ☐ Escreva testes unitários para seu plugin

- ☐ Publique o plugin com a tag `dsh-plugin`
- ☐ Crie um plugin que valida código antes de enviar
- ☐ Implemente o validador multi-linguagem

6. Conclusão

Neste capítulo, você criou seu primeiro plugin para o DeepSeek Harness. Viu a estrutura padrão, o ciclo de vida, as APIs disponíveis e como publicar. A arquitetura de plugins é o que torna o Harness extensível — você pode adicionar qualquer funcionalidade sem modificar o core [1].

Você também aprendeu padrões de design (processamento, ferramenta, observer, hook) e anti-patterns a evitar. Os testes são parte fundamental do desenvolvimento de plugins — um plugin com bugs pode comprometer todo o sistema [1].

No próximo capítulo, você mergulha nas bibliotecas de baixo nível — FlashMLA, DeepEP e DeepGEMM — que tornam a inferência DeepSeek ultrarrápida. É aqui que a eficiência arquitetural se encontra com a performance bruta de hardware.

7. Referências Bibliográficas

[1] DEEPSEEK-AI. *DeepSeek Harness: Everything is a Plugin*. Disponível em: <https://github.com/deepseek-ai/deepseek-harness>. Acesso em: 21 ago. 2026.

[2] CORDIVERSE. *Cordis: A Meta-Framework of Spatiotemporal Composability*. Disponível em: <https://github.com/cordiverse/cordis>. Acesso em: 21 ago. 2026.

[3] DEEPSEEK-AI. *DeepSeek API Documentation*. Disponível em: <https://api-docs.deepseek.com/>. Acesso em: 21 ago. 2026.

[4] DEEPSEEK-AI. *DeepSeek-V3 Technical Report*. Disponível em: <https://arxiv.org/abs/2412.19437>. Acesso em: 21 ago. 2026.

[5] DEEPSEEK-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Disponível em: <https://arxiv.org/abs/2501.12948>. Acesso em: 21 ago. 2026.

[6] DEEPSEEK-AI. *Awesome DeepSeek Agent*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-agent>. Acesso em: 21 ago. 2026.

[7] DEEPSEEK-AI. *Awesome DeepSeek Integration*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-integration>. Acesso em: 21 ago. 2026.

- [8] DEEPSEEK-AI. *FlashMLA: Efficient Multi-head Latent Attention Kernels*. Disponível em: <https://github.com/deepseek-ai/FlashMLA>. Acesso em: 21 ago. 2026.
- [9] DEEPSEEK-AI. *DeepEP: An Efficient Expert-Parallel Communication Library*. Disponível em: <https://github.com/deepseek-ai/DeepEP>. Acesso em: 21 ago. 2026.
- [10] DEEPSEEK-AI. *DeepGEMM: Clean and Efficient BLAS Kernel Library on GPU*. Disponível em: <https://github.com/deepseek-ai/DeepGEMM>. Acesso em: 21 ago. 2026.
- [11] DEEPSEEK-AI. *3FS: A High-Performance Distributed File System for AI*. Disponível em: <https://github.com/deepseek-ai/3FS>. Acesso em: 21 ago. 2026.
- [12] DEEPSEEK-AI. *DeepSpec: Full-Stack Codebase for Speculative Decoding*. Disponível em: <https://github.com/deepseek-ai/DeepSpec>. Acesso em: 21 ago. 2026.
- [13] DEEPSEEK-AI. *TileKernels: Kernel Library Written in Tilelang*. Disponível em: <https://github.com/deepseek-ai/TileKernels>. Acesso em: 21 ago. 2026.
- [14] DEEPSEEK-AI. *DeepSeek-OCR-2: Visual Causal Flow*. Disponível em: <https://github.com/deepseek-ai/DeepSeek-OCR-2>. Acesso em: 21 ago. 2026.
- [15] VASWANI, A. et al. *Attention Is All You Need*. In: Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [16] TYPESCRIPT. *TypeScript Handbook*. Disponível em: <https://www.typescriptlang.org/docs/handbook/>. Acesso em: 21 ago. 2026.
- [17] ANTHROPIC. *Model Context Protocol Specification*. Disponível em: <https://spec.modelcontextprotocol.io/>. Acesso em: 21 ago. 2026.
- [18] DEEPSEEK-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. Disponível em: <https://arxiv.org/abs/2405.04434>. Acesso em: 21 ago. 2026.
- [19] FEDUS, W. et al. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. In: Journal of Machine Learning Research, 2022.
- [20] DEEPSEEK-AI. *DeepSeek Platform — Pricing*. Disponível em: <https://platform.deepseek.com/>. Acesso em: 21 ago. 2026.

8. Apêndice — Testes Avançados de Plugins

Testes de Integração

```
// tests/integration.test.ts
import { describe, it, expect, beforeAll } from "vitest";
import { createHarness } from "@deepseek-ai/dsh";
import { validadorCodigoPlugin } from "../src/index";

describe("Integração: validador + harness", () => {
  let harness: any;

  beforeAll(async () => {
    harness = await createHarness({
      plugins: [validadorCodigoPlugin],
      config: {
        "python-validator": {
          linguagens: ["python"],
          maxLinhas: 100
        }
      }
    });
  });

  it("deve detectar código inválido na mensagem", async () => {
    const resposta = await harness.processar({
      content: "Analise este código:\n```python\ndef foo(\n    pass\n```",
      userId: "test-user"
    });

    // Plugin deve ter detectado erro
    expect(resposta.validacao).toBeDefined();
    expect(resposta.validacao[0].valido).toBe(false);
  });

  it("deve ignorar mensagens sem código", async () => {
    const resposta = await harness.processar({
      content: "Qual a capital da França?",
      userId: "test-user"
    });

    expect(resposta.validacao).toBeUndefined();
  });
});
```

```

it("deve validar múltiplos blocos", async () => {
  const resposta = await harness.processar({
    content: "Código 1:\n``python\ndef a(): pass\n``\nCódigo 2:\n\n``python\ndef b( pass\n``",
    userId: "test-user"
  });

  expect(resposta.validacao).toHaveLength(2);
});

```

Testes de Performance

```

// tests/performance.test.ts
import { describe, it, expect } from "vitest";
import { validadorCodigoPlugin } from "../src/index";

describe("Performance: validador", () => {
  it("deve validar código em menos de 100ms", async () => {
    const codigo = "def hello():\n    print('hello')\n".repeat(10);

    const inicio = Date.now();
    const resultado = await validadorCodigoPlugin.validar(codigo);
    const duracao = Date.now() - inicio;

    expect(duracao).toBeLessThan(100);
    expect(resultado.valido).toBe(true);
  });

  it("deve processar 100 mensagens em menos de 1s", async () => {
    const mensagens = Array(100).fill(null).map((_, i) => ({
      content: `Código ${i}:\n``python\ndef func_${i}(): pass\n``,
      userId: `user-${i}`
    }));

    const inicio = Date.now();
    for (const msg of mensagens) {
      await validadorCodigoPlugin.onMessage(msg as any);
    }
    const duracao = Date.now() - inicio;

    expect(duracao).toBeLessThan(1000);
  });
});

```

Mock de Context

```
// helpers/mock-context.ts
export function criarMockContext(config: any = {}) {
  const storage = new Map();

  return {
    config: {
      get: (chave: string) => config[chave],
      set: (chave: string, valor: any) => { config[chave] = valor; },
    },
    storage: {
      get: async (chave: string) => storage.get(chave),
      set: async (chave: string, valor: any) => storage.set(chave, valor),
      delete: async (chave: string) => storage.delete(chave)
    },
    logger: {
      info: (...args: any[]) => console.log("[INFO]", ...args),
      warn: (...args: any[]) => console.warn("[WARN]", ...args),
      error: (...args: any[]) => console.error("[ERROR]", ...args),
      debug: (...args: any[]) => console.log("[DEBUG]", ...args)
    }
  };
}
```

9. Apêndice — Publicação e Distribuição de Plugins

Preparando para Publicação

```
# 1. Adicione .npmignore
cat > .npmignore << 'EOF'
src/
tests/
tsconfig.json
.git/
.github/
EOF

# 2. Atualize package.json
cat > package.json << 'EOF'
{
```

```

    "name": "@deepseek-ai/dsh-plugin-meu-plugin",
    "version": "1.0.0",
    "description": "Plugin que faz X para o DeepSeek Harness",
    "keywords": ["dsh-plugin", "deepseek", "harness"],
    "main": "dist/index.js",
    "types": "dist/index.d.ts",
    "files": ["dist/"],
    "scripts": {
      "build": "tsc",
      "prepublishOnly": "npm run build"
    }
  }
}
EOF

# 3. Build
npm run build

# 4. Teste local
npm pack
npm install -g deepseek-ai-dsh-plugin-meu-plugin-1.0.0.tgz

# 5. Publique
npm publish --access public

# 6. Adicione a tag dsh-plugin
gh repo edit <seu-usuario>/dsh-plugin-meu-plugin --add-topic dsh-plugin

```

Documentação do Plugin

DSH Plugin: Meu Plugin

Plugin para o DeepSeek Harness que faz X.

Instalação

```

` `` bash
npx @deepseek-ai/dsh web --plugins meu-plugin

```

Configuração

```
{
  "plugins": {
    "meu-plugin": {
      "enabled": true,
      "opcao1": "valor"
    }
  }
}
```

Funcionalidades

- Feature 1: Descrição
- Feature 2: Descrição

Desenvolvimento

```
git clone https://github.com/usuario/dsh-plugin-meu-plugin.git
cd dsh-plugin-meu-plugin
npm install
npm run dev
```

Licença

MIT

```
### Métricas de Adoção

```python
Script para verificar métricas do plugin no GitHub
import requests
```

```
def verificar_metricas(repo):
 """Verifica métricas de adoção do plugin."""
 url = f"https://api.github.com/repos/{repo}"
 r = requests.get(url)
 dados = r.json()

 print(f"Repositório: {repo}")
 print(f"Stars: {dados.get('stargazers_count', 0)}")
 print(f"Forks: {dados.get('forks_count', 0)}")
 print(f"Watchers: {dados.get('subscribers_count', 0)}")
 print(f"Open Issues: {dados.get('open_issues_count', 0)}")
 print(f"Último push: {dados.get('pushed_at', 'N/A')}")

verificar_metricas("deepseek-ai/dsh-plugin-meu-plugin")
```



## Capítulo 7: Otimização de Inferência: FlashMLA, DeepEP e DeepGEMM

---

### 1. Introdução

---

Nos capítulos anteriores, você construiu soluções com DeepSeek — configurou a API, integrou ferramentas, criou plugins. Mas todos esses recursos dependem de uma coisa: que o modelo responda rápido o suficiente para ser útil. Se a inferência é lenta, não importa quão bom seja o prompt ou quão elegante seja o plugin — o usuário vai desistir de esperar [1].

Neste capítulo, você vai entender as três bibliotecas de baixo nível que tornam a inferência DeepSeek ultrarrápida: FlashMLA (atenção otimizada), DeepEP (comunicação expert-parallel) e DeepGEMM (kernels BLAS). São essas bibliotecas que permitem ao DeepSeek-V3 processar tokens com latência mínima, mesmo com 671 bilhões de parâmetros [1]. A compreensão dessas bibliotecas é o que separa um desenvolvedor que usa IA de um engenheiro que otimiza IA.

### 2. Explica

---

#### FlashMLA: Atenção Otimizada

A Multi-head Latent Attention (MLA) é a 技术创新 que torna a inferência do DeepSeek eficiente. O FlashMLA é a implementação em kernels CUDA dessa técnica. Ele oferece dois tipos de kernels [2]:

**Dense Attention** — Atenção padrão para todas as posições. Usada no prefill (processamento do prompt completo) e no decoding quando não há restrição de memória. Alcança até 660 TFLOPS em H800 SXM5 — isso significa que o kernel processa 660 trilhões de operações de ponto flutuante por segundo [2].

**Sparse Attention** — Atenção seletiva que processa apenas os tokens mais relevantes. Em vez de calcular a atenção para todas as combinações de tokens, o sparse attention seleciona apenas os top-k tokens mais relevantes para cada query. Reduz drasticamente o custo de memória e compute. Alcança até 640 TFLOPS no prefill e 410 TFLOPS no decoding [2].

O kernel de decoding sparse utiliza FP8 KV cache — cada token ocupa apenas 656 bytes (512 bytes quantizados em FP8 + 16 bytes de escala + 128 bytes RoPE em BF16). Isso permite armazenar muito mais contexto na mesma quantidade de memória [2].

**Suporte a hardware:** - SM90 (Hopper): Dense decoding, Sparse decoding, Sparse prefill - SM100 (Blackwell): Todos os kernels, incluindo MHA dense prefill

O FlashMLA também suporta GPUs de outros fabricantes através de ports: MetaX, Moore Threads, Hygon DCU, AMD Instinct [2].

## DeepEP: Comunicação Expert-Parallel

Quando você treina ou inferencia um modelo MoE em múltiplas GPUs, os experts precisam se comunicar. Cada GPU contém um subconjunto dos experts, e os tokens precisam ser roteados entre GPUs. Essa comunicação é tradicionalmente um gargalo — os dados precisam trafegar pela rede (InfiniBand, RDMA) ou pelo barramento PCIe [3].

O DeepEP é uma biblioteca de comunicação que otimiza essa troca de dados. Na versão V2, ele introduziu várias inovações:

**ElasticBuffer** — Interface unificada que combina dispatch de alta throughput e baixa latência em uma única abstração. Diferente da V1, que tinha APIs separadas para alta throughput e baixa latência, a V2 unifica tudo em uma interface de buffer elástico [3].

**Cálculo analítico de SM e QP** — Em vez de auto-tuning (que gasta tempo e recursos), o DeepEP V2 calcula analiticamente a contagem ideal de SM (Streaming Multiprocessors) e QP (Queue Pairs). Isso elimina a necessidade de calibração manual [3].

**Zero SM** — Operações que não ocupam nenhum SM de computação. Isso permite que a comunicação aconteça em paralelo com a computação, sem competir por recursos [3].

**NCCL Gin backend** — Backend mais leve que o NVSHMEM, reduzindo overhead de inicialização e memória [3].

**Performance:** Na configuração V3 (8K tokens, 7168 hidden, top-8 experts, FP8 dispatch), o DeepEP V2 alcança: - EP 8×2: 90 GB/s dispatch, 81 GB/s combine (12 SMs) - EP 8×4: 61 GB/s dispatch, 61 GB/s combine (6 SMs) - EP 8 NVLink: 726 GB/s dispatch, 740 GB/s combine (64 SMs)

## DeepGEMM: Kernels BLAS

DeepGEMM é uma biblioteca de kernels BLAS (Basic Linear Algebra Subprograms) otimizados para GPU. Ele fornece operações matriciais de alta performance que são o fundamento de tudo: multiplicação de matrizes, convoluções, transformadas [4].

O DeepGEMM se destaca por usar compilação JIT (Just-In-Time) — os kernels são compilados em tempo de execução, permitindo otimizações específicas para a configuração de hardware atual. Isso significa que o mesmo código se adapta automaticamente a diferentes GPUs [4].

## 3FS: Sistema de Arquivos Distribuído

O 3FS é um sistema de arquivos de alta performance projetado para os desafios de treinamento e inferência de IA. Ele resolve o gargalo de I/O que ocorre quando múltiplas GPUs precisam acessar dados simultaneamente. Em treinamento de modelos grandes, o I/O de dados pode ser mais lento que a computação — o 3FS resolve isso com paralelismo massivo e caching inteligente [5].

## DeepSpec: Speculative Decoding

O DeepSpec é um framework para treinar e avaliar algoritmos de speculative decoding. Essa técnica usa um modelo menor para “adivinhar” tokens e o modelo maior para validar. Se as adivinhações estiverem corretas, a inferência é muito mais rápida. Se estiverem erradas, o modelo maior gera os tokens corretos. O resultado é uma redução significativa de latência sem perda de qualidade [6].

## 3. Ilustra

---

### A Estação de Alta Performance

Se o laboratório DeepSeek é um conjunto de estações, FlashMLA, DeepEP e DeepGEMM são as estações de alta performance — as que processam a maior quantidade de trabalho no menor tempo possível. Elas trabalham em camadas diferentes, cada uma otimizando um aspecto da pipeline [1].

%% legenda: Camadas de otimização de inferência DeepSeek  
flowchart TD

```

A[Requisição do Usuário] --> B[FlashMLA
Atenção Otimizada
660 TFLOPS]
B --> C[DeepGEMM
Kernels BLAS JIT]
C --> D[DeepEP
Comunicação Multi-GPU
90 GB/s]
D --> E[3FS
Arquivo Distribuído]
E --> F[GPUs Físicas
H100/H200]

B --> G[FP8 KV Cache
656 bytes/token]
C --> H[Compilação JIT
Adapta ao hardware]
D --> I[ElasticBuffer
Zero SM]

```

Cada camada otimiza um aspecto diferente: FlashMLA otimiza a atenção (o cálculo mais custoso de um Transformer), DeepGEMM otimiza as operações matriciais (o fundamento de tudo), DeepEP otimiza a comunicação entre GPUs (o gargalo em treinamento distribuído), e 3FS otimiza o acesso a dados (o gargalo em I/O). Juntos, eles formam uma pipeline de inferência que aproveita cada ciclo de clock da GPU [1].

## A Pirâmide de Performance

```

%% legenda: Pirâmide de otimização – de cima para baixo
flowchart TD
 A[Aplicação
Plugin + Prompt] --> B[Framework
SGLang / vLLM]
 B --> C[Modelo
V3 / R1 / V4]
 C --> D[Kernel
FlashMLA + DeepGEMM]
 D --> E[Comunicação
DeepEP]
 E --> F[Hardware
GPU + Rede]

```

Cada camada da pirâmide depende da camada abaixo. Uma otimização no kernel (FlashMLA) melhora todos os modelos que o usam. Uma otimização no hardware (mais GPUs) melhora todas as aplicações. A chave é otimizar da base para cima [1].

## 4. Técnica

---

### Instalando FlashMLA

```

Requisitos: SM90+ (Hopper/Blackwell), CUDA 12.8+, PyTorch 2.0+
git clone https://github.com/deepseek-ai/FlashMLA.git flash-mla
cd flash-mla

```

```
git submodule update --init --recursive
pip install -v .
```

## Usando FlashMLA para Decoding

```
from flash_mla import get_mla_metadata, flash_mla_with_kvcache

Configuração do modelo
cache_seqLens = [1024, 2048, 512] # Tamanho do cache por sequência
s_q = 1 # Tokens por query (1 no decoding)
h_q = 128 # Cabeças de query
h_kv = 8 # Cabeças de key-value (MLA comprime)
dv = 512 # Dimensão do valor
is_fp8 = True # Usar FP8 KV cache

Obtém metadata do scheduler (chamada única antes do loop)
tile_scheduler_metadata, num_splits = get_mla_metadata(
 cache_seqLens, # Tamanhos do cache
 s_q * h_q // h_kv, # Cabeças por grupo
 h_kv, # Cabeças KV
 h_q, # Cabeças Q
 is_fp8, # FP8 KV cache
 8 # Top-k experts
)

Loop de decoding (executado para cada token)
for i in range(num_layers):
 # q_i: query tensor [batch, seq_q, h_q, d_qk]
 # kvcache_i: KV cache tensor
 # block_table: mapeamento de blocos
 o_i, lse_i = flash_mla_with_kvcache(
 q_i, kvcache_i, block_table, cache_seqLens, dv,
 tile_scheduler_metadata, num_splits,
 is_causal=True,
 is_fp8_kvcache=is_fp8
)
 # o_i: resultado da atenção
 # lse_i: log-sum-exp para normalização
```

## Instalando DeepEP

```
Instale NCCL
pip install "nvidia-nccl-cu13>=2.30.4" --no-deps
```

```
Instale DeepEP
python setup.py install
```

## Usando DeepEP para MoE

```
import torch
import torch.distributed as dist
from deep_ep import ElasticBuffer, EPHandle, EventOverlap

Inicializa o buffer elástico
buffer = ElasticBuffer(
 group, # Process group do torch.distributed
 num_max_tokens_per_rank=8192, # Tokens máximos por rank
 hidden=7168, # Dimensão oculta
 num_topk=8, # Experts selecionados por token
 use_fp8_dispatch=True # Usar FP8 para dispatch
)

Calcula SM ideal analiticamente (sem auto-tuning)
num_comm_sms = buffer.get_theoretical_num_sms(num_experts=256, num_topk=8)

Dispatch: roteia tokens para experts across ranks
recv_x, recv_topk_idx, recv_topk_weights, handle, event = buffer.dispatch(
 x, # Input tensor
 topk_idx=topk_idx, # Índices dos experts selecionados
 topk_weights=topk_weights, # Pesos dos experts
 num_experts=256, # Total de experts
 num_max_tokens_per_rank=8192,
 expert_alignment=1,
 num_sms=num_comm_sms,
 async_with_compute_stream=True # Assíncrono com compute
)

Espera comunicação terminar antes de usar resultados
event.current_stream_wait()

Combine: reduz saídas dos experts de volta ao rank original
combined_x, _, event = buffer.combine(
 x, # Saída dos experts
 handle=handle, # Metadata do dispatch
 num_sms=num_comm_sms,
 async_with_compute_stream=True
)
```

## Benchmark de Performance

```
import time
import torch

def benchmark_flash_mla():
 """Mede throughput do kernel FlashMLA."""
 from flash_mla import get_mla_metadata, flash_mla_with_kvcache

 batch_sizes = [1, 8, 32, 128]
 seq_lens = [256, 1024, 4096, 16384]

 print("Benchmark FlashMLA Dense Decoding")
 print(f"{'Batch':>6} {'SeqLen':>8} {'Throughput':>12} {'Latência':>10}")
 print("-" * 40)

 for batch in batch_sizes:
 for seq in seq_lens:
 # Configuração
 cache_seqlens = [seq] * batch
 h_q, h_kv, dv = 128, 8, 512

 # Metadata
 metadata, splits = get_mla_metadata(
 cache_seqlens, 1 * h_q // h_kv, h_kv, h_q, False, 8
)

 # Benchmark (100 iterações)
 start = time.perf_counter()
 for _ in range(100):
 # ... chamada do kernel ...
 pass
 elapsed = (time.perf_counter() - start) / 100

 throughput = batch / elapsed
 print(f"{'batch':>6} {'seq':>8} {'throughput':>10.1f} b/s {'elapsed*1000:>8.2f} ms")

def benchmark_fp8_vs_bf16():
 """Compara uso de memória FP8 vs BF16."""
 tokens = 10000

 fp8_bytes = tokens * 656 # 656 bytes por token em FP8
 bf16_bytes = tokens * 2624 # ~2.6KB por token em BF16

 print(f"Tokens: {tokens:,}")
```

```
print(f"FP8 KV Cache: {fp8_bytes/1024/1024:.2f} MB")
print(f"BF16 KV Cache: {bf16_bytes/1024/1024:.2f} MB")
print(f"Redução: {(1 - fp8_bytes/bf16_bytes)*100:.1f}%")
```

## 5. Aplica

### O Gargalo que Ninguém Vê

Sua empresa treinou um modelo MoE com 8 GPUs. O treinamento funciona, mas é 3x mais lento do que o esperado. O problema não é o código do modelo — é a comunicação entre GPUs. Quando cada GPU precisa acessar dados de outras GPUs para processar tokens, o tráfego de rede se torna o gargalo [3].

O DeepEP resolve isso com o ElasticBuffer, que permite sobrepor comunicação e computação. Enquanto uma GPU envia dados, ela já pode estar computando com os dados que já recebeu. O resultado: o gargalo de comunicação desaparece [3].

Outro problema comum é o KV cache crescendo além da memória disponível. Com FlashMLA e FP8 KV cache, cada token ocupa apenas 656 bytes — 4x menos que BF16. Isso permite processar sequências 4x mais longas na mesma GPU [2].

### Métricas de Performance

Métrica	Sem Otimização	Com FlashMLA	Melhoria
Throughput decoding	50 tokens/s	300 tokens/s	6x
KV cache por token	2.6 KB	656 bytes	4x
Latência p50	200ms	35ms	5.7x
Custo por 1M tokens	\$0.10	\$0.02	5x

### Exercício

- ☐ Instale FlashMLA e rode o benchmark de decoding
- ☐ Compare throughput com e sem sparse attention
- ☐ Instale DeepEP e teste a inicialização do ElasticBuffer
- ☐ Meça o uso de memória com e sem FP8 KV cache



- ☐ Crie um script de benchmark que compara latência de diferentes configs
- ☐ Compare performance entre H100 e H200

## 6. Conclusão

---

Neste capítulo, você entendeu as bibliotecas de baixo nível que tornam a inferência DeepSeek ultrarrápida. FlashMLA otimiza a atenção com kernels esparsos e FP8, alcançando 660 TFLOPS em H800. DeepEP otimiza a comunicação entre GPUs com ElasticBuffer e cálculo analítico de SM. DeepGEMM fornece kernels BLAS de alta performance com compilação JIT [1][2][3].

Juntas, essas bibliotecas são o que permite ao DeepSeek-V3 competir com modelos muito maiores. A combinação de arquitetura eficiente (MoE, MLA) com otimizações de baixo nível (FlashMLA, DeepEP) cria um sistema que é maior que a soma de suas partes [1].

No próximo capítulo — e último — você vai juntar tudo: deploy em produção, escalabilidade, e otimização de custos. É aqui que a teoria encontra a realidade do mercado.

## 7. Referências Bibliográficas

---

[1] DEEPSEEK-AI. *DeepSeek-V3 Technical Report*. Disponível em: <https://arxiv.org/abs/2412.19437>. Acesso em: 21 ago. 2026.

[2] LI, Jiashi; LIU, Shengyu. *FlashMLA: Efficient Multi-head Latent Attention Kernels*. Disponível em: <https://github.com/deepseek-ai/FlashMLA>. Acesso em: 21 ago. 2026.

[3] DEEPSEEK-AI. *DeepEP: An Efficient Expert-Parallel Communication Library*. Disponível em: <https://github.com/deepseek-ai/DeepEP>. Acesso em: 21 ago. 2026.

[4] DEEPSEEK-AI. *DeepGEMM: Clean and Efficient BLAS Kernel Library on GPU*. Disponível em: <https://github.com/deepseek-ai/DeepGEMM>. Acesso em: 21 ago. 2026.

[5] DEEPSEEK-AI. *3FS: A High-Performance Distributed File System for AI*. Disponível em: <https://github.com/deepseek-ai/3FS>. Acesso em: 21 ago. 2026.

[6] DEEPSEEK-AI. *DeepSpec: Full-Stack Codebase for Speculative Decoding*. Disponível em: <https://github.com/deepseek-ai/DeepSpec>. Acesso em: 21 ago. 2026.

[7] DEEPSEEK-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Disponível em: <https://arxiv.org/abs/2501.12948>. Acesso em: 21 ago. 2026.

[8] DEEPSEEK-AI. *DeepSeek Harness: Everything is a Plugin*. Disponível em: <https://github.com/deepseek-ai/deepseek-harness>. Acesso em: 21 ago. 2026.

- [9] DEEPSEEK-AI. *Awesome DeepSeek Agent*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-agent>. Acesso em: 21 ago. 2026.
- [10] DEEPSEEK-AI. *Awesome DeepSeek Integration*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-integration>. Acesso em: 21 ago. 2026.
- [11] CORDIVERSE. *Cordis: A Meta-Framework of Spatiotemporal Composability*. Disponível em: <https://github.com/cordiverse/cordis>. Acesso em: 21 ago. 2026.
- [12] DEEPSEEK-AI. *TileKernels: Kernel Library Written in Tilelang*. Disponível em: <https://github.com/deepseek-ai/TileKernels>. Acesso em: 21 ago. 2026.
- [13] DEEPSEEK-AI. *DeepSeek-OCR-2: Visual Causal Flow*. Disponível em: <https://github.com/deepseek-ai/DeepSeek-OCR-2>. Acesso em: 21 ago. 2026.
- [14] VASWANI, A. et al. *Attention Is All You Need*. In: Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [15] DEEPSEEK-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. Disponível em: <https://arxiv.org/abs/2405.04434>. Acesso em: 21 ago. 2026.
- [16] FEDUS, W. et al. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. In: Journal of Machine Learning Research, 2022.
- [17] DEEPSEEK-AI. *DeepSeek Platform — Pricing*. Disponível em: <https://platform.deepseek.com/>. Acesso em: 21 ago. 2026.
- [18] DEEPSEEK-AI. *DeepSeek API Documentation*. Disponível em: <https://api-docs.deepseek.com/>. Acesso em: 21 ago. 2026.
- [19] NVIDIA. *CUDA Programming Guide*. Disponível em: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>. Acesso em: 21 ago. 2026.
- [20] PYTORCH. *PyTorch Documentation — Distributed Training*. Disponível em: <https://pytorch.org/docs/stable/distributed/>. Acesso em: 21 ago. 2026.

## 8. Apêndice — Comparativo de Hardware para Inferência

### GPUs e Suporte

GPU	VRAM	SM	FlashMLA	DeepEP	Custo/hora
A100 80GB	80 GB	SM80	✗	✓	~\$1.50
H100 80GB	80 GB	SM90	✓	✓	~\$3.00
H200 141GB	141 GB	SM90	✓	✓	~\$4.50
B200 192GB	192 GB	SM100	✓	✓	~\$6.00

### Configurações Recomendadas

**DeepSeek-V3 (671B parâmetros):** - Mínimo: 8× H100 80GB (tensor parallelism) - Recomendado: 8× H200 141GB (maior contexto) - Ótimo: 16× H100 (multi-node, 128K contexto completo)

**DeepSeek-R1-Distill-Qwen-32B:** - Mínimo: 1× A100 80GB - Recomendado: 2× A100 80GB (tensor parallelism) - Ótimo: 1× H100 80GB (maior velocidade)

**DeepSeek-V4-Flash:** - Mínimo: 1× A100 80GB - Recomendado: 1× H100 80GB - Ótimo: 2× H100 (maior throughput)

### Custos de Infraestrutura

```
def estimar_custos_infr(tipo_gpu, horas_mes, n_gpus):
 """Estima custos de infraestrutura para inferência."""
 precos = {
 "A100": 1.50,
 "H100": 3.00,
 "H200": 4.50,
 "B200": 6.00
 }

 custo_gpu = precos.get(tipo_gpu, 3.00)
 custo_total = custo_gpu * horas_mes * n_gpus

 print(f"Configuração: {n_gpus}× {tipo_gpu}")
```

```

print(f"Horas/mês: {horas_mes}")
print(f"Custo/mês: ${custo_total:,.2f}")
print(f"Custo/dia: ${custo_total/30:,.2f}")

return custo_total

Exemplos
estimar_custos_infr("H100", 720, 8) # Full-time 8 GPUs
estimar_custos_infr("A100", 720, 2) # Full-time 2 GPUs
estimar_custos_infr("H100", 240, 1) # 8h/dia 1 GPU

```

## 9. Apêndice — Otimizações Avançadas de Inferência

---

### Quantização

A quantização reduz a precisão dos pesos do modelo, diminuindo o uso de memória e aumentando a velocidade. O DeepSeek-V3 suporta FP8 nativamente [1]:

```

Exemplo de quantização com vLLM
FP8: reduz memória 50%, sem perda significativa de qualidade
vllm serve deepseek-ai/DeepSeek-V3-Base \
 --quantization fp8 \
 --tensor-parallel-size 8

INT8: reduz memória 75%, perda mínima
vllm serve deepseek-ai/DeepSeek-V3-Base \
 --quantization awq \
 --tensor-parallel-size 4

INT4: reduz memória 87%, perda perceptível
vllm serve deepseek-ai/DeepSeek-V3-Base \
 --quantization gptq \
 --tensor-parallel-size 2

```

### KV Cache Optimization

O KV cache é o maior consumidor de memória em inferência. FlashMLA com FP8 reduz o custo por token de ~2.6KB para 656 bytes [4]:

```

Comparação de uso de memória
def comparar_kv_cache(n_tokens):
 bf16 = n_tokens * 2624 # ~2.6KB por token
 fp8 = n_tokens * 656 # 656 bytes por token

 print(f"Tokens: {n_tokens:,}")
 print(f"BF16: {bf16/1024/1024:.2f} MB")
 print(f"FP8: {fp8/1024/1024:.2f} MB")
 print(f"Economia: {(1-fp8/bf16)*100:.1f}%")

 # Com FP8, cabe 4x mais contexto
 contexto_bf16 = (80 * 1024 * 1024 * 1024) / 2624 # 80GB GPU
 contexto_fp8 = (80 * 1024 * 1024 * 1024) / 656

 print(f"\nEm 80GB GPU:")
 print(f"BF16: {contexto_bf16/1000:.0f}K tokens")
 print(f"FP8: {contexto_fp8/1000:.0f}K tokens")

comparar_kv_cache(10000)

```

## Speculative Decoding

O speculative decoding usa um modelo menor para “adivinhar” tokens e o modelo maior para validar. Se as adivinhações estiverem corretas, a inferência é muito mais rápida [5]:

```

Conceito de speculative decoding
def speculative_decoding_concept():
 """Demonstra o conceito de speculative decoding."""

 # Modelo grande (DeepSeek-V3): 671B parâmetros, lento mas preciso
 modelo_grande = "deepseek-v3"

 # Modelo pequeno (Distill-1.5B): 1.5B parâmetros, rápido mas menos
 # preciso
 modelo_pequeno = "deepseek-r1-distill-qwen-1.5b"

 # Fluxo:
 # 1. Modelo pequeno gera 5 tokens candidatos
 # 2. Modelo grande valida todos de uma vez (batch)
 # 3. Tokens aceitos são usados, rejeitados são regenerados

 # Resultado: 2-3x mais rápido com qualidade igual

```

## Batching Eficiente

```
import asyncio
from typing import List

class BatchProcessor:
 """Processa múltiplas requests em batch para eficiência."""

 def __init__(self, cliente, batch_size=32, timeout_ms=100):
 self.cliente = cliente
 self.batch_size = batch_size
 self.timeout = timeout_ms / 1000
 self.fila = []

 async def adicionar(self, mensagem):
 """Adiciona mensagem à fila e aguarda processamento."""
 future = asyncio.Future()
 self.fila.append((mensagem, future))

 if len(self.fila) >= self.batch_size:
 await self.processar_batch()

 return await future

 async def processar_batch(self):
 """Processa um batch de mensagens."""
 if not self.fila:
 return

 batch = self.fila[:self.batch_size]
 self.fila = self.fila[self.batch_size:]

 # Processa todas as mensagens de uma vez
 mensagens = [m[0] for m in batch]
 futures = [m[1] for m in batch]

 # Chama a API com batch
 resposta = await self.cliente.chat.completions.create(
 model="deepseek-v4-flash",
 messages=mensagens,
 batch=True
)

 # Resolve os futures
 for i, future in enumerate(futures):
 future.set_result(resposta.choices[i])
```

## Capítulo 8: Deploy, Escalabilidade e Custo

---

### 1. Introdução

---

Ao longo deste livro, você percorreu todo o laboratório DeepSeek: dos modelos à API, das integrações ao Harness, dos prompts ao plugins, das otimizações de inferência às bibliotecas de baixo nível. Agora é hora de levar tudo isso para a produção. Deploy, escalabilidade e custo são as três questões que separam um protótipo de um sistema que gera valor real [1].

A boa notícia é que o ecossistema DeepSeek oferece múltiplos caminhos para produção — desde a API gerenciada (sem infraestrutura) até o deploy local com frameworks como SGLang e vLLM. Cada caminho tem seu trade-off entre custo, controle e complexidade. Neste capítulo, você vai mapear esses caminhos, entender as estratégias de escalabilidade e aprender a otimizar custos para cada cenário [1].

### 2. Explica

---

#### Três Caminhos para Produção

**API Gerenciada** — O caminho mais simples. Você usa a API da DeepSeek diretamente, sem gerenciar infraestrutura. Custo por token, sem compromisso de hardware. Ideal para MVPs, startups e equipes pequenas. A API suporta streaming, thinking mode e todos os recursos dos modelos [1].

Vantagens: zero infraestrutura, setup em minutos, pagamento por uso, sem compromisso de longo prazo. Desvantagens: dependência de terceiros, custo cresce linearmente com volume, latência de rede.

**Deploy Local com Frameworks** — Você roda o modelo em seus próprios GPUs. Maior controle, custo fixo por hardware, latência previsível. Ideal para empresas com volume alto de requests e requisitos de privacidade [2].

Vantagens: controle total, latência mínima, sem dependência de terceiros, dados não saem da empresa. Desvantagens: custo inicial alto, manutenção de hardware, necessidade de expertise em infraestrutura.

**Deploy Distribuído** — Múltiplas máquinas, tensor parallelism, pipeline parallelism. O caminho para escalar além de uma GPU. Necessário para modelos grandes como o V3 (671B parâmetros) que não cabem em uma única GPU [2].

Vantagens: escala massiva, redundant, performance linear. Desvantagens: complexidade operacional alta, custo de rede, necessidade de equipamento especializado.

## Frameworks de Inferência

A DeepSeek recomenda quatro frameworks principais para inferência local [2]:

**SGLang** — O framework recomendado. Suporta MLA optimizations, DP Attention, FP8 (W8A8), FP8 KV Cache e Torch Compile. Funciona em NVIDIA e AMD GPUs. Suporta multi-node tensor parallelism. É o framework mais maduro e otimizado para DeepSeek [2].

**vLLM** — Framework amplamente utilizado pela comunidade. Suporta DeepSeek-V3 em FP8 e BF16, pipeline parallelism e tensor parallelism. Uma opção madura e testada, com grande comunidade de usuários [2].

**LMDeploy** — Framework flexível para inferência e serving. Suporta FP8 e BF16, com integração PyTorch. Bom para ambientes que já usam PyTorch [2].

**TensorRT-LLM** — Framework da NVIDIA otimizado para GPUs NVIDIA. Suporta BF16, INT4/INT8 e FP8 (em desenvolvimento). Oferece a melhor performance em hardware NVIDIA, mas é menos flexível [2].

## Estratégias de Escalabilidade

**Tensor Parallelism** — Divide cada tensor entre múltiplas GPUs. Cada GPU processa uma parte de cada camada. Reduz latência, aumenta throughput. Ideal quando o modelo cabe em memória mas a latência é alta [2].

**Pipeline Parallelism** — Divide o modelo em estágios, cada um em uma GPU diferente. GPU 1 processa camadas 1-10, GPU 2 processa camadas 11-20, etc. Reduz memória por GPU. Ideal quando o modelo não cabe em uma GPU [2].

**Data Parallelism** — Cada GPU tem uma cópia completa do modelo. Dados diferentes são processados em paralelo. Aumenta throughput linearmente. Ideal quando o gargalo é throughput, não latência [2].

**Multi-Node** — Distribui o modelo em múltiplas máquinas. Necessário quando o modelo não cabe em uma máquina. Requer rede de alta velocidade (InfiniBand, RDMA). Para DeepSeek-V3 (671B), são necessárias pelo menos 8 GPUs H100 [2].



## Otimização de Custo

**Modelos Distilados** — Use DeepSeek-R1-Distill-Qwen-32B em vez de R1 completo quando a qualidade aceitável for menor. Custo ~10x menor, performance comparable para muitas tarefas [3].

**FP8** — Formato de precisão reduzida que reduz uso de memória e aumenta throughput. DeepSeek-V3 foi treinado nativamente em FP8, então não há perda de qualidade [2].

**KV Cache** — Com FlashMLA e FP8, cada token ocupa 656 bytes em vez de ~2.6KB. Isso permite 4x mais contexto na mesma GPU [4].

**Speculative Decoding** — Usa um modelo menor para “adivinhar” tokens e o modelo maior para validar. Reduz latência sem perder qualidade [5].

**Cache de Respostas** — Para perguntas frequentes, cacheie as respostas. Reduz custos de API drasticamente para queries repetitivas.

**Batching** — Agrupe múltiplas requests em um batch. GPUs são mais eficientes processando batches grandes requests isoladas.

## 3. Ilustra

### A Rota até a Produção

Cada caminho de deploy tem seu custo e complexidade. A escolha depende do volume de requests, budget de hardware e requisitos de latência [1].

```
%% legenda: Mapa de rotas para produção do DeepSeek
flowchart TD
 A[Necessidade de IA] --> B{Volume de requests?}
 B -->|Baixo/Médio
< 10k/dia| C[API Gerenciada
Custo por token]
 B -->|Alto
> 10k/dia| D{Budget de hardware?}
 D -->|Limitado| E[API + Cache
Cache de respostas]
 D -->|Disponível| F{Modelo cabe em 1 GPU?}
 F -->|Sim| G[Deploy Local
SGLang ou vLLM]
 F -->|Não| H{Múltiplas GPUs?}
 H -->|Sim| I[Multi-GPU
Tensor/Pipeline Parallelism]
 H -->|Não| J[Multi-Node
Cluster distribuído]

 C --> K[Custo: $0.07-1.10/1M tokens]
 E --> K2[Custo: $0.02-0.30/1M tokens]
 G --> K3[Custo: ~$1.50/h GPU]
```

```
I --> K4[Custo: ~$6-12/h cluster]
J --> K5[Custo: ~$24-48/h cluster]
```

## Comparativo de Custos

```
class AnaliseCustos:
 """Compara custos entre diferentes estratégias de deploy."""

 def custo_api(self, tokens_mes, modelo="flash"):
 """Custo com API gerenciada."""
 precos = {
 "flash": {"input": 0.07, "output": 0.27},
 "pro": {"input": 0.27, "output": 1.10}
 }
 p = precos[modelo]
 # Assumindo 70% input, 30% output
 custo = (tokens_mes * 0.7 / 1_000_000 * p["input"] +
 tokens_mes * 0.3 / 1_000_000 * p["output"])
 return custo

 def custo_api_cache(self, tokens_mes, taxa_cache=0.3):
 """Custo com API + cache (30% das queries são cacheadas)."""
 tokens_efetivos = tokens_mes * (1 - taxa_cache)
 return self.custo_api(tokens_efetivos, "flash")

 def custo_local(self, horas_mes, gpu_hora=1.50, n_gpus=1):
 """Custo com deploy local."""
 return horas_mes * gpu_hora * n_gpus

 def custo_distribuido(self, horas_mes, gpu_hora=1.50, n_gpus=8):
 """Custo com deploy distribuído."""
 return horas_mes * gpu_hora * n_gpus

 def comparar(self, tokens_mes):
 """Compara todas as estratégias."""
 print(f"Volume: {tokens_mes:,} tokens/mês\n")

 # API simples
 c1 = self.custo_api(tokens_mes, "flash")
 print(f"API Flash: ${c1:>8.2f}/mês")

 # API pro
 c2 = self.custo_api(tokens_mes, "pro")
 print(f"API Pro: ${c2:>8.2f}/mês")
```

```

API + cache
c3 = self.custo_api_cache(tokens_mes)
print(f"API Flash + Cache: ${c3:>8.2f}/mês")

Local 1 GPU (24h/dia)
c4 = self.custo_local(720, 1.50, 1)
print(f"Local 1xA100: ${c4:>8.2f}/mês")

Local 2 GPUs
c5 = self.custo_local(720, 1.50, 2)
print(f"Local 2xA100: ${c5:>8.2f}/mês")

Distribuído 8 GPUs
c6 = self.custo_distribuido(720, 1.50, 8)
print(f"Distribuído 8xA100: ${c6:>8.2f}/mês")

Ponto de equilíbrio
print(f"\nPonto de equilíbrio API vs Local:")
custo_api = self.custo_api(1_000_000, "flash")
custo_local = self.custo_local(720, 1.50, 2)
tokens_equilibrio = custo_local / (custo_api / 1_000_000)
print(f" Acima de {tokens_equilibrio:,.0f} tokens/mês → Local é
mais barato")

Análise
analise = AnaliseCustos()
analise.comparar(100_000) # Startup pequena
print()
analise.comparar(1_000_000) # Startup em crescimento
print()
analise.comparar(10_000_000) # Empresa grande

```

## 4. Técnica

---

### Deploy com SGLang (Recomendado)

```

Instale o SGLang
pip install sglang[all]

Inicie o servidor com DeepSeek-V3 (8 GPUs)
python3 -m sglang.launch_server \

```

```

--model deepseek-ai/DeepSeek-V3-Base \
--tp 8 \
--trust-remote-code \
--host 0.0.0.0 \
--port 30000

Para modelos menores (1-2 GPUs)
python3 -m sglang.launch_server \
--model deepseek-ai/DeepSeek-R1-Distill-Qwen-32B \
--tp 2 \
--trust-remote-code \
--host 0.0.0.0 \
--port 30000

Teste a API
curl http://localhost:30000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
 "model": "deepseek-ai/DeepSeek-V3-Base",
 "messages": [{"role": "user", "content": "Olá!"}]
}'

```

## Deploy com vLLM

```

Instale o vLLM
pip install vllm

Inicie o servidor com DeepSeek-V3
vllm serve deepseek-ai/DeepSeek-V3-Base \
--tensor-parallel-size 8 \
--max-model-len 32768 \
--enforce-eager

Para modelos distilados (mais leves)
vllm serve deepseek-ai/DeepSeek-R1-Distill-Qwen-32B \
--tensor-parallel-size 2 \
--max-model-len 32768 \
--enforce-eager

Com FP8 (reduz memória 50%)
vllm serve deepseek-ai/DeepSeek-V3-Base \
--tensor-parallel-size 8 \
--quantization fp8 \
--max-model-len 65536

```

## Docker Compose para Produção

```
version: '3.8'
services:
 deepseek-api:
 image: vllm/vllm-openai:latest
 command: >
 --model deepseek-ai/DeepSeek-V3-Base
 --tensor-parallel-size 8
 --max-model-len 32768
 --host 0.0.0.0
 --port 8000
 ports:
 - "8000:8000"
 deploy:
 resources:
 reservations:
 devices:
 - driver: nvidia
 count: 8
 capabilities: [gpu]
 volumes:
 - model-cache:/root/.cache/huggingface
 restart: unless-stopped
 healthcheck:
 test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
 interval: 30s
 timeout: 10s
 retries: 3

 nginx:
 image: nginx:alpine
 ports:
 - "80:80"
 volumes:
 - ./nginx.conf:/etc/nginx/nginx.conf
 depends_on:
 - deepseek-api

volumes:
 model-cache:
```

## Script de Monitoramento de Custo

```
import time
from dataclasses import dataclass, field
from typing import List

@dataclass
class MetricasCusto:
 """Monitora custos de uso da API DeepSeek."""
 tokens_entrada: int = 0
 tokens_saida: int = 0
 tempo_total: float = 0.0
 historico: List[dict] = field(default_factory=list)

 PRECOS = {
 "flash": {"input": 0.07, "output": 0.27},
 "pro": {"input": 0.27, "output": 1.10}
 }

 def registrar(self, modelo, tokens_in, tokens_out, latencia):
 """Registra uma chamada."""
 custo = self._calcular_custo(modelo, tokens_in, tokens_out)
 self.tokens_entrada += tokens_in
 self.tokens_saida += tokens_out
 self.tempo_total += latencia
 self.historico.append({
 "modelo": modelo,
 "tokens_in": tokens_in,
 "tokens_out": tokens_out,
 "latencia": latencia,
 "custo": custo,
 "timestamp": time.time()
 })
 return custo

 def _calcular_custo(self, modelo, tokens_in, tokens_out):
 p = self.PRECOS.get(modelo, {"input": 0, "output": 0})
 return (tokens_in / 1_000_000 * p["input"] +
 tokens_out / 1_000_000 * p["output"])

 def custo_total(self):
 return sum(h["custo"] for h in self.historico)

 def throughput(self):
 total = self.tokens_entrada + self.tokens_saida
 return total / self.tempo_total if self.tempo_total > 0 else 0
```

```

def relatorio(self):
 print("=== Relatório de Custos DeepSeek ===")
 print(f"Total de chamadas: {len(self.historico)}")
 print(f"Tokens entrada: {self.tokens_entrada:,}")
 print(f"Tokens saída: {self.tokens_saida:,}")
 print(f"Custo total: ${self.custo_total():.6f}")
 print(f"Throughput: {self.throughput():.1f} tok/s")
 print(f"Latência média: {self.tempo_total/
len(self.historico)*1000:.0f}ms")

 # Por modelo
 for modelo in set(h["modelo"] for h in self.historico):
 chamadas = [h for h in self.historico if h["modelo"] == modelo]
 custo = sum(h["custo"] for h in chamadas)
 print(f"\n {modelo}: {len(chamadas)} chamadas, ${custo:.6f}")

Uso
monitor = MetricasCustos()
... após cada chamada à API ...
monitor.registrar("flash", 500, 200, 1.5)
monitor.relatorio()

```

## Cache de Respostas

```

import hashlib
import json
from typing import Optional

class CacheRespostas:
 """Cache de respostas da API DeepSeek."""

 def __init__(self, ttl_horas=24):
 self.cache = {}
 self.ttl = ttl_horas * 3600

 def _chave(self, mensagens, modelo):
 """Gera chave de cache baseada no conteúdo."""
 conteudo = json.dumps(mensagens, sort_keys=True) + modelo
 return hashlib.sha256(conteudo.encode()).hexdigest()[:16]

 def buscar(self, mensagens, modelo) -> Optional[str]:
 """Busca no cache."""
 chave = self._chave(mensagens, modelo)
 if chave in self.cache:
 entrada = self.cache[chave]

```

```

 if time.time() - entrada["timestamp"] < self.ttl:
 return entrada["resposta"]
 else:
 del self.cache[chave]
 return None

def salvar(self, mensagens, modelo, resposta):
 """Salva no cache."""
 chave = self._chave(mensagens, modelo)
 self.cache[chave] = {
 "resposta": resposta,
 "timestamp": time.time()
 }

def hit_rate(self):
 """Taxa de acerto do cache."""
 if not self.cache:
 return 0
 hits = sum(1 for v in self.cache.values()
 if time.time() - v["timestamp"] < self.ttl)
 return hits / len(self.cache)

Uso
cache = CacheRespostas(ttl_horas=24)

def responder_com_cache(cliente, mensagens, modelo):
 """Responde com cache para reduzir custos."""
 # Busca no cache
 resposta_cache = cache.buscar(mensagens, modelo)
 if resposta_cache:
 print("Cache hit!")
 return resposta_cache

 # Chama a API
 inicio = time.time()
 resposta = cliente.chat.completions.create(
 model=modelo,
 messages=mensagens
)
 latencia = time.time() - inicio

 # Salva no cache
 conteudo = resposta.choices[0].message.content
 cache.salvar(mensagens, modelo, conteudo)

 print(f"Cache miss ({latencia:.1f}s)")
 return conteudo

```



## 5. Aplica

---

### A Decisão de Deploy

Sua startup tem 10.000 requests/dia para um chatbot. O CEO quer reduzir custos. Hoje você usa a API da OpenAI a \$0.03/1K tokens. A pergunta é: vale a pena migrar para DeepSeek?

Com DeepSeek-V4-Flash (\$0.07/1M input, \$0.27/1M output), o custo por request média (500 tokens input, 200 tokens output) é ~\$0.00009. Para 10.000 requests/dia: ~\$0.90/dia vs. ~\$1.95/dia com OpenAI. Economia de ~54% [1].

Mas se o volume crescer para 100.000 requests/dia, o custo com API chega a ~\$9/dia (~\$270/mês). Nesse ponto, deploy local com SGLang em 2 GPUs A100 pode ser mais econômico (~\$1.50/hora de GPU = ~\$1.080/mês, mas com capacidade ilimitada).

A regra é: - < **50k requests/dia**: API gerenciada - **50k-500k requests/dia**: API + cache de respostas - **500k-5M requests/dia**: Deploy local 1-2 GPUs - > **5M requests/dia**: Deploy distribuído 4-8 GPUs

### Checklist de Produção

- ☐ Modelo selecionado (flash para velocidade, pro para qualidade)
- ☐ API key configurada e testada
- ☐ Rate limiting implementado
- ☐ Retry com backoff exponencial
- ☐ Logging de todas as chamadas
- ☐ Monitoramento de custos ativo
- ☐ Cache de respostas implementado (se aplicável)
- ☐ Fallback para outro modelo se falhar
- ☐ Health check configurado
- ☐ Alertas de custo configurados

### Exercício

- ☐ Calcule o custo mensal da API DeepSeek para seu volume de requests
- ☐ Compare com o custo de deploy local (2x A100 vs. API)
- ☐ Configure um servidor SGLang com DeepSeek-V4-Flash
- ☐ Implemente o script de monitoramento de custo
- ☐ Implemente cache de respostas para queries frequentes
- ☐ Teste o streaming e meça a latência perceptiva

- ☐ Monte o checklist de produção para seu projeto

## 6. Conclusão

---

Neste capítulo final, você juntou tudo o que aprendeu ao longo deste livro. Viu que o DeepSeek oferece três caminhos para produção — API gerenciada, deploy local e deploy distribuído — cada um com seus trade-offs. Os frameworks SGLang, vLLM e LMDeploy tornam o deploy direto, enquanto as otimizações de FP8, KV cache e speculative decoding reduzem custo e latência [1][2].

A chave é escolher o caminho certo para o seu volume e budget. Para a maioria das startups, a API gerenciada é suficiente — o custo por token é baixo e não há overhead de infraestrutura. Para empresas maiores, deploy local pode ser mais econômico. Para volumes massivos, deploy distribuído é o único caminho [1].

Ao longo destes oito capítulos, você percorreu o caminho completo: do mapa do ecossistema à API, das integrações aos prompts, do Harness ao plugins, das otimizações ao deploy. Você não é mais um usuário casual do DeepSeek — você é um Engenheiro de IA que domina cada estação do laboratório. Agora é hora de construir.

## 7. Referências Bibliográficas

---

[1] DEEPSEEK-AI. *DeepSeek API Documentation*. Disponível em: <https://api-docs.deepseek.com/>. Acesso em: 21 ago. 2026.

[2] DEEPSEEK-AI. *DeepSeek-V3 Technical Report*. Disponível em: <https://arxiv.org/abs/2412.19437>. Acesso em: 21 ago. 2026.

[3] DEEPSEEK-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Disponível em: <https://arxiv.org/abs/2501.12948>. Acesso em: 21 ago. 2026.

[4] LI, Jiashi; LIU, Shengyu. *FlashMLA: Efficient Multi-head Latent Attention Kernels*. Disponível em: <https://github.com/deepseek-ai/FlashMLA>. Acesso em: 21 ago. 2026.

[5] DEEPSEEK-AI. *DeepSpec: Full-Stack Codebase for Speculative Decoding*. Disponível em: <https://github.com/deepseek-ai/DeepSpec>. Acesso em: 21 ago. 2026.

[6] DEEPSEEK-AI. *DeepEP: An Efficient Expert-Parallel Communication Library*. Disponível em: <https://github.com/deepseek-ai/DeepEP>. Acesso em: 21 ago. 2026.

- [7] DEEPSEEK-AI. *DeepGEMM: Clean and Efficient BLAS Kernel Library on GPU*. Disponível em: <https://github.com/deepseek-ai/DeepGEMM>. Acesso em: 21 ago. 2026.
- [8] DEEPSEEK-AI. *3FS: A High-Performance Distributed File System for AI*. Disponível em: <https://github.com/deepseek-ai/3FS>. Acesso em: 21 ago. 2026.
- [9] DEEPSEEK-AI. *DeepSeek Harness: Everything is a Plugin*. Disponível em: <https://github.com/deepseek-ai/deepseek-harness>. Acesso em: 21 ago. 2026.
- [10] CORDIVERSE. *Cordis: A Meta-Framework of Spatiotemporal Composability*. Disponível em: <https://github.com/cordiverse/cordis>. Acesso em: 21 ago. 2026.
- [11] DEEPSEEK-AI. *Awesome DeepSeek Agent*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-agent>. Acesso em: 21 ago. 2026.
- [12] DEEPSEEK-AI. *Awesome DeepSeek Integration*. Disponível em: <https://github.com/deepseek-ai/awesome-deepseek-integration>. Acesso em: 21 ago. 2026.
- [13] DEEPSEEK-AI. *TileKernels: Kernel Library Written in Tilelang*. Disponível em: <https://github.com/deepseek-ai/TileKernels>. Acesso em: 21 ago. 2026.
- [14] DEEPSEEK-AI. *DeepSeek-OCR-2: Visual Causal Flow*. Disponível em: <https://github.com/deepseek-ai/DeepSeek-OCR-2>. Acesso em: 21 ago. 2026.
- [15] VASWANI, A. et al. *Attention Is All You Need*. In: Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [16] DEEPSEEK-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. Disponível em: <https://arxiv.org/abs/2405.04434>. Acesso em: 21 ago. 2026.
- [17] FEDUS, W. et al. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. In: Journal of Machine Learning Research, 2022.
- [18] DEEPSEEK-AI. *DeepSeek Platform — Pricing*. Disponível em: <https://platform.deepseek.com/>. Acesso em: 21 ago. 2026.
- [19] SGLANG. *SGLang Documentation*. Disponível em: <https://github.com/sgl-project/sglang>. Acesso em: 21 ago. 2026.
- [20] VLLM. *vLLM Documentation*. Disponível em: <https://github.com/vllm-project/vllm>. Acesso em: 21 ago. 2026.

## 8. Apêndice — Casos de Uso Reais

---

### Caso 1: Chatbot de Atendimento ao Cliente

**Cenário:** E-commerce com 50.000 produtos, 10.000 clientes ativos. **Solução:** API DeepSeek-V4-Flash com cache de respostas. **Resultado:** - Tempo de resposta: 2s (vs. 4h com atendente humano) - Custo: \$0.002 por interação (vs. \$15 com atendente) - Disponibilidade: 24/7 (vs. 8h/dia) - Satisfação: 89% (vs. 72%)

```
Implementação simplificada
cache = CacheInteligente(ttl_horas=48)

async def atender_cliente(pergunta):
 # Busca no cache
 resposta = cache.get(pergunta)
 if resposta:
 return resposta

 # Chama API
 resposta = await cliente.chat_simples(
 f"""\Voc\u00ea \u00e9 um assistente de atendimento da loja X.

Pergunta do cliente: {pergunta}

Responda de forma amig\u00e1vel e precisa. Se n\u00e3o souber, diga que vai
encaminhar para um atendente humano.""")
)

 # Salva no cache
 cache.set(pergunta, resposta)
 return resposta
```

### Caso 2: Code Review Automatizado

**Cen\u00e1rio:** Time de 20 devs, 50 PRs por semana. **Solu\u00e7\u00e3o:** DeepSeek-V4-Pro com thinking mode + MCP GitHub. **Resultado:** - PRs revisados: 100% (vs. 60% com review manual) - Bugs encontrados: 3x mais (vs. review manual) - Tempo de review: 5min (vs. 2h manual) - Custo: \$0.50 por PR (vs. \$50 com dev s\u00eanior)

### Caso 3: Geração de Documentação

**Cenário:** Projeto com 200K linhas de código, sem documentação. **Solução:** DeepSeek-V4-Flash para gerar docs automáticas. **Resultado:** - Cobertura: 85% do código documentado - Tempo: 2 dias (vs. 6 meses manual) - Custo: ~\$200 total (vs. \$30.000 com tech writer)

### Caso 4: Análise de Sentimento em Tempo Real

**Cenário:** Monitoramento de redes sociais para marca com 1M de seguidores. **Solução:** DeepSeek-V4-Flash com streaming + cache. **Resultado:** - Menções analisadas: 10.000/hora - Latência: <500ms por análise - Custo: \$0.50/dia - Alertas negativos: em tempo real

```
async def analisar_sentimento(texto):
 """Análise de sentimento com classificação."""
 resultado = await cliente.chat_simples(
 f"""Classifique o sentimento do texto abaixo:

 Texto: {texto}

 Responda em JSON:
 {{
 "sentimento": "positivo|neutro|negativo",
 "confianca": 0.0-1.0,
 "temas": ["tema1", "tema2"]}
 """
)
 return json.loads(resultado)

Monitoramento em tempo real
async def monitorar_marca(marca):
 """Monitora menções da marca em tempo real."""
 async for mencao in stream_mencoes(marca):
 sentimento = await analisar_sentimento(mencao["texto"])

 if sentimento["sentimento"] == "negativo" and
 sentimento["confianca"] > 0.8:
 await alertar_equipe(mencao, sentimento)
```

## Conclusão Geral

---

Sintetizar a jornada do leitor de iniciante a especialista, conectando todos os módulos do laboratório em uma visão holística e projetando os próximos passos de carreira em IA.

